Spring Boot Complete Guide with Hands-On Code Examples

1. Introduction to Spring Boot

1.1 What is Spring Boot?

Spring Boot is an open-source, Java-based framework used to create stand-alone, production-ready Spring applications with minimal configuration. Built on top of the Spring Framework, it simplifies bootstrapping and development by offering default settings and reducing boilerplate code.

1.2 Features of Spring Boot

- Auto-Configuration: Automatically configures your application based on the dependencies declared in the project.
- Standalone Applications: Enables creating applications that can run independently using embedded servers like Tomcat or Jetty.
- Production-Ready: Includes built-in features like metrics, health checks, and external configuration via Spring Boot Actuator.
- Opinionated Defaults: Reduces the need for manual configurations with pre-defined, best-practice defaults.
- No XML Configuration: Encourages convention over configuration using annotations instead of XML files.

1.3 Advantages of Using Spring Boot

- Rapid Application Development
- Auto-Configuration
- Embedded Web Servers (Tomcat, Jetty, Undertow)
- Run applications using: `java -jar app.jar`
- Microservices-Friendly
- Production-Ready Features: Health checks, metrics, externalized config via Actuator
- Reduced Boilerplate Code
- Easy Dependency Management
- Seamless Integration with Spring Ecosystem
- Compatible with Spring Data JPA, Spring Security, Spring WebFlux, Spring Batch
- Flexible Configuration (supports `application.properties` and `application.yml`)
- Community and Ecosystem Support

1.4 Use Cases

Spring Boot is used in a wide variety of domains:

- RESTful API development
- Microservices architecture
- Rapid prototyping
- Enterprise web applications
- Cloud-native applications

Real Life Examples:

- E-Commerce Platforms
- Banking & Financial Services
- Healthcare Systems
- Travel and Booking Platforms

- Learning Management Systems (LMS)
- Logistics & Supply Chain Systems
- Content Management Systems (CMS)
- HR & Payroll Systems
- IoT Backend Services
- Microservices in Large Enterprises

Language Support:

- Java (Primary Language)
- Kotlin
- Groovy
- Scala (Limited Support)

1.5 Spring Boot vs Spring Framework

Feature / Aspect	Spring Framework	Spring Boot
Setup & Configuration	Manual XML/ Java-based configuration	Auto-configuration, zero XML
Application Startup	Requires external servlet container	Comes with embedded servers (Tomcat, Jetty, Undertow)
Dependency Management	Developer-managed dependencies	Pre-configured starter dependencies
Boilerplate Code	More boilerplate and configuration	Minimal configuration and boilerplate
Project Structure	Developer decides from scratch	Convention over configuration
Production Readiness	Requires manual setup of monitoring	Built-in Actuator endpoints for health,

	tools	metrics, etc.
Build Output	WAR (typically)	Executable JAR by default
Learning Curve	Steeper for beginners	Easier and quicker to get started
CLI Support	No	Yes, via Spring Boot CLI
Microservices Development	Needs custom setup	Tailored for microservices architecture

1.6 Spring Boot Architecture

Spring Boot architecture is based on:

- Spring Core: Base dependency injection and AOP framework
- Auto Configuration: Automatically configures beans based on classpath
- Starter Dependencies: Predefined sets of dependencies for various features (web, data, etc.)
- Spring Boot CLI: Command-line tool to run and test applications
- Embedded Server: Tomcat, Jetty, or Undertow
- Spring Boot Actuator: Provides operational endpoints for monitoring and management

---

2. Getting Started

2.1 Setting up Spring Boot Project (Spring Initializr, Maven, Gradle)

Visit https://start.spring.io/ to generate your project with the following dependencies:

- Spring Web
- Spring Data JPA
- H2 Database
- Spring Boot DevTools
- Spring Boot Actuator

2.2 Project Structure Overview

text

CopyDownload

```
springboot-coding-structure/
    .mvn/
    .settings/
```

```
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── net/guides/springboot2/
│   │   │       ├── controller/
│   │   │       │   ├── CustomerController.java
│   │   │       │   └── OrderController.java
│   │   │       ├── dao/
│   │   │       │   ├── CustomerRepository.java
│   │   │       │   └── OrderRepository.java
│   │   │       ├── model/
│   │   │       │   ├── Customer.java
│   │   │       │   └── Order.java
│   │   │       ├── service/
│   │   │       │   ├── CustomerService.java
│   │   │       │   └── OrderService.java
│   │   │       └── Application.java
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── static/
│   │       └── templates/
│   └── test/
├── target/
│   └── .gitignore
├── pom.xml
└── README.md
```

2.3 Application Entry Point (@SpringBootApplication)

Hands-On Code:

java

CopyDownload

```
/**
 * Main application entry point for Spring Boot application.
 * The @SpringBootApplication annotation combines three annotations:
 * @Configuration + @EnableAutoConfiguration + @ComponentScan
 */
package net.guides.springboot2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;
import org.springframework.cache.annotation.EnableCaching;

@SpringBootApplication // Marks this as the main Spring Boot application class
@EnableScheduling // Enables scheduled task execution
@EnableCaching // Enables caching functionality
public class Application {

    /**
     * The main method serves as the entry point for the application.
     * SpringApplication.run() bootstraps the application and starts the embedded server.
     *
     * @param args Command line arguments passed to the application
```

```
    */
    public static void main(String[] args) {
        // Launch the Spring Boot application
        SpringApplication.run(Application.class, args);
        System.out.println("Application started successfully!");
    }
}
```

2.4 Running the Application

Hands-On Code:

bash

CopyDownload

```
# Build the application using Maven
mvn clean package

# Run the JAR file
java -jar target/myapp-0.0.1-SNAPSHOT.jar

# Or run with custom port
java -jar target/myapp-0.0.1-SNAPSHOT.jar --server.port=9090

# Or using Spring Boot Maven plugin during development
mvn spring-boot:run
```

---

3. Core Concepts

3.1 Spring Boot Annotations

1. @SpringBootApplication

java

CopyDownload

```
/**
 * @SpringBootApplication is a convenience annotation that combines:
 * - @Configuration: Marks class as source of bean definitions
 * - @EnableAutoConfiguration: Enables Spring Boot's auto-configuration
 * - @ComponentScan: Enables component scanning for @Component, @Service, @Repository,
@Controller
 */
package net.guides.springboot2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Application {

    public static void main(String[] args) {
        // Bootstrap the Spring application context
        SpringApplication.run(Application.class, args);
    }
}
```

2. @Component, @Service, @Repository, @Controller, @RestController

Hands-On Code:

java

CopyDownload

```
// ---- @Component Example ----
package net.guides.springboot2.component;

import org.springframework.stereotype.Component;

/**
 * @Component is a generic stereotype annotation that marks a class as a Spring-managed component.
 * Used when the class doesn't clearly fall into service, repository, or controller layers.
 */
@Component
public class MyComponent {

    public String greet() {
        return "Hello from generic component!";
    }
}

// ---- @Service Example ----
package net.guides.springboot2.service;

import org.springframework.stereotype.Service;

/**
 * @Service is a specialization of @Component used to annotate service layer classes.
 * These classes contain business logic and are often the core of your application.
 */
@Service
public class OrderService {

    private final PaymentService paymentService;

    // Constructor injection (recommended for mandatory dependencies)
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }

    /**
     * Processes an order by validating inventory, applying discounts,
     * and delegating payment processing.
     */
    public Order processOrder(Order order) {
        // Validate order items
        if (order.getItems().isEmpty()) {
            throw new IllegalArgumentException("Order cannot be empty");
        }
```

```
        // Calculate total and apply discounts
        double total = calculateTotal(order);
        double discount = applyDiscounts(order);
        order.setTotalAmount(total - discount);

        // Process payment
        paymentService.processPayment(order.getTotalAmount());

        // Update order status
        order.setStatus(OrderStatus.COMPLETED);
        return order;
    }

    private double calculateTotal(Order order) {
        return order.getItems().stream()
                .mapToDouble(Item::getPrice)
                .sum();
    }

    private double applyDiscounts(Order order) {
        // Business logic for discounts
        return 0.0; // Placeholder
    }
}

// ---- @Repository Example ----
package net.guides.springboot2.repository;

import org.springframework.stereotype.Repository;

/**
 * @Repository is a specialization of @Component used for DAO classes.
 * It automatically translates database exceptions to Spring's DataAccessException.
 */
@Repository
public class ProductRepository {

    private final List<Product> products = new ArrayList<>();

    /**
     * Saves a product to the in-memory database.
     *
     * @param product The product to save
     * @return The saved product with generated ID
     */
    public Product save(Product product) {
        product.setId(UUID.randomUUID().toString());
        products.add(product);
        return product;
    }

    /**
     * Finds a product by its ID.
     *
```

```
     * @param id The product ID
     * @return The found product or null if not found
     */
    public Product findById(String id) {
        return products.stream()
                .filter(p -> p.getId().equals(id))
                .findFirst()
                .orElse(null);
    }

    /**
     * Finds all products.
     *
     * @return List of all products
     */
    public List<Product> findAll() {
        return new ArrayList<>(products);
    }

    /**
     * Updates an existing product.
     *
     * @param product The product with updated information
     * @return The updated product
     */
    public Product update(Product product) {
        Product existing = findById(product.getId());
        if (existing != null) {
            products.remove(existing);
            products.add(product);
            return product;
        }
        throw new RuntimeException("Product not found with id: " + product.getId());
    }
}

// ---- @Controller Example (Traditional MVC) ----
package net.guides.springboot2.controller;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

/**
 * @Controller is used for traditional web applications with view rendering.
 * Returns view names that get resolved to HTML templates.
 */
@Controller
public class HomeController {

    @GetMapping("/")
    public String index(Model model) {
        // Add attributes to the model that will be accessible in the view
        model.addAttribute("message", "Welcome to Spring Boot!");
```

```
        model.addAttribute("timestamp", LocalDateTime.now());

        // Returns the name of the view to render (resolved to templates/index.html)
        return "index";
    }
}

// ---- @RestController Example (REST API) ----
package net.guides.springboot2.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

/**
 * @RestController = @Controller + @ResponseBody
 * Used to create RESTful web services where methods return data directly,
 * not view names. The returned objects are automatically serialized to JSON/XML.
 */
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;

    // Constructor injection for required dependency
    public UserController(UserService userService) {
        this.userService = userService;
    }

    /**
     * Handles GET requests to /api/users
     * Returns all users as JSON.
     */
    @GetMapping
    public List<User> getAllUsers() {
        // Return data directly - Spring will serialize to JSON
        return userService.findAll();
    }

    /**
     * Handles GET requests to /api/users/{id}
     * Returns a single user as JSON.
     */
    @GetMapping("/{id}")
    public ResponseEntity<User> getUserById(@PathVariable Long id) {
        return userService.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }
}
```

Summary Table:

Annotatio	Layer	Inherits	Purpose	Returns

n		From		Response Body
@Component	Generic	-	General-purpose component	No
@Service	Service Layer	@Component	Business logic and services	No
@Repository	Data Layer	@Component	DAO & exception translation	No
@Controller	Web Layer	@Component	MVC controller for views	No (view name)
@RestController	Web Layer	@Controller + @ResponseBody	REST API endpoints	Yes (JSON/XML)

3. @Autowired and @Qualifier

Hands-On Code:

java

CopyDownload

```
// ---- @Autowired Example ----
package net.guides.springboot2.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

/**
 * @Autowired automatically injects a bean from the Spring container by type.
 * It can be used on fields, constructors, and setter methods.
 */
@Service
public class CarService {

    // FIELD INJECTION (Quick but less recommended for testing)
    @Autowired
    private Engine engine;

    // CONSTRUCTOR INJECTION (Recommended for mandatory dependencies)
    private final Transmission transmission;

    @Autowired
    public CarService(Transmission transmission) {
        // Constructor injection ensures the dependency is not null
```

```
        // and makes the class immutable
        this.transmission = transmission;
    }

    // SETTER INJECTION (Recommended for optional dependencies)
    private Wheels wheels;

    @Autowired
    public void setWheels(Wheels wheels) {
        this.wheels = wheels;
    }

    public void start() {
        engine.start();
        transmission.shift(1);
        System.out.println("Car started successfully!");
    }
}

// ---- @Qualifier Example ----
package net.guides.springboot2.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.stereotype.Service;

/**
 * @Qualifier is used with @Autowired to specify which bean to inject
 * when multiple candidates of the same type are available.
 */
// Define multiple implementations of the same interface
interface PaymentGateway {
    void processPayment(double amount);
}

@Component("paypalGateway")
class PayPalGateway implements PaymentGateway {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via PayPal");
    }
}

@Component("stripeGateway")
class StripeGateway implements PaymentGateway {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via Stripe");
    }
}

@Service
public class CheckoutService {
```

```
    // Use @Qualifier to specify which bean to inject
    @Autowired
    @Qualifier("stripeGateway")
    private PaymentGateway paymentGateway;

    public void checkout(double total) {
        // This will use the Stripe gateway implementation
        paymentGateway.processPayment(total);
    }
}

// Alternative: Use @Primary to set a default bean
@Component
@Primary // This makes PayPal the default when no @Qualifier is specified
class DefaultPayPalGateway implements PaymentGateway {
    @Override
    public void processPayment(double amount) {
        System.out.println("Processing $" + amount + " via PayPal (default)");
    }
}
```

4. @Configuration and @Bean

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;
import org.springframework.web.client.RestTemplate;

/**
 * @Configuration indicates that this class contains bean definitions
 * for the Spring IoC container. It serves as a replacement for XML configuration.
 */
@Configuration
public class AppConfig {

    /**
     * @Bean is a method-level annotation used inside a @Configuration class
     * to define a bean explicitly.
     */
    @Bean
    public DataSource dataSource() {
        // Create and configure a DataSource bean
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:mysql://localhost:3306/mydb");
        config.setUsername("root");
        config.setPassword("password");
        config.setMaximumPoolSize(10);
        config.setMinimumIdle(5);
        config.setIdleTimeout(30000);
        return new HikariDataSource(config);
```

```
    }

    @Bean
    public RestTemplate restTemplate() {
        // Create a RestTemplate bean for making HTTP requests
        return new RestTemplate();
    }

    @Bean
    public ObjectMapper objectMapper() {
        // Create a custom ObjectMapper for JSON serialization
        ObjectMapper mapper = new ObjectMapper();
        mapper.setSerializationInclusion(JsonInclude.Include.NON_NULL);
        mapper.setDateFormat(new SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ss.SSSZ"));
        return mapper;
    }

    @Bean
    @Profile("dev") // This bean will only be created when 'dev' profile is active
    public String devEnvironmentBean() {
        return "Development Environment Configuration";
    }

    @Bean
    @Profile("prod") // This bean will only be created when 'prod' profile is active
    public String prodEnvironmentBean() {
        return "Production Environment Configuration";
    }
}

// Usage of @ConfigurationProperties with @Bean
package net.guides.springboot2.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.jdbc.DataSourceBuilder;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

import javax.sql.DataSource;

@Configuration
public class DatabaseConfig {

    /**
     * Creates a DataSource bean using properties from application.yml
     * with prefix "app.datasource".
     */
    @Bean
    @ConfigurationProperties(prefix = "app.datasource")
    public DataSource dataSource() {
        // DataSourceBuilder will automatically bind properties
        // from application.yml with the given prefix
        return DataSourceBuilder.create().build();
    }
```

}

3.2 Dependency Injection (DI)

Dependency Injection is a programming technique where an object receives its dependencies from an external source rather than creating them internally.

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.di;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

/**
 * Example showing different types of Dependency Injection
 * and why it's beneficial.
 */

// --- INTERFACE FOR LOOSE COUPLING ---
interface EmailSender {
    void sendEmail(String to, String subject, String body);
}

// --- CONCRETE IMPLEMENTATION ---
@Component
class SmtpEmailSender implements EmailSender {

    @Value("${email.smtp.host}") // Inject property value
    private String smtpHost;

    @Value("${email.smtp.port}")
    private int smtpPort;

    @Override
    public void sendEmail(String to, String subject, String body) {
        System.out.println("Sending email via SMTP server " + smtpHost + ":" + smtpPort);
        System.out.println("To: " + to);
        System.out.println("Subject: " + subject);
        System.out.println("Body: " + body);
    }
}

// --- SERVICE USING CONSTRUCTOR INJECTION (RECOMMENDED) ---
@Service
public class NotificationService {

    private final EmailSender emailSender; // Final for immutability
    private final String defaultFromEmail;

    /**
     * Constructor injection - the preferred way to inject dependencies.
     * Benefits:
```

```
     * 1. Dependencies are clearly defined and mandatory
     * 2. Class is immutable (final fields)
     * 3. Easy to unit test (just pass mocks to constructor)
     * 4. Null safety - dependencies must be provided
     */
    @Autowired
    public NotificationService(
            EmailSender emailSender,
            @Value("${email.default.from}") String defaultFromEmail) {
        this.emailSender = emailSender;
        this.defaultFromEmail = defaultFromEmail;
    }

    public void sendWelcomeEmail(String to, String userName) {
        String subject = "Welcome to our application!";
        String body = "Hello " + userName + ",\n\n" +
                "Welcome to our platform. We're excited to have you on board!\n" +
                "Best regards,\n" +
                "The Team";
        emailSender.sendEmail(to, subject, body);
    }
}

// --- SERVICE WITH SETTER INJECTION (FOR OPTIONAL DEPENDENCIES) ---
@Service
public class ReportService {

    private EmailSender emailSender; // Not final
    private boolean sendReports = false;

    // Setter injection - for optional dependencies
    @Autowired(required = false) // Not required, so can be null
    public void setEmailSender(EmailSender emailSender) {
        this.emailSender = emailSender;
        this.sendReports = true;
        System.out.println("Email sender configured for reports");
    }

    public void generateReport(Report report) {
        // Process report...

        // Only send email if dependency was provided
        if (sendReports && emailSender != null) {
            emailSender.sendEmail(
                "admin@example.com",
                "Daily Report",
                report.toString()
            );
        }
    }
}

// --- FIELD INJECTION (NOT RECOMMENDED FOR TESTING) ---
@Service
```

```
public class OrderService {

    // Field injection - convenient but has drawbacks:
    // 1. Hard to test (need reflection or Spring context)
    // 2. Violates Single Responsibility Principle
    // 3. Dependencies are hidden from the class API
    @Autowired
    private PaymentService paymentService;

    @Autowired
    private InventoryService inventoryService;

    @Autowired
    private EmailService emailService;

    public void placeOrder(Order order) {
        inventoryService.reserveItems(order.getItems());
        paymentService.charge(order.getTotal());
        emailService.sendOrderConfirmation(order);
    }
}

// --- COMPARISON: DI vs MANUAL INSTANTIATION ---
// Without DI (Tightly coupled, hard to test)
class WithoutDIController {
    private UserService userService;

    public WithoutDIController() {
        // UserService is hard-coded, cannot be changed or mocked
        this.userService = new UserService(new UserRepository());
    }
}

// With DI (Loosely coupled, easy to test)
@RestController
class WithDIController {
    private final UserService userService;

    // UserService is injected, can be replaced with a mock in tests
    @Autowired
    public WithDIController(UserService userService) {
        this.userService = userService;
    }
}

// Test class using DI
class UserServiceTest {
    private UserService userService;
    private UserRepository mockRepository;

    @BeforeEach
    void setUp() {
        mockRepository = mock(UserRepository.class);
        // Inject the mock dependency
```

```
        userService = new UserService(mockRepository);
    }

    @Test
    void testGetUser() {
        when(mockRepository.findById(1L)).thenReturn(Optional.of(new User(1L, "John")));
        User user = userService.getUser(1L);
        assertEquals("John", user.getName());
    }
}
```

3.3 Inversion of Control (IoC)

Hands-On Code:

java

CopyDownload

```
/**
 * Inversion of Control (IoC) is a design principle where the control of object
 * creation and dependency resolution is transferred from the program to a framework/container.
 */
package net.guides.springboot2.ioc;

import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.AnnotationConfigApplicationContext;

// --- WITHOUT IOC (Traditional approach) ---
class WithoutIoCExample {

    // The developer creates and manages all objects explicitly
    public void run() {
        // Hard-coded dependencies - tightly coupled
        UserRepository userRepo = new MySQLUserRepository();
        EmailService emailService = new SMTPEmailService();
        UserService userService = new UserService(userRepo, emailService);

        // Every change requires code modification and recompilation
        User user = userService.registerUser("john@example.com", "password123");
        System.out.println("User registered: " + user);
    }
}

// --- WITH IOC (Spring container manages objects) ---
@Configuration
class IoCAppConfig {

    @Bean
    public UserRepository userRepository() {
        // Spring container will manage this bean's lifecycle
        return new PostgreSQLUserRepository(); // Change implementation here only
    }

    @Bean
    public EmailService emailService() {
        return new SendGridEmailService();
```

```
    }

    @Bean
    public UserService userService() {
        // Spring will handle dependency injection
        return new UserService(userRepository(), emailService());
    }
}

@RestController
class UserRegistrationController {

    @Autowired
    private UserService userService; // Spring injects the dependency

    @PostMapping("/register")
    public User register(@RequestBody RegisterRequest request) {
        // We don't know where the UserService comes from
        // Spring handles the wiring based on configuration
        return userService.registerUser(request.getEmail(), request.getPassword());
    }
}

// --- Spring IoC Container Workflow ---
/**
 * How Spring IoC Container Works:
 *
 * 1. Application starts
 * 2. Spring scans the classpath for @Component, @Service, @Repository, @Controller
 * 3. Spring finds beans and their dependencies
 * 4. Spring creates instances of beans (by calling constructors or @Bean methods)
 * 5. Spring injects dependencies (@Autowired)
 * 6. Spring manages bean lifecycle (init, destroy, scopes)
 * 7. Application runs with fully wired components
 * 8. On shutdown, Spring destroys beans gracefully
 */

@SpringBootApplication
public class IoCDemoApplication {

    public static void main(String[] args) {
        // This line does ALL of the above:
        ApplicationContext context = SpringApplication.run(IoCDemoApplication.class, args);

        // We can retrieve beans from the container
        UserService userService = context.getBean(UserService.class);
        OrderService orderService = context.getBean(OrderService.class);

        // All dependencies are already wired by Spring
        System.out.println("All beans are managed by the Spring IoC container");
    }
}

// --- DEMONSTRATING BEAN SCOPES ---
```

```
package net.guides.springboot2.scope;

import org.springframework.context.annotation.Scope;
import org.springframework.stereotype.Component;

/**
 * Bean scopes determine how many instances of a bean Spring creates.
 */
// Singleton (default) - One instance per container
@Component
class SingletonBean {
    private int counter = 0;

    public void increment() {
        System.out.println("Singleton count: " + ++counter);
    }
}

// Prototype - New instance each time requested
@Component
@Scope("prototype")
class PrototypeBean {
    private int counter = 0;

    public void increment() {
        System.out.println("Prototype count: " + ++counter);
    }
}

// Request scope - One instance per HTTP request (web applications)
@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)
class RequestScopedBean {
    private final String requestId = UUID.randomUUID().toString();

    public String getRequestId() {
        return requestId;
    }
}

// Session scope - One instance per user session (web applications)
@Component
@Scope(value = "session", proxyMode = ScopedProxyMode.TARGET_CLASS)
class SessionScopedBean {
    private String userId;

    public void setUserId(String userId) {
        this.userId = userId;
    }

    public String getUserId() {
        return userId;
    }
}
```

```
@RestController
@RequestMapping("/demo")
class ScopeDemoController {

    @Autowired
    private SingletonBean singletonBean;

    @Autowired
    private PrototypeBean prototypeBean;

    @GetMapping("/scope-test")
    public String testScopes() {
        // Singleton beans share the same instance across requests
        singletonBean.increment(); // Counter increases across calls

        // Prototype beans get a new instance for each injection point
        prototypeBean.increment(); // Counter resets to 1 each time

        return "Check console logs for scope behavior";
    }
}
```

---

4. Configuration

4.1 application.properties vs application.yml

Hands-On Code:

properties

CopyDownload

```
# application.properties (Key-Value pairs)
# ----------------------------------------------
# Server Configuration
server.port=8081
server.servlet.context-path=/myapp

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true

# Logging Configuration
logging.level.root=INFO
logging.level.com.myapp=DEBUG
logging.file.name=logs/app.log
logging.pattern.console=%d{yyyy-MM-dd HH:mm:ss} - %msg%n
```

```
logging.pattern.file=%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n

# Actuator Configuration
management.endpoints.web.exposure.include=health,info,metrics,env
management.info.env.enabled=true
management.metrics.export.simple.enabled=true

# Custom Application Properties
app.name=MySpringBootApp
app.version=1.0.0
app.description=Spring Boot Application with Properties
app.admin.email=admin@example.com
app.feature.enabled=true
app.batch.size=100

# Internationalization
spring.messages.basename=i18n/messages
spring.messages.encoding=UTF-8
spring.messages.cache-duration=3600

# File Upload
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB
```

Hands-On Code:

yaml

CopyDownload

```
# application.yml (Hierarchical, indentation-based)
# ----------------------------------------------
server:
  port: 8081
  servlet:
    context-path: /myapp

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC
    username: root
    password: secret
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 5
      idle-timeout: 30000
      pool-name: HikariCPPool

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true
```

```
  servlet:
    multipart:
      max-file-size: 10MB
      max-request-size: 10MB

  messages:
    basename: i18n/messages
    encoding: UTF-8
    cache-duration: 3600

logging:
  level:
    root: INFO
    com.myapp: DEBUG
  file:
    name: logs/app.log
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss} - %msg%n"
    file: "%d{yyyy-MM-dd HH:mm:ss} [%thread] %-5level %logger{36} - %msg%n"

management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,env
  info:
    env:
      enabled: true
  metrics:
    export:
      simple:
        enabled: true

app:
  name: MySpringBootApp
  version: 1.0.0
  description: Spring Boot Application with YAML
  admin:
    email: admin@example.com
  feature:
    enabled: true
  batch:
    size: 100
```

4.2 External Configuration

Hands-On Code:

bash

CopyDownload

```
# External Configuration Sources (Order of precedence - highest to lowest):
# 1. Command line arguments
# 2. OS environment variables
# 3. application-{profile}.properties/yml
```

```
# 4. application.properties/yml
# 5. @PropertySource annotations
# 6. Default properties (Spring Boot defaults)

# --- Using Environment Variables ---
# Spring automatically converts env vars to property keys
# Format: SPRING_APPLICATION_JSON='{"server":{"port":8080}}'
# Or individual properties: export SPRING_DATASOURCE_URL=...

# Setting environment variables in Unix/Linux/macOS:
export SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/prod_db
export SPRING_DATASOURCE_USERNAME=prod_user
export SPRING_DATASOURCE_PASSWORD=prod_secure_password
export SERVER_PORT=9090
export LOGGING_LEVEL_ROOT=WARN
export APP_NAME=ProductionApp

# Setting environment variables in Windows:
set SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/prod_db
set SPRING_DATASOURCE_USERNAME=prod_user
set SPRING_DATASOURCE_PASSWORD=prod_secure_password

# Using an .env file with Spring Boot (needs spring-boot-dotenv dependency)
# .env file:
SERVER_PORT=9090
SPRING_DATASOURCE_URL=jdbc:mysql://localhost:3306/dev_db
SPRING_PROFILES_ACTIVE=dev

# --- Using Command-Line Arguments ---
# Override configuration values when running the JAR
java -jar myapp.jar --server.port=9090 \
    --spring.datasource.url=jdbc:mysql://localhost:3306/test_db \
    --logging.level.root=DEBUG \
    --app.feature.enabled=false \
    --spring.profiles.active=test

# Using JSON to set multiple properties at once
java -jar myapp.jar --spring.application.json='{"server":{"port":8080},"spring":{"datasource":
{"url":"jdbc:mysql://localhost:3306/db"}}}'

# --- Using Random Values ---
# application.yml with random values
app:
  secret: ${random.uuid}
  token: ${random.uuid}
  number: ${random.int(100,200)}
  long: ${random.long(1000000)}
```

4.3 Profiles (@Profile)

Hands-On Code:

java

CopyDownload

```
// ---- Profile-Specific Configuration Files ----
```

```
// application-dev.properties
server.port=8082
spring.datasource.url=jdbc:h2:mem:devdb
spring.h2.console.enabled=true
logging.level.com.myapp=DEBUG
app.feature.enabled=true

// application-prod.properties
server.port=80
spring.datasource.url=jdbc:mysql://prod-server:3306/proddb
spring.datasource.username=${PROD_DB_USER}
spring.datasource.password=${PROD_DB_PASSWORD}
logging.level.com.myapp=WARN
app.feature.enabled=false

// application-test.properties
server.port=0  # Random port for testing
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=create-drop
logging.level.com.myapp=TRACE

// ---- Using @Profile Annotation ----
package net.guides.springboot2.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.Profile;

@Configuration
public class ProfileConfig {

   @Bean
   @Profile("dev")
   public DataSource devDataSource() {
      System.out.println("Creating Development DataSource");
      return new H2DataSource();
   }

   @Bean
   @Profile("prod")
   public DataSource prodDataSource() {
      System.out.println("Creating Production DataSource with high performance settings");
      HikariConfig config = new HikariConfig();
      config.setMaximumPoolSize(50); // Higher pool for production
      config.setConnectionTimeout(10000);
      return new HikariDataSource(config);
   }

   @Bean
   @Profile("test")
   public DataSource testDataSource() {
      System.out.println("Creating Test DataSource with ephemeral database");
      return new H2DataSource();
   }
```

```
    // Beans that are active in multiple profiles
    @Bean
    @Profile({"dev", "test"}) // Active in dev OR test profiles
    public CacheManager devTestCacheManager() {
        System.out.println("Creating CacheManager for Dev/Test with small cache");
        return new CaffeineCacheManager();
    }

    // Beans that are NOT active in specific profiles
    @Bean
    @Profile("!prod") // Active for all profiles EXCEPT prod
    public DevToolsConfig devToolsConfig() {
        System.out.println("DevTools enabled (not in production)");
        return new DevToolsConfig();
    }
}

// ---- Activating Profiles ----
// 1. In application.properties:
spring.profiles.active=dev

// 2. Using command line:
java -jar myapp.jar --spring.profiles.active=prod

// 3. Using environment variable:
export SPRING_PROFILES_ACTIVE=dev,test

// 4. Multiple profiles active:
spring.profiles.active=dev,test,reports

// 5. Programmatically in code:
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication app = new SpringApplication(Application.class);
        app.setAdditionalProfiles("dev");
        app.run(args);
    }
}

// ---- @Profile on @Configuration classes ----
@Configuration
@Profile("prod")
public class ProductionConfig {

    @Bean
    public SecurityConfig prodSecurityConfig() {
        return new SecurityConfig(true); // Enable security in prod
    }
}

@Configuration
@Profile("!prod")
```

```
public class NonProductionConfig {

    @Bean
    public SecurityConfig devSecurityConfig() {
        return new SecurityConfig(false); // Disable security in dev/test
    }
}

// ---- Profile-specific Property Injection ----
@Service
public class ProfileAwareService {

    @Value("${app.feature.enabled}")
    private boolean featureEnabled;

    @Value("${app.name}")
    private String appName;

    @Value("${app.environment:default}")  // Default value if not set
    private String environment;

    @PostConstruct
    public void init() {
        System.out.println("Application: " + appName);
        System.out.println("Environment: " + environment);
        System.out.println("Feature enabled: " + featureEnabled);
    }
}
```

4.4 Property Injection: @Value and @ConfigurationProperties

Hands-On Code:

java

CopyDownload

```
// ---- @Value Annotation (Inject individual values) ----
package net.guides.springboot2.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class AppConfigValues {

    // Inject simple values
    @Value("${app.name}")
    private String appName;

    @Value("${app.version:1.0.0}")  // Default value if not specified
    private String appVersion;

    // Inject numbers with default
    @Value("${app.batch.size:100}")
    private int batchSize;
```

```
    // Inject boolean values
    @Value("${app.feature.enabled:false}")
    private boolean featureEnabled;

    // Inject lists (comma-separated)
    @Value("${app.admin.emails:admin@example.com}")
    private String[] adminEmails;

    // Inject maps (using Spring Expression Language - SpEL)
    @Value("#{${app.feature.flags:{'default':'true'}}}")
    private Map<String, Boolean> featureFlags;

    // Inject system properties
    @Value("${java.home}")
    private String javaHome;

    // Inject environment variables using SpEL
    @Value("#{systemEnvironment['USER']}")
    private String osUser;

    // Inject random values
    @Value("${random.uuid}")
    private String uuid;

    @Value("${random.int(1000,9999)}")
    private int randomCode;

    // Complex expressions with SpEL
    @Value("#{${app.timeout.ms:5000} * ${app.timeout.multiplier:1}}")
    private long effectiveTimeout;

    @PostConstruct
    public void printValues() {
        System.out.println("App Name: " + appName);
        System.out.println("App Version: " + appVersion);
        System.out.println("Batch Size: " + batchSize);
        System.out.println("Feature Enabled: " + featureEnabled);
        System.out.println("Admin Emails: " + Arrays.toString(adminEmails));
        System.out.println("Java Home: " + javaHome);
        System.out.println("OS User: " + osUser);
        System.out.println("UUID: " + uuid);
        System.out.println("Random Code: " + randomCode);
        System.out.println("Effective Timeout: " + effectiveTimeout);
    }

    // Usage in methods
    public void processItems() {
        if (featureEnabled) {
            // Process with new feature
        } else {
            // Process with legacy code
        }
    }
}
```

```
// ---- @ConfigurationProperties (Bind group of properties to POJO) ----
package net.guides.springboot2.config;

import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.boot.context.properties.EnableConfigurationProperties;
import org.springframework.context.annotation.Configuration;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import javax.validation.constraints.Email;
import javax.validation.constraints.Min;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;
import java.time.Duration;
import java.util.List;
import java.util.Map;

/**
 * @ConfigurationProperties binds a group of related properties to a POJO.
 * Much cleaner than using many @Value annotations for related properties.
 */
@Component
@ConfigurationProperties(prefix = "app")
@Validated  // Enable validation for the properties
public class AppProperties {

    @NotBlank(message = "Application name cannot be blank")
    private String name;

    @Pattern(regexp = "^\\d+\\.\\d+\\.\\d+$", message = "Version must follow semantic
versioning")
    private String version = "1.0.0";

    private String description;

    private Admin admin = new Admin();

    private Feature feature = new Feature();

    private Batch batch = new Batch();

    private Security security = new Security();

    private List<String> domains;

    private Map<String, String> metadata;

    // Nested class for admin properties
    public static class Admin {
        @Email(message = "Admin email must be valid")
        private String email;

        private String name;
```

```
    private List<String> roles;

    // Getters and Setters
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public List<String> getRoles() { return roles; }
    public void setRoles(List<String> roles) { this.roles = roles; }
}

public static class Feature {
    private boolean enabled = true;
    private Cache cache = new Cache();
    private Database database = new Database();

    public static class Cache {
        private boolean enabled = true;
        private Duration ttl = Duration.ofMinutes(10);
        private int maxSize = 1000;

        // Getters and Setters
        public boolean isEnabled() { return enabled; }
        public void setEnabled(boolean enabled) { this.enabled = enabled; }
        public Duration getTtl() { return ttl; }
        public void setTtl(Duration ttl) { this.ttl = ttl; }
        public int getMaxSize() { return maxSize; }
        public void setMaxSize(int maxSize) { this.maxSize = maxSize; }
    }

    public static class Database {
        private int poolSize = 10;
        private String dialect = "org.hibernate.dialect.MySQL8Dialect";

        // Getters and Setters
        public int getPoolSize() { return poolSize; }
        public void setPoolSize(int poolSize) { this.poolSize = poolSize; }
        public String getDialect() { return dialect; }
        public void setDialect(String dialect) { this.dialect = dialect; }
    }

    public Cache getCache() { return cache; }
    public void setCache(Cache cache) { this.cache = cache; }
    public Database getDatabase() { return database; }
    public void setDatabase(Database database) { this.database = database; }
    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) { this.enabled = enabled; }
}

public static class Batch {
    @Min(value = 1, message = "Batch size must be at least 1")
    private int size = 100;

    private Duration timeout = Duration.ofSeconds(30);
```

```
    private boolean retryEnabled = true;
    private int maxRetries = 3;

    // Getters and Setters
    public int getSize() { return size; }
    public void setSize(int size) { this.size = size; }
    public Duration getTimeout() { return timeout; }
    public void setTimeout(Duration timeout) { this.timeout = timeout; }
    public boolean isRetryEnabled() { return retryEnabled; }
    public void setRetryEnabled(boolean retryEnabled) { this.retryEnabled = retryEnabled; }
    public int getMaxRetries() { return maxRetries; }
    public void setMaxRetries(int maxRetries) { this.maxRetries = maxRetries; }
}

public static class Security {
    private boolean enabled = true;
    private Jwt jwt = new Jwt();

    public static class Jwt {
        private String secret;
        private Duration expiration = Duration.ofHours(24);
        private String issuer = "myapp";

        // Getters and Setters
        public String getSecret() { return secret; }
        public void setSecret(String secret) { this.secret = secret; }
        public Duration getExpiration() { return expiration; }
        public void setExpiration(Duration expiration) { this.expiration = expiration; }
        public String getIssuer() { return issuer; }
        public void setIssuer(String issuer) { this.issuer = issuer; }
    }

    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) { this.enabled = enabled; }
    public Jwt getJwt() { return jwt; }
    public void setJwt(Jwt jwt) { this.jwt = jwt; }
}

// Getters and Setters for all top-level properties
public String getName() { return name; }
public void setName(String name) { this.name = name; }
public String getVersion() { return version; }
public void setVersion(String version) { this.version = version; }
public String getDescription() { return description; }
public void setDescription(String description) { this.description = description; }
public Admin getAdmin() { return admin; }
public void setAdmin(Admin admin) { this.admin = admin; }
public Feature getFeature() { return feature; }
public void setFeature(Feature feature) { this.feature = feature; }
public Batch getBatch() { return batch; }
public void setBatch(Batch batch) { this.batch = batch; }
public Security getSecurity() { return security; }
public void setSecurity(Security security) { this.security = security; }
public List<String> getDomains() { return domains; }
```

```
    public void setDomains(List<String> domains) { this.domains = domains; }
    public Map<String, String> getMetadata() { return metadata; }
    public void setMetadata(Map<String, String> metadata) { this.metadata = metadata; }
}

// ---- Enabling Configuration Properties ----
@Configuration
@EnableConfigurationProperties(AppProperties.class)
public class PropertiesConfig {

    @Autowired
    private AppProperties appProperties;

    @Bean
    public BeanWithProperties beanWithProperties() {
        // Use the bound properties
        return new BeanWithProperties(appProperties);
    }
}

// ---- Usage Example ----
@Service
public class PropertyUsageService {

    @Autowired
    private AppProperties appProperties;

    public void demonstratePropertyUsage() {
        // Access nested properties cleanly
        String appName = appProperties.getName();
        String adminEmail = appProperties.getAdmin().getEmail();
        boolean cacheEnabled = appProperties.getFeature().getCache().isEnabled();
        Duration cacheTtl = appProperties.getFeature().getCache().getTtl();
        int batchSize = appProperties.getBatch().getSize();
        String jwtSecret = appProperties.getSecurity().getJwt().getSecret();
        List<String> domains = appProperties.getDomains();
        Map<String, String> metadata = appProperties.getMetadata();

        System.out.println("App: " + appName);
        System.out.println("Admin Email: " + adminEmail);
        System.out.println("Cache Enabled: " + cacheEnabled);
        System.out.println("Cache TTL: " + cacheTtl);
        System.out.println("Batch Size: " + batchSize);
        System.out.println("JWT Secret: " + jwtSecret);
        System.out.println("Domains: " + domains);
        System.out.println("Metadata: " + metadata);
    }
}

// Corresponding YAML configuration:
/*
app:
  name: MySpringBootApp
  version: 2.1.3
```

```
  description: Advanced Spring Boot Application
  admin:
    email: john.doe@example.com
    name: John Doe
    roles:
      - ADMIN
      - MANAGER
  feature:
    enabled: true
    cache:
      enabled: true
      ttl: PT10M
      max-size: 5000
    database:
      pool-size: 20
      dialect: org.hibernate.dialect.PostgreSQLDialect
  batch:
    size: 200
    timeout: PT30S
    retry-enabled: true
    max-retries: 5
  security:
    enabled: true
    jwt:
      secret: 0d3b31f9d7e8a1c2b4f6a8e9c0d3f7b8e1a4c6d9f2b8e3a7c9d4f6e8a1b3c5d7
      expiration: PT8H
      issuer: myapp
  domains:
    - example.com
    - myapp.com
    - api.myapp.com
  metadata:
    created-by: admin
    department: engineering
    environment: production
    build-number: 12345
*/
```

Comparison Table:

Feature	**@Value**	**@ConfigurationProperties**
Purpose	Inject individual values	Bind multiple related properties
Use Case	Simple property access	Structured/grouped configuration
Validation	Limited	Strong

Support	(SpEL)	(with JSR-303 annotations)
Default Value Support	Yes (`@Value("${key:default}")`)	No (use validation instead)
Type Safety	Basic	Strong, type-safe
IDE Support	Good	Excellent (auto-completion)
Nested Objects	Not supported	Supported
Complexity	Simple	More complex but cleaner

---

5. Spring Boot Starters

5.1 What are Starters?

A Starter is a pre-defined set of dependencies bundled under a single starter module, designed to support specific application needs (e.g., web, JPA, security).

Hands-On Code:

xml

CopyDownloadRun

```
<!-- pom.xml - Complete Maven configuration with common starters -->
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>spring-boot-app</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.1.5</version>
        <relativePath/>
    </parent>
```

```
<properties>
  <java.version>17</java.version>
</properties>

<dependencies>
  <!-- Core Starter - Essential for any Spring Boot application -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter</artifactId>
  </dependency>

  <!-- Web Starter - For building web applications and RESTful services -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- WebFlux Starter - For reactive web applications -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
  </dependency>

  <!-- Data JPA Starter - For database access with JPA/Hibernate -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Security Starter - For authentication and authorization -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- Test Starter - For unit and integration testing -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <!-- Thymeleaf Starter - For server-side templating -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>

  <!-- Actuator Starter - For monitoring and management -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
```

```
<!-- Validation Starter - For bean validation -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- DevTools Starter - For development productivity -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <optional>true</optional>
    <scope>runtime</scope>
</dependency>

<!-- Mail Starter - For sending emails -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
</dependency>

<!-- AMQP Starter - For messaging with RabbitMQ -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<!-- Kafka Starter - For streaming with Kafka -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>

<!-- Data Redis Starter - For Redis caching -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>

<!-- MongoDB Starter - For MongoDB integration -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>

<!-- Batch Starter - For batch processing -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-batch</artifactId>
</dependency>

<!-- JTA Starter - For distributed transactions -->
<dependency>
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter-jta-atomikos</artifactId>
</dependency>

<!-- OAuth2 Client Starter - For OAuth2 authentication -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>

<!-- OAuth2 Resource Server Starter - For OAuth2 resource server -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>

<!-- Cache Starter - For caching support -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-cache</artifactId>
</dependency>

<!-- Quartz Starter - For scheduling -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-quartz</artifactId>
</dependency>

<!-- Database Drivers (Runtime only) -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- Cloud Starters for Microservices -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<dependency>
```

```
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-gateway</artifactId>
      </dependency>

      <!-- Resilience4j for Circuit Breaker -->
      <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
      </dependency>
    </dependencies>

    <dependencyManagement>
      <dependencies>
        <dependency>
          <groupId>org.springframework.cloud</groupId>
          <artifactId>spring-cloud-dependencies</artifactId>
          <version>2022.0.4</version>
          <type>pom</type>
          <scope>import</scope>
        </dependency>
      </dependencies>
    </dependencyManagement>

    <build>
      <plugins>
        <plugin>
          <groupId>org.springframework.boot</groupId>
          <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
      </plugins>
    </build>
</project>
```

5.2 Common Starters with Examples

5.2.1 spring-boot-starter-web

Hands-On Code:

java

CopyDownload

```
/**
 * spring-boot-starter-web includes:
 * - Spring MVC
 * - Jackson (for JSON binding)
 * - Embedded Tomcat (by default)
 * - Spring Web
 */
package net.guides.springboot2.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import jakarta.validation.Valid;
```

```
@RestController
@RequestMapping("/api/hello")
public class HelloController {

    private static final Logger logger = LoggerFactory.getLogger(HelloController.class);

    /**
     * Simple GET endpoint returning a greeting message
     */
    @GetMapping
    public String greet() {
        logger.info("GET /api/hello called");
        return "Hello, World!";
    }

    /**
     * GET endpoint with path variable
     */
    @GetMapping("/{name}")
    public String greetByName(@PathVariable String name) {
        return "Hello, " + name + "!";
    }

    /**
     * GET endpoint with query parameters
     */
    @GetMapping("/greet")
    public String greetWithParams(@RequestParam String name,
                                  @RequestParam(defaultValue = "en") String lang) {
        if ("es".equals(lang)) {
            return "¡Hola, " + name + "!";
        } else if ("fr".equals(lang)) {
            return "Bonjour, " + name + "!";
        }
        return "Hello, " + name + "!";
    }

    /**
     * POST endpoint with JSON request body
     */
    @PostMapping("/user")
    public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
        logger.info("Creating user: {}", user);
        // Process user creation
        user.setId(1L);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }

    /**
     * File upload endpoint
     */
    @PostMapping("/upload")
    public ResponseEntity<String> uploadFile(@RequestParam("file") MultipartFile file) {
        if (file.isEmpty()) {
```

```
            return ResponseEntity.badRequest().body("Please select a file to upload");
        }
        logger.info("Received file: {}", file.getOriginalFilename());
        logger.info("File size: {} bytes", file.getSize());
        return ResponseEntity.ok("File uploaded: " + file.getOriginalFilename());
    }
}
```

5.2.2 spring-boot-starter-data-jpa

Hands-On Code:

java

CopyDownload

```
/**
 * spring-boot-starter-data-jpa includes:
 * - Spring Data JPA
 * - Hibernate
 * - Spring ORM
 */
package net.guides.springboot2.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Past;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "users",
      uniqueConstraints = @UniqueConstraint(columnNames = "email"))
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(nullable = false)
    private String firstName;

    @NotBlank
    @Column(nullable = false)
    private String lastName;

    @NotBlank
    @Email
    @Column(nullable = false, unique = true)
    private String email;

    @NotBlank
```

```
@Column(nullable = false)
private String password;

@Past
@Column(name = "date_of_birth")
private LocalDateTime dateOfBirth;

@Column(name = "active")
private boolean active = true;

@CreationTimestamp
@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt;

@UpdateTimestamp
@Column(name = "updated_at")
private LocalDateTime updatedAt;

@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Order> orders = new ArrayList<>();

@ManyToMany(fetch = FetchType.EAGER)
@JoinTable(
    name = "user_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
private List<Role> roles = new ArrayList<>();

// Constructors
public User() {}

public User(String firstName, String lastName, String email, String password) {
    this.firstName = firstName;
    this.lastName = lastName;
    this.email = email;
    this.password = password;
}

// Getters and Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }
public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }
public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }
public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }
public String getPassword() { return password; }
public void setPassword(String password) { this.password = password; }
public LocalDateTime getDateOfBirth() { return dateOfBirth; }
public void setDateOfBirth(LocalDateTime dateOfBirth) { this.dateOfBirth = dateOfBirth; }
public boolean isActive() { return active; }
public void setActive(boolean active) { this.active = active; }
```

```
    public LocalDateTime getCreatedAt() { return createdAt; }
    public LocalDateTime getUpdatedAt() { return updatedAt; }
    public List<Order> getOrders() { return orders; }
    public void setOrders(List<Order> orders) { this.orders = orders; }
    public List<Role> getRoles() { return roles; }
    public void setRoles(List<Role> roles) { this.roles = roles; }
}

// ---- Repository Interface ----
package net.guides.springboot2.repository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Optional;

@Repository
public interface UserRepository extends JpaRepository<User, Long> {

    // Derived query methods
    Optional<User> findByEmail(String email);

    boolean existsByEmail(String email);

    List<User> findByLastName(String lastName);

    List<User> findByActiveTrue();

    List<User> findByFirstNameAndLastName(String firstName, String lastName);

    Optional<User> findByEmailAndPassword(String email, String password);

    List<User> findByCreatedAtAfter(LocalDateTime date);

    // Pagination
    Page<User> findAllByActiveTrue(Pageable pageable);

    Page<User> findByLastNameContainingIgnoreCase(String lastName, Pageable pageable);

    // Custom JPQL query
    @Query("SELECT u FROM User u WHERE u.email LIKE CONCAT('%', :domain, '%')")
    List<User> findUsersByDomain(@Param("domain") String domain);

    // Native SQL query
    @Query(value = "SELECT * FROM users u WHERE u.created_at > :date", nativeQuery = true)
    List<User> findRecentUsers(@Param("date") String date);
```

```
    // Update query
    @Modifying
    @Transactional
    @Query("UPDATE User u SET u.active = :active WHERE u.id = :userId")
    int updateUserActiveStatus(@Param("userId") Long userId, @Param("active") boolean active);

    // Delete query
    @Modifying
    @Transactional
    @Query("DELETE FROM User u WHERE u.id = :userId")
    void deleteUserById(@Param("userId") Long userId);

    // Complex query with JOIN
    @Query("SELECT DISTINCT u FROM User u JOIN u.roles r WHERE r.name IN :roleNames")
    List<User> findUsersByRoles(@Param("roleNames") List<String> roleNames);
}

// ---- Service Layer ----
package net.guides.springboot2.service;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder passwordEncoder;

    public User registerUser(UserRegistrationDto dto) {
        // Check if email already exists
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new EmailAlreadyExistsException("Email already registered: " + dto.getEmail());
        }

        // Create new user
        User user = new User();
        user.setFirstName(dto.getFirstName());
        user.setLastName(dto.getLastName());
        user.setEmail(dto.getEmail());
        user.setPassword(passwordEncoder.encode(dto.getPassword()));
        user.setActive(true);

        // Save user
        return userRepository.save(user);
```

```
    }

    public Page<User> getActiveUsers(int page, int size) {
        Pageable pageable = PageRequest.of(page, size, Sort.by("lastName").ascending());
        return userRepository.findAllByActiveTrue(pageable);
    }

    @Transactional(readOnly = true)
    public Optional<User> getUserByEmail(String email) {
        return userRepository.findByEmail(email);
    }

    @Transactional(readOnly = true)
    public List<User> searchByLastName(String lastName) {
        return userRepository.findByLastNameContainingIgnoreCase(lastName,
Pageable.unpaged()).getContent();
    }

    public User updateUser(User updatedUser) {
        User existing = userRepository.findById(updatedUser.getId())
            .orElseThrow(() -> new UserNotFoundException("User not found with ID: " +
updatedUser.getId()));

        existing.setFirstName(updatedUser.getFirstName());
        existing.setLastName(updatedUser.getLastName());
        existing.setDateOfBirth(updatedUser.getDateOfBirth());
        // Don't update email or password without extra validation

        return userRepository.save(existing);
    }

    public void deactivateUser(Long userId) {
        int updated = userRepository.updateUserActiveStatus(userId, false);
        if (updated == 0) {
            throw new UserNotFoundException("User not found with ID: " + userId);
        }
    }

    public void permanentlyDeleteUser(Long userId) {
        userRepository.deleteUserById(userId);
    }
}
```

5.2.3 spring-boot-starter-security

Hands-On Code:

java

CopyDownload

```
/**
 * spring-boot-starter-security includes:
 * - Spring Security
 * - Core security features
 * - Authentication and authorization
 */
package net.guides.springboot2.security;
```

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.AuthenticationProvider;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfi
guration;
import
org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;

import java.util.Arrays;
import java.util.List;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private JwtAuthenticationFilter jwtAuthenticationFilter;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/manager/**").hasAnyRole("ADMIN", "MANAGER")
                .requestMatchers("/api/users/**").authenticated()
                .anyRequest().authenticated()
            )
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .authenticationProvider(authenticationProvider())
```

```
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public AuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config)
throws Exception {
        return config.getAuthenticationManager();
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration configuration = new CorsConfiguration();
        configuration.setAllowedOrigins(Arrays.asList("http://localhost:3000", "https://
myapp.com"));
        configuration.setAllowedMethods(Arrays.asList("GET", "POST", "PUT", "PATCH", "DELETE",
"OPTIONS"));
        configuration.setAllowedHeaders(Arrays.asList("Authorization", "Content-Type"));
        configuration.setExposedHeaders(List.of("Authorization"));
        configuration.setAllowCredentials(true);
        configuration.setMaxAge(3600L);

        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", configuration);
        return source;
    }
}

// ---- JWT Authentication Filter ----
package net.guides.springboot2.security;

import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.web.filter.OncePerRequestFilter;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
```

```
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtTokenProvider tokenProvider;

    @Autowired
    private UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        try {
            // Get JWT token from request header
            String jwt = getJwtFromRequest(request);

            if (jwt != null && tokenProvider.validateToken(jwt)) {
                // Extract user details from token
                String username = tokenProvider.getUsernameFromToken(jwt);
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);

                // Create authentication object
                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(userDetails, null,
userDetails.getAuthorities());
                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

                // Set authentication in SecurityContext
                SecurityContextHolder.getContext().setAuthentication(authentication);

                // Add user info to request for controller usage
                request.setAttribute("userId", userDetails.getUserId());
                request.setAttribute("username", username);
            }
        } catch (Exception ex) {
            logger.error("Could not set user authentication in security context", ex);
        }

        filterChain.doFilter(request, response);
    }

    private String getJwtFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}

// ---- JWT Token Provider ----
package net.guides.springboot2.security;
```

```
import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.Date;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtTokenProvider {

    @Value("${app.jwt.secret}")
    private String jwtSecret;

    @Value("${app.jwt.expiration}")
    private long jwtExpirationMs;

    @Value("${app.jwt.refresh-expiration}")
    private long refreshExpirationMs;

    private SecretKey getSigningKey() {
        return Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));
    }

    public String generateAccessToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", userDetails.getUserId());
        claims.put("roles", userDetails.getAuthorities().stream()
            .map(auth -> auth.getAuthority())
            .collect(java.util.stream.Collectors.toList()));
        claims.put("email", userDetails.getUsername());

        return generateToken(claims, userDetails.getUsername(), jwtExpirationMs);
    }

    public String generateRefreshToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("userId", userDetails.getUserId());
        claims.put("type", "refresh");

        return generateToken(claims, userDetails.getUsername(), refreshExpirationMs);
    }

    private String generateToken(Map<String, Object> claims, String subject, long expirationMs) {
        Instant now = Instant.now();
        Instant expiry = now.plusMillis(expirationMs);
```

```
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(subject)
        .setIssuedAt(Date.from(now))
        .setExpiration(Date.from(expiry))
        .signWith(getSigningKey(), SignatureAlgorithm.HS512)
        .compact();
}

public String getUsernameFromToken(String token) {
    return getClaimFromToken(token, Claims::getSubject);
}

public Long getUserIdFromToken(String token) {
    Claims claims = getAllClaimsFromToken(token);
    return claims.get("userId", Long.class);
}

public List<String> getRolesFromToken(String token) {
    Claims claims = getAllClaimsFromToken(token);
    return claims.get("roles", List.class);
}

public Date getExpirationDateFromToken(String token) {
    return getClaimFromToken(token, Claims::getExpiration);
}

public <T> T getClaimFromToken(String token, Function<Claims, T> claimsResolver) {
    final Claims claims = getAllClaimsFromToken(token);
    return claimsResolver.apply(claims);
}

private Claims getAllClaimsFromToken(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(getSigningKey())
        .build()
        .parseClaimsJws(token)
        .getBody();
}

public boolean isTokenExpired(String token) {
    final Date expiration = getExpirationDateFromToken(token);
    return expiration.before(new Date());
}

public boolean validateToken(String token) {
    try {
        getAllClaimsFromToken(token);
        return !isTokenExpired(token);
    } catch (Exception e) {
        return false;
    }
}
```

```
}

// ---- Authentication Controller ----
package net.guides.springboot2.controller;

import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtTokenProvider tokenProvider;

    @Autowired
    private UserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        // Authenticate user
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
        );

        // Get user details
        UserDetails userDetails = (UserDetails) authentication.getPrincipal();

        // Generate tokens
        String accessToken = tokenProvider.generateAccessToken(userDetails);
        String refreshToken = tokenProvider.generateRefreshToken(userDetails);

        // Create response
        AuthResponse response = AuthResponse.builder()
            .accessToken(accessToken)
            .refreshToken(refreshToken)
            .tokenType("Bearer")
            .expiresIn(tokenProvider.getExpirationDateFromToken(accessToken))
            .user(convertToDto(userDetails))
            .build();

        return ResponseEntity.ok(response);
    }

    @PostMapping("/refresh")
```

```
    public ResponseEntity<AuthResponse> refreshToken(@RequestBody RefreshTokenRequest
request) {
        String refreshToken = request.getRefreshToken();

        if (!tokenProvider.validateToken(refreshToken)) {
            throw new InvalidTokenException("Invalid refresh token");
        }

        String username = tokenProvider.getUsernameFromToken(refreshToken);
        UserDetails userDetails = userDetailsService.loadUserByUsername(username);

        String newAccessToken = tokenProvider.generateAccessToken(userDetails);

        AuthResponse response = AuthResponse.builder()
            .accessToken(newAccessToken)
            .refreshToken(refreshToken)
            .tokenType("Bearer")
            .expiresIn(tokenProvider.getExpirationDateFromToken(newAccessToken))
            .user(convertToDto(userDetails))
            .build();

        return ResponseEntity.ok(response);
    }

    @PostMapping("/logout")
    public ResponseEntity<?> logout() {
        // In JWT-based authentication, logout is client-side
        // We can add token to blacklist if needed
        return ResponseEntity.ok("Logout successful");
    }
}

// ---- Auth Request DTOs ----
package net.guides.springboot2.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.NotBlank;
import lombok.Builder;
import lombok.Data;

@Data
@Builder
public class LoginRequest {
    @NotBlank @Email
    private String email;

    @NotBlank
    private String password;
}

@Data
@Builder
public class AuthResponse {
    private String accessToken;
```

```
    private String refreshToken;
    private String tokenType;
    private Date expiresIn;
    private UserDto user;
}

@Data
@Builder
public class RefreshTokenRequest {
    @NotBlank
    private String refreshToken;
}
```

5.2.4 spring-boot-starter-test

Hands-On Code:

java

CopyDownload

```
/**
 * spring-boot-starter-test includes:
 * - JUnit 5 (JUnit Jupiter)
 * - Mockito
 * - Spring Test
 * - AssertJ
 * - JSONassert
 * - Spring Boot Test
 */
package net.guides.springboot2.service;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.springframework.test.util.ReflectionTestUtils;

import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.assertThatThrownBy;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

/**
 * Unit testing with JUnit 5 and Mockito
 */
@ExtendWith(MockitoExtension.class) // Enable Mockito annotations
class UserServiceTest {

    @Mock
    private UserRepository userRepository;

    @Mock
    private PasswordEncoder passwordEncoder;
```

```
@InjectMocks // Creates an instance of the class and injects mocks
private UserService userService;

private User testUser;
private UserRegistrationDto registrationDto;

@BeforeEach
void setUp() {
    // Set up test data
    testUser = new User();
    testUser.setId(1L);
    testUser.setFirstName("John");
    testUser.setLastName("Doe");
    testUser.setEmail("john@example.com");
    testUser.setPassword("encodedPassword");
    testUser.setActive(true);

    registrationDto = UserRegistrationDto.builder()
        .firstName("John")
        .lastName("Doe")
        .email("john@example.com")
        .password("plainPassword")
        .build();
}

@Test
void registerUser_Success_ReturnsUser() {
    // Arrange
    when(passwordEncoder.encode("plainPassword")).thenReturn("encodedPassword");
    when(userRepository.existsByEmail("john@example.com")).thenReturn(false);
    when(userRepository.save(any(User.class))).thenReturn(testUser);

    // Act
    User result = userService.registerUser(registrationDto);

    // Assert
    assertThat(result).isNotNull();
    assertThat(result.getId()).isEqualTo(1L);
    assertThat(result.getEmail()).isEqualTo("john@example.com");
    assertThat(result.getPassword()).isEqualTo("encodedPassword");
    assertThat(result.isActive()).isTrue();

    verify(passwordEncoder).encode("plainPassword");
    verify(userRepository).existsByEmail("john@example.com");
    verify(userRepository).save(any(User.class));
}

@Test
void registerUser_EmailExists_ThrowsException() {
    // Arrange
    when(userRepository.existsByEmail("john@example.com")).thenReturn(true);

    // Act & Assert
```

```
        assertThatThrownBy(() -> userService.registerUser(registrationDto))
            .isInstanceOf(EmailAlreadyExistsException.class)
            .hasMessageContaining("Email already registered");

        verify(userRepository, never()).save(any(User.class));
    }

    @Test
    void getUserByEmail_UserExists_ReturnsUser() {
        // Arrange
        when(userRepository.findByEmail("john@example.com")).thenReturn(Optional.of(testUser));

        // Act
        Optional<User> result = userService.getUserByEmail("john@example.com");

        // Assert
        assertThat(result).isPresent();
        assertThat(result.get()).isEqualTo(testUser);
        verify(userRepository).findByEmail("john@example.com");
    }

    @Test
    void getUserByEmail_UserNotFound_ReturnsEmpty() {
        // Arrange

when(userRepository.findByEmail("nonexistent@example.com")).thenReturn(Optional.empty());

        // Act
        Optional<User> result = userService.getUserByEmail("nonexistent@example.com");

        // Assert
        assertThat(result).isEmpty();
        verify(userRepository).findByEmail("nonexistent@example.com");
    }
}

// ---- Integration Testing ----
package net.guides.springboot2;

import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.test.context.ActiveProfiles;

/**
 * Integration test that loads full application context
 */
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
class ApplicationIntegrationTest {

    @Autowired
```

```
    private MockMvc mockMvc;

    @Autowired
    private UserService userService;

    @Autowired
    private UserRepository userRepository;

    @Test
    void testFullUserRegistrationFlow() throws Exception {
        // Create user
        UserRegistrationDto dto = UserRegistrationDto.builder()
            .firstName("Jane")
            .lastName("Smith")
            .email("jane.smith@example.com")
            .password("SecurePass123")
            .build();

        User createdUser = userService.registerUser(dto);
        assertThat(createdUser.getId()).isNotNull();

        // Verify user exists
        Optional<User> foundUser = userService.getUserByEmail("jane.smith@example.com");
        assertThat(foundUser).isPresent();
        assertThat(foundUser.get().getFirstName()).isEqualTo("Jane");

        // Test REST API
        mockMvc.perform(get("/api/users/{id}", createdUser.getId()))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.email").value("jane.smith@example.com"));

        // Clean up
        userRepository.deleteAll();
    }
}

// ---- Web Layer Testing (@WebMvcTest) ----
package net.guides.springboot2.controller;

import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class) // Tests only web layer
class UserControllerTest {

    @Autowired
    private MockMvc mockMvc;
```

```
    @MockBean // Mock the service bean
    private UserService userService;

    @Test
    void getUserById_UserExists_ReturnsUser() throws Exception {
        // Arrange
        User user = new User(1L, "John", "Doe", "john@example.com");
        when(userService.getUserById(1L)).thenReturn(Optional.of(user));

        // Act & Assert
        mockMvc.perform(get("/api/users/1"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(1L))
            .andExpect(jsonPath("$.firstName").value("John"))
            .andExpect(jsonPath("$.lastName").value("Doe"))
            .andExpect(jsonPath("$.email").value("john@example.com"));
    }

    @Test
    void getUserById_UserNotFound_ReturnsNotFound() throws Exception {
        // Arrange
        when(userService.getUserById(999L)).thenReturn(Optional.empty());

        // Act & Assert
        mockMvc.perform(get("/api/users/999"))
            .andExpect(status().isNotFound());
    }

    @Test
    void createUser_ValidUser_ReturnsCreated() throws Exception {
        // Arrange
        User user = new User(1L, "John", "Doe", "john@example.com");
        when(userService.registerUser(any())).thenReturn(user);

        String userJson = """
            {
                "firstName": "John",
                "lastName": "Doe",
                "email": "john@example.com",
                "password": "SecurePass123"
            }
            """;

        // Act & Assert
        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content(userJson))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(1L))
            .andExpect(jsonPath("$.email").value("john@example.com"));
    }
}

// ---- Data Layer Testing (@DataJpaTest) ----
```

```
package net.guides.springboot2.repository;

import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;

@DataJpaTest // Configures H2 in-memory database and JPA components
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    void saveUser_ValidUser_ReturnsSavedUser() {
        // Arrange
        User user = new User("John", "Doe", "john@example.com", "password");

        // Act
        User savedUser = userRepository.save(user);

        // Assert
        assertThat(savedUser.getId()).isNotNull();
        assertThat(savedUser.getEmail()).isEqualTo("john@example.com");
        assertThat(savedUser.getCreatedAt()).isNotNull();
        assertThat(savedUser.getUpdatedAt()).isNotNull();
    }

    @Test
    void findUserByEmail_UserExists_ReturnsUser() {
        // Arrange
        User user = new User("John", "Doe", "john@example.com", "password");
        User savedUser = userRepository.save(user);

        // Act
        Optional<User> foundUser = userRepository.findByEmail("john@example.com");

        // Assert
        assertThat(foundUser).isPresent();
        assertThat(foundUser.get().getId()).isEqualTo(savedUser.getId());
        assertThat(foundUser.get().getFirstName()).isEqualTo("John");
        assertThat(foundUser.get().getLastName()).isEqualTo("Doe");
    }

    @Test
    void updateUser_ExistingUser_UpdatesFields() {
        // Arrange
        User user = userRepository.save(new User("John", "Doe", "john@example.com", "password"));

        // Act
        user.setFirstName("Jonathan");
        user.setLastName("Smith");
        User updatedUser = userRepository.save(user);
```

```
        // Assert
        assertThat(updatedUser.getFirstName()).isEqualTo("Jonathan");
        assertThat(updatedUser.getLastName()).isEqualTo("Smith");
        assertThat(updatedUser.getUpdatedAt()).isAfter(updatedUser.getCreatedAt());
    }

    @Test
    void deleteUser_ExistingUser_RemovesUser() {
        // Arrange
        User user = userRepository.save(new User("John", "Doe", "john@example.com", "password"));
        Long userId = user.getId();

        // Act
        userRepository.deleteById(userId);

        // Assert
        assertThat(userRepository.findById(userId)).isEmpty();
    }
}
```

Other Useful Starters:

Starter	Purpose
spring-boot-starter-thymeleaf	HTML template rendering with Thymelea f
spring-boot-starter-mail	Sending emails via JavaMail
spring-boot-starter-actuator	Monitoring and managing applicatio ns
spring-boot-starter-validation	Bean validation using Hibernate Validator
spring-boot-starter-cache	Caching support
spring-boot-	Schedulin g with

starter-quartz	Quartz
spring-boot-starter-batch	Batch processing
spring-boot-starter-amqp	Messaging with RabbitMQ
spring-boot-starter-kafka	Streaming with Apache Kafka
spring-boot-starter-data-redis	Redis integration
spring-boot-starter-data-mongodb	MongoDB integration
spring-boot-starter-oauth2-client	OAuth2 client authentication
spring-boot-starter-oauth2-resource-server	OAuth2 resource server

---

6. Web Development with Spring Boot (Spring MVC)

6.1 Creating REST APIs (@RestController)

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.controller;

import org.springframework.web.bind.annotation.*;
import org.springframework.http.ResponseEntity;
```

```
import org.springframework.http.HttpStatus;

import jakarta.validation.Valid;
import java.util.List;

@RestController // Combines @Controller and @ResponseBody
@RequestMapping("/api/products") // Base URL for all endpoints
public class ProductController {

    private final ProductService productService;

    public ProductController(ProductService productService) {
        this.productService = productService;
    }

    /**
     * GET /api/products - Get all products
     */
    @GetMapping
    public ResponseEntity<List<Product>> getAllProducts() {
        return ResponseEntity.ok(productService.findAll());
    }

    /**
     * GET /api/products/{id} - Get product by ID
     */
    @GetMapping("/{id}")
    public ResponseEntity<Product> getProductById(@PathVariable Long id) {
        return productService.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * POST /api/products - Create new product
     */
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Product createProduct(@Valid @RequestBody Product product) {
        return productService.save(product);
    }

    /**
     * PUT /api/products/{id} - Update product
     */
    @PutMapping("/{id}")
    public ResponseEntity<Product> updateProduct(@PathVariable Long id,
                                                 @Valid @RequestBody Product product) {
        Product updated = productService.update(id, product);
        return ResponseEntity.ok(updated);
    }

    /**
     * PATCH /api/products/{id} - Partial update product
```

```
    */
    @PatchMapping("/{id}")
    public ResponseEntity<Product> partialUpdateProduct(@PathVariable Long id,
                                                        @RequestBody Map<String, Object> updates) {
        Product updated = productService.partialUpdate(id, updates);
        return ResponseEntity.ok(updated);
    }

    /**
     * DELETE /api/products/{id} - Delete product
     */
    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteProduct(@PathVariable Long id) {
        productService.delete(id);
    }
}
```

6.2 Handling Request Parameters

Hands-On Code:

java

CopyDownload

```
@RestController
@RequestMapping("/api/orders")
public class OrderController {

    /**
     * @PathVariable - Binds URL segment to method parameter
     * Example: /api/orders/123
     */
    @GetMapping("/{orderId}")
    public ResponseEntity<Order> getOrder(@PathVariable Long orderId) {
        return orderService.findById(orderId)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * Multiple @PathVariable parameters
     * Example: /api/orders/123/items/456
     */
    @GetMapping("/{orderId}/items/{itemId}")
    public ResponseEntity<OrderItem> getOrderItem(@PathVariable Long orderId,
                                                  @PathVariable Long itemId) {
        return orderService.findItem(orderId, itemId)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * @RequestParam - Extracts query parameters from URL
     * Example: /api/orders?status=active&page=0&size=20
     */
```

```
    @GetMapping
    public ResponseEntity<Page<Order>> getOrders(
            @RequestParam(required = false) String status,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size,
            @RequestParam(defaultValue = "createdAt,desc") String sort) {

        Pageable pageable = PageRequest.of(page, size, Sort.by(sort.split(",")));
        return ResponseEntity.ok(orderService.findByStatus(status, pageable));
    }

    /**
     * @RequestBody - Binds request JSON body to object
     * Example: POST /api/orders with JSON body
     */
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Order createOrder(@Valid @RequestBody CreateOrderRequest request) {
        return orderService.createOrder(request);
    }

    /**
     * Combined usage of all parameter annotations
     * Example: PUT /api/orders/123?includeHistory=true
     * Request body: JSON order data
     */
    @PutMapping("/{orderId}")
    public ResponseEntity<Order> updateOrder(
            @PathVariable Long orderId,
            @RequestParam(defaultValue = "false") boolean includeHistory,
            @Valid @RequestBody Order order) {
        return ResponseEntity.ok(orderService.update(orderId, order, includeHistory));
    }

    /**
     * @RequestHeader - Access HTTP headers
     */
    @GetMapping("/headers")
    public ResponseEntity<Map<String, String>> getHeaders(
            @RequestHeader("Authorization") String authToken,
            @RequestHeader(value = "Accept-Language", defaultValue = "en") String language) {
        return ResponseEntity.ok(Map.of("auth", authToken, "lang", language));
    }
}
```

6.3 ResponseEntity and Status Codes

Hands-On Code:

java

CopyDownload

```
@RestController
@RequestMapping("/api/customers")
public class CustomerController {
```

```
    /**
     * Using ResponseEntity for full control over response
     */
    @PostMapping
    public ResponseEntity<Customer> createCustomer(@Valid @RequestBody Customer
customer) {
        // Validate business rules
        if (customer.getEmail() == null || customer.getEmail().isEmpty()) {
            return ResponseEntity.badRequest()
                    .header("X-Error-Code", "MISSING_EMAIL")
                    .build();
        }

        Customer saved = customerService.save(customer);

        // Created with location header
        URI location = ServletUriComponentsBuilder
                .fromCurrentRequest()
                .path("/{id}")
                .buildAndExpand(saved.getId())
                .toUri();

        return ResponseEntity
                .created(location)
                .body(saved);
    }

    /**
     * Different HTTP status codes
     */
    @GetMapping("/{id}")
    public ResponseEntity<Customer> getCustomer(@PathVariable Long id) {
        Customer customer = customerService.findById(id);
        if (customer == null) {
            return ResponseEntity
                    .status(HttpStatus.NOT_FOUND)
                    .body(null);
        }

        if (!customer.isActive()) {
            return ResponseEntity
                    .status(HttpStatus.GONE)
                    .body(customer);
        }

        return ResponseEntity
                .ok(customer);
    }

    /**
     * Using ResponseEntity with custom headers
     */
    @GetMapping("/report")
    public ResponseEntity<byte[]> getReport(@RequestParam String type) {
```

```
    byte[] reportData = customerService.generateReport(type);

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_PDF);
    headers.setContentDisposition(ContentDisposition.builder("attachment")
            .filename("customers.pdf")
            .build());
    headers.setCacheControl("no-cache, no-store, must-revalidate");
    headers.setPragma("no-cache");
    headers.setExpires(0);

    return new ResponseEntity<>(reportData, headers, HttpStatus.OK);
}

/**
 * ResponseEntity with status and body builders
 */
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteCustomer(@PathVariable Long id) {
    if (!customerService.exists(id)) {
        return ResponseEntity
                .notFound()
                .build();
    }

    customerService.delete(id);
    return ResponseEntity
            .noContent()
            .build();
}

/**
 * Common status codes with their usage
 */
@GetMapping("/examples")
public String showStatusExamples() {
    // 200 OK - Success
    ResponseEntity.ok("Success");

    // 201 Created - New resource created
    ResponseEntity.created(URI.create("/api/example/1")).build();

    // 204 No Content - Success but no content to return
    ResponseEntity.noContent().build();

    // 400 Bad Request - Invalid request
    ResponseEntity.badRequest().body("Invalid input");

    // 404 Not Found - Resource not found
    ResponseEntity.notFound().build();

    // 401 Unauthorized - Authentication required
    ResponseEntity.status(HttpStatus.UNAUTHORIZED).build();
```

```
        // 403 Forbidden - Authenticated but not authorized
        ResponseEntity.status(HttpStatus.FORBIDDEN).build();

        // 500 Internal Server Error - Server error
        ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body("Error occurred");

        return "Status codes example";
    }
}
```

6.4 Exception Handling

Hands-On Code:

java

CopyDownload

```
/**
 * Custom exceptions
 */
@ResponseStatus(HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) {
        super(message);
    }

    public ResourceNotFoundException(String resource, Long id) {
        super(resource + " not found with id: " + id);
    }
}

@ResponseStatus(HttpStatus.BAD_REQUEST)
public class ValidationException extends RuntimeException {
    private final Map<String, String> errors;

    public ValidationException(Map<String, String> errors) {
        super("Validation failed");
        this.errors = errors;
    }

    public Map<String, String> getErrors() {
        return errors;
    }
}

/**
 * Global Exception Handler using @ControllerAdvice
 */
@ControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    /**
     * Handle ResourceNotFoundException
     */
```

```
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleResourceNotFound(ResourceNotFoundException ex) {
        logger.warn("Resource not found: {}", ex.getMessage());
        ErrorResponse error = ErrorResponse.builder()
                .timestamp(LocalDateTime.now())
                .status(HttpStatus.NOT_FOUND.value())
                .error("Not Found")
                .message(ex.getMessage())
                .path(getPath())
                .build();
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    /**
     * Handle ValidationException
     */
    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ValidationErrorResponse> handleValidation(ValidationException ex) {
        logger.warn("Validation error: {}", ex.getErrors());
        ValidationErrorResponse response = new ValidationErrorResponse();
        response.setTimestamp(LocalDateTime.now());
        response.setStatus(HttpStatus.BAD_REQUEST.value());
        response.setError("Validation Failed");
        response.setMessage("Validation failed for request");
        response.setErrors(ex.getErrors());
        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST);
    }

    /**
     * Handle MethodArgumentNotValidException (from @Valid)
     */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ValidationErrorResponse> handleMethodArgumentNotValid(
            MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        ex.getBindingResult().getFieldErrors().forEach(error ->
            errors.put(error.getField(), error.getDefaultMessage())
        );

        ValidationErrorResponse response = new ValidationErrorResponse();
        response.setTimestamp(LocalDateTime.now());
        response.setStatus(HttpStatus.BAD_REQUEST.value());
        response.setError("Validation Failed");
        response.setMessage("Validation failed for request");
        response.setErrors(errors);

        return ResponseEntity.badRequest().body(response);
    }

    /**
     * Handle HttpMessageNotReadableException (malformed JSON)
     */
    @ExceptionHandler(HttpMessageNotReadableException.class)
```

```
    public ResponseEntity<ErrorResponse>
handleMalformedJson(HttpMessageNotReadableException ex) {
        ErrorResponse error = ErrorResponse.builder()
                .timestamp(LocalDateTime.now())
                .status(HttpStatus.BAD_REQUEST.value())
                .error("Bad Request")
                .message("Malformed JSON request")
                .path(getPath())
                .build();
        return ResponseEntity.badRequest().body(error);
    }

    /**
     * Handle AccessDeniedException (security)
     */
    @ExceptionHandler(AccessDeniedException.class)
    public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex) {
        ErrorResponse error = ErrorResponse.builder()
                .timestamp(LocalDateTime.now())
                .status(HttpStatus.FORBIDDEN.value())
                .error("Forbidden")
                .message("Access denied: " + ex.getMessage())
                .path(getPath())
                .build();
        return new ResponseEntity<>(error, HttpStatus.FORBIDDEN);
    }

    /**
     * Handle DataIntegrityViolationException (database constraints)
     */
    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse>
handleDataIntegrityViolation(DataIntegrityViolationException ex) {
        ErrorResponse error = ErrorResponse.builder()
                .timestamp(LocalDateTime.now())
                .status(HttpStatus.CONFLICT.value())
                .error("Conflict")
                .message("Data integrity violation: " + extractConstraintViolation(ex))
                .path(getPath())
                .build();
        return new ResponseEntity<>(error, HttpStatus.CONFLICT);
    }

    /**
     * Generic exception handler for all other exceptions
     */
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex,
HttpServletRequest request) {
        logger.error("Unexpected error occurred: ", ex);

        ErrorResponse error = ErrorResponse.builder()
                .timestamp(LocalDateTime.now())
                .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
```

```
                .error("Internal Server Error")
                .message("An unexpected error occurred")
                .path(request.getRequestURI())
                .build();

        // Only include exception details in development
        if (isDevelopmentProfile()) {
            error.setDebug(ex.getMessage());
        }

        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }

    private String getPath() {
        ServletRequestAttributes attributes = (ServletRequestAttributes)
RequestContextHolder.getRequestAttributes();
        if (attributes != null) {
            return attributes.getRequest().getRequestURI();
        }
        return "Unknown path";
    }

    private String extractConstraintViolation(DataIntegrityViolationException ex) {
        // Extract meaningful message from constraint violation
        if (ex.getCause() instanceof ConstraintViolationException) {
            return "Database constraint violation";
        }
        return "Data integrity error";
    }

    private boolean isDevelopmentProfile() {
        String[] activeProfiles = new SpringProfileUtils().getActiveProfiles();
        return Arrays.asList(activeProfiles).contains("dev");
    }
}

/**
 * Error Response DTOs
 */
@Data
@Builder
public class ErrorResponse {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;
    private String debug;
}

@Data
public class ValidationErrorResponse {
    private LocalDateTime timestamp;
    private int status;
```

```
    private String error;
    private String message;
    private Map<String, String> errors;
}

// ---- Controller-specific exception handling ----
@RestController
@RequestMapping("/api/items")
public class ItemController {

    /**
     * Local exception handling within controller
     */
    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleIllegalArgument(IllegalArgumentException ex) {
        // Only handles exceptions thrown within this controller
        return ResponseEntity.badRequest().body(new ErrorResponse(...));
    }
}
```

6.5 Content Negotiation

Hands-On Code:

java

CopyDownload

```
@RestController
@RequestMapping("/api/content")
public class ContentNegotiationController {

    /**
     * Content negotiation based on Accept header
     * Accept: application/json -> JSON response
     * Accept: application/xml -> XML response
     * Accept: text/html -> HTML response
     */
    @GetMapping(value = "/user",
            produces = {MediaType.APPLICATION_JSON_VALUE,
                    MediaType.APPLICATION_XML_VALUE,
                    MediaType.TEXT_HTML_VALUE})
    public User getUser() {
        return new User(1L, "John Doe", "john@example.com");
    }

    /**
     * Multiple produces with different content types
     */
    @GetMapping(value = "/report",
            produces = {MediaType.APPLICATION_PDF_VALUE,
                    MediaType.APPLICATION_JSON_VALUE,
                    MediaType.APPLICATION_XML_VALUE})
    public ResponseEntity<byte[]> getReport(@RequestParam String format) {
        // Based on the Accept header or format parameter
        byte[] data = reportService.generateReport(format);
        return ResponseEntity.ok(data);
```

```
    }

    /**
     * Content negotiation using file extension
     * /api/user.json -> JSON
     * /api/user.xml -> XML
     */
    @GetMapping(value = "/user.{format}",
            produces = {MediaType.APPLICATION_JSON_VALUE,
                    MediaType.APPLICATION_XML_VALUE})
    public User getUserWithFormat(@PathVariable String format) {
        return new User(1L, "John Doe", "john@example.com");
    }

    /**
     * Produces for different response formats
     */
    @GetMapping(value = "/product",
            produces = MediaType.APPLICATION_JSON_VALUE)
    public Product getProductAsJson() {
        return new Product(1L, "Laptop", 999.99);
    }

    @GetMapping(value = "/product",
            produces = MediaType.APPLICATION_XML_VALUE)
    public Product getProductAsXml() {
        return new Product(1L, "Laptop", 999.99);
    }

    /**
     * Consumes for different request formats
     */
    @PostMapping(value = "/product",
             consumes = MediaType.APPLICATION_JSON_VALUE)
    public Product createProductFromJson(@RequestBody Product product) {
        return productService.save(product);
    }

    @PostMapping(value = "/product",
             consumes = MediaType.APPLICATION_XML_VALUE)
    public Product createProductFromXml(@RequestBody Product product) {
        return productService.save(product);
    }
}

// ---- Content Negotiation Configuration ----
@Configuration
public class ContentNegotiationConfig implements WebMvcConfigurer {

    @Override
    public void configureContentNegotiation(ContentNegotiationConfigurer configurer) {
        // Configure content negotiation strategy
        configurer
            .favorParameter(true) // Enable format parameter: ?format=json
```

```
        .parameterName("format") // Default format parameter name
        .favorPathExtension(false) // Disable path extension
        .ignoreAcceptHeader(false) // Don't ignore Accept header
        .useRegisteredExtensionsOnly(true)
        .defaultContentType(MediaType.APPLICATION_JSON)
        .mediaType("json", MediaType.APPLICATION_JSON)
        .mediaType("xml", MediaType.APPLICATION_XML)
        .mediaType("html", MediaType.TEXT_HTML)
        .mediaType("pdf", MediaType.APPLICATION_PDF);
    }
}
```

---

7. Data Access in Spring Boot

7.1 Spring Data JPA

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.NotNull;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "products")
@Data
@NoArgsConstructor
@AllArgsConstructor
@EntityListeners(AuditingEntityListener.class)
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank
    @Column(nullable = false)
    private String name;

    private String description;
```

```
    @NotNull
    @Min(0)
    private Double price;

    @NotNull
    @Min(0)
    private Integer stockQuantity;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "category_id")
    private Category category;

    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<ProductReview> reviews = new ArrayList<>();

    @CreatedDate
    @Column(name = "created_at", updatable = false)
    private LocalDateTime createdAt;

    @LastModifiedDate
    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @Version
    private Long version; // For optimistic locking
}

// ---- Repository Interface with custom queries ----
package net.guides.springboot2.repository;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Modifying;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Repository
public interface ProductRepository extends JpaRepository<Product, Long> {

    // Derived query methods
    List<Product> findByName(String name);
    List<Product> findByNameContainingIgnoreCase(String name);
    Optional<Product> findByNameAndPrice(String name, Double price);
    List<Product> findByPriceBetween(Double minPrice, Double maxPrice);
    List<Product> findByStockQuantityLessThan(Integer minStock);
    List<Product> findByCategoryName(String categoryName);
    Page<Product> findByCategoryId(Long categoryId, Pageable pageable);
```

```
    List<Product> findByCategoryNameAndPriceGreaterThanEqual(String categoryName, Double
minPrice);

    // Custom JPQL queries
    @Query("SELECT p FROM Product p WHERE p.price > :minPrice ORDER BY p.price DESC")
    List<Product> findExpensiveProducts(@Param("minPrice") Double minPrice);

    @Query("SELECT p FROM Product p WHERE p.name LIKE %:keyword% OR p.description LIKE
%:keyword%")
    List<Product> searchByKeyword(@Param("keyword") String keyword);

    @Query("SELECT p FROM Product p WHERE p.category.id = :categoryId AND p.price
BETWEEN :minPrice AND :maxPrice")
    List<Product> findProductsByCategoryAndPriceRange(
        @Param("categoryId") Long categoryId,
        @Param("minPrice") Double minPrice,
        @Param("maxPrice") Double maxPrice
    );

    // Projections
    @Query("SELECT p.name as name, p.price as price FROM Product p WHERE p.id = :id")
    ProductSummary findProductSummaryById(@Param("id") Long id);

    // Pagination with custom query
    @Query("SELECT p FROM Product p WHERE p.price > :minPrice")
    Page<Product> findProductsAbovePrice(@Param("minPrice") Double minPrice, Pageable
pageable);

    // Native SQL queries
    @Query(value = "SELECT * FROM products p WHERE p.price > :minPrice AND p.stock_quantity
> :minStock",
           nativeQuery = true)
    List<Product> findAvailableProducts(@Param("minPrice") Double minPrice,
                                        @Param("minStock") Integer minStock);

    @Query(value = "SELECT COUNT(*) FROM products WHERE category_id = :categoryId",
           nativeQuery = true)
    long countProductsByCategory(@Param("categoryId") Long categoryId);

    // Update queries with @Modifying
    @Modifying
    @Transactional
    @Query("UPDATE Product p SET p.stockQuantity = :newStock WHERE p.id = :productId")
    int updateStockQuantity(@Param("productId") Long productId,
                            @Param("newStock") Integer newStock);

    @Modifying
    @Transactional
    @Query("UPDATE Product p SET p.price = p.price * (1 + :percentageIncrease/100) " +
           "WHERE p.category.id = :categoryId")
    int applyPriceIncrease(@Param("categoryId") Long categoryId,
                           @Param("percentageIncrease") Double percentageIncrease);

    @Modifying
```

```
    @Transactional
    @Query("DELETE FROM Product p WHERE p.stockQuantity = 0")
    int deleteOutOfStockProducts();

    // Boolean checks
    boolean existsByName(String name);

    long countByCategoryId(Long categoryId);
}

// ---- ProductSummary Projection Interface ----
public interface ProductSummary {
    String getName();
    Double getPrice();
}

// ---- Service Layer ----
package net.guides.springboot2.service;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@Transactional(readOnly = true)
public class ProductService {

    @Autowired
    private ProductRepository productRepository;

    @Autowired
    private CategoryRepository categoryRepository;

    /**
     * Create new product
     */
    @Transactional
    public Product createProduct(CreateProductRequest request) {
        // Check if product name already exists
        if (productRepository.existsByName(request.getName())) {
            throw new DuplicateResourceException("Product with name '" + request.getName() + "'
already exists");
        }

        // Get category
        Category category = categoryRepository.findById(request.getCategoryId())
            .orElseThrow(() -> new ResourceNotFoundException("Category",
request.getCategoryId()));

        // Create product
        Product product = new Product();
```

```
        product.setName(request.getName());
        product.setDescription(request.getDescription());
        product.setPrice(request.getPrice());
        product.setStockQuantity(request.getStockQuantity());
        product.setCategory(category);

        return productRepository.save(product);
    }

    /**
     * Get all products with pagination and sorting
     */
    public Page<Product> getProducts(int page, int size, String sortBy, String direction) {
        Sort.Direction sortDirection = Sort.Direction.fromString(direction.toUpperCase());
        Pageable pageable = PageRequest.of(page, size, sortDirection, sortBy);
        return productRepository.findAll(pageable);
    }

    /**
     * Get product by ID
     */
    public Product getProduct(Long id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product", id));
    }

    /**
     * Search products with multiple criteria
     */
    public List<Product> searchProducts(String keyword, Double minPrice, Double maxPrice, Long
categoryId) {
        // Complex search logic combining multiple conditions
        if (keyword != null && !keyword.isEmpty()) {
            return productRepository.searchByKeyword(keyword);
        } else if (minPrice != null && maxPrice != null && categoryId != null) {
            return productRepository.findProductsByCategoryAndPriceRange(categoryId, minPrice,
maxPrice);
        } else if (minPrice != null) {
            return productRepository.findExpensiveProducts(minPrice);
        } else if (categoryId != null) {
            return productRepository.findByCategoryId(categoryId, Pageable.unpaged()).getContent();
        }
        return productRepository.findAll();
    }

    /**
     * Update product with optimistic locking
     */
    @Transactional
    public Product updateProduct(Long id, UpdateProductRequest request) {
        Product product = getProduct(id);

        // Version check for optimistic locking
        if (request.getVersion() != null && !request.getVersion().equals(product.getVersion())) {
```

```
            throw new OptimisticLockingException("Product was modified by another user");
        }

        // Update fields
        if (request.getName() != null) {
            product.setName(request.getName());
        }
        if (request.getDescription() != null) {
            product.setDescription(request.getDescription());
        }
        if (request.getPrice() != null) {
            product.setPrice(request.getPrice());
        }
        if (request.getStockQuantity() != null) {
            product.setStockQuantity(request.getStockQuantity());
        }
        if (request.getCategoryId() != null) {
            Category category = categoryRepository.findById(request.getCategoryId())
                .orElseThrow(() -> new ResourceNotFoundException("Category",
request.getCategoryId()));
            product.setCategory(category);
        }

        return productRepository.save(product);
    }

    /**
     * Bulk update stock quantities
     */
    @Transactional
    public void bulkUpdateStock(Map<Long, Integer> stockUpdates) {
        stockUpdates.forEach((productId, newStock) -> {
            int updated = productRepository.updateStockQuantity(productId, newStock);
            if (updated == 0) {
                throw new ResourceNotFoundException("Product", productId);
            }
        });
    }

    /**
     * Apply price increase to category
     */
    @Transactional
    public int applyCategoryPriceIncrease(Long categoryId, Double percentage) {
        return productRepository.applyPriceIncrease(categoryId, percentage);
    }

    /**
     * Delete product
     */
    @Transactional
    public void deleteProduct(Long id) {
        if (!productRepository.existsById(id)) {
            throw new ResourceNotFoundException("Product", id);
```

```
        }
        productRepository.deleteById(id);
    }
}
```

7.2 JDBC with Spring Boot

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.dao;

import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.jdbc.core.RowMapper;
import org.springframework.jdbc.support.GeneratedKeyHolder;
import org.springframework.jdbc.support.KeyHolder;
import org.springframework.stereotype.Repository;

import java.sql.PreparedStatement;
import java.sql.Statement;
import java.util.List;
import java.util.Optional;

@Repository
public class ProductJdbcDao {

    private final JdbcTemplate jdbcTemplate;

    // RowMapper for converting ResultSet to Product
    private final RowMapper<Product> productRowMapper = (rs, rowNum) -> {
        Product product = new Product();
        product.setId(rs.getLong("id"));
        product.setName(rs.getString("name"));
        product.setDescription(rs.getString("description"));
        product.setPrice(rs.getDouble("price"));
        product.setStockQuantity(rs.getInt("stock_quantity"));
        product.setCreatedAt(rs.getTimestamp("created_at").toLocalDateTime());
        product.setUpdatedAt(rs.getTimestamp("updated_at").toLocalDateTime());
        return product;
    };

    public ProductJdbcDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    /**
     * Create product using JDBC with generated key
     */
    public Product save(Product product) {
        String sql = "INSERT INTO products (name, description, price, stock_quantity, category_id) " +
                     "VALUES (?, ?, ?, ?, ?)";

        KeyHolder keyHolder = new GeneratedKeyHolder();
```

```
        jdbcTemplate.update(connection -> {
            PreparedStatement ps = connection.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS);
            ps.setString(1, product.getName());
            ps.setString(2, product.getDescription());
            ps.setDouble(3, product.getPrice());
            ps.setInt(4, product.getStockQuantity());
            ps.setObject(5, product.getCategoryId());
            return ps;
        }, keyHolder);

        Number key = keyHolder.getKey();
        if (key != null) {
            product.setId(key.longValue());
        }

        return product;
    }

    /**
     * Find product by ID using RowMapper
     */
    public Optional<Product> findById(Long id) {
        String sql = "SELECT * FROM products WHERE id = ?";
        try {
            Product product = jdbcTemplate.queryForObject(sql, productRowMapper, id);
            return Optional.of(product);
        } catch (Exception e) {
            return Optional.empty();
        }
    }

    /**
     * Find all products
     */
    public List<Product> findAll() {
        String sql = "SELECT * FROM products ORDER BY id";
        return jdbcTemplate.query(sql, productRowMapper);
    }

    /**
     * Find products by category
     */
    public List<Product> findByCategory(Long categoryId) {
        String sql = "SELECT * FROM products WHERE category_id = ? ORDER BY name";
        return jdbcTemplate.query(sql, productRowMapper, categoryId);
    }

    /**
     * Find products with low stock
     */
    public List<Product> findLowStock(int threshold) {
        String sql = "SELECT * FROM products WHERE stock_quantity < ? ORDER BY stock_quantity";
        return jdbcTemplate.query(sql, productRowMapper, threshold);
```

```
    }

    /**
     * Find products by name pattern
     */
    public List<Product> searchByName(String namePattern) {
        String sql = "SELECT * FROM products WHERE name LIKE ?";
        return jdbcTemplate.query(sql, productRowMapper, "%" + namePattern + "%");
    }

    /**
     * Update product
     */
    public int update(Product product) {
        String sql = "UPDATE products SET name = ?, description = ?, price = ?, " +
                     "stock_quantity = ?, category_id = ?, updated_at = CURRENT_TIMESTAMP " +
                     "WHERE id = ?";
        return jdbcTemplate.update(sql,
            product.getName(),
            product.getDescription(),
            product.getPrice(),
            product.getStockQuantity(),
            product.getCategoryId(),
            product.getId()
        );
    }

    /**
     * Update stock using named parameters
     */
    public int updateStock(Long productId, int newStock) {
        String sql = "UPDATE products SET stock_quantity = ?, updated_at = CURRENT_TIMESTAMP
" +
                     "WHERE id = ?";
        return jdbcTemplate.update(sql, newStock, productId);
    }

    /**
     * Delete product
     */
    public int deleteById(Long id) {
        String sql = "DELETE FROM products WHERE id = ?";
        return jdbcTemplate.update(sql, id);
    }

    /**
     * Batch insert multiple products
     */
    public int[] batchInsert(List<Product> products) {
        String sql = "INSERT INTO products (name, description, price, stock_quantity, category_id) " +
                     "VALUES (?, ?, ?, ?, ?)";

        return jdbcTemplate.batchUpdate(sql, products, products.size(),
            (ps, product) -> {
```

```
            ps.setString(1, product.getName());
            ps.setString(2, product.getDescription());
            ps.setDouble(3, product.getPrice());
            ps.setInt(4, product.getStockQuantity());
            ps.setObject(5, product.getCategoryId());
        });
    }

    /**
     * Batch update with different operations
     */
    public int[] batchUpdate(List<Long> productIds, double percentageIncrease) {
        String sql = "UPDATE products SET price = price * ? WHERE id = ?";
        return jdbcTemplate.batchUpdate(sql, productIds, productIds.size(),
            (ps, id) -> {
                ps.setDouble(1, 1 + percentageIncrease / 100);
                ps.setLong(2, id);
            });
    }

    /**
     * Execute a stored procedure
     */
    public void applyCategoryDiscount(Long categoryId, Double discountPercentage) {
        String sql = "CALL apply_category_discount(?, ?)";
        jdbcTemplate.update(sql, categoryId, discountPercentage);
    }

    /**
     * Query with custom projection
     */
    public List<ProductProjection> getProductProjections() {
        String sql = "SELECT p.id, p.name, c.name as category_name, p.price " +
                     "FROM products p JOIN categories c ON p.category_id = c.id";

        return jdbcTemplate.query(sql, (rs, rowNum) ->
            new ProductProjection(
                rs.getLong("id"),
                rs.getString("name"),
                rs.getString("category_name"),
                rs.getDouble("price")
            ));
    }
}
```

7.3 MongoDB / NoSQL

Hands-On Code:

java

CopyDownload

```
// ---- MongoDB Document ----
package net.guides.springboot2.document;

import org.springframework.data.annotation.Id;
```

```
import org.springframework.data.mongodb.core.mapping.Document;
import org.springframework.data.mongodb.core.mapping.Field;

import java.time.LocalDateTime;
import java.util.List;

@Document(collection = "products")
public class ProductDocument {

    @Id
    private String id;

    private String name;
    private String description;
    private Double price;
    private Integer stockQuantity;

    @Field("category_id")
    private String categoryId;

    private List<String> tags;
    private ProductMetadata metadata;
    private List<Review> reviews;

    @Field("created_at")
    private LocalDateTime createdAt;

    @Field("updated_at")
    private LocalDateTime updatedAt;

    // Nested documents
    public static class ProductMetadata {
        private String manufacturer;
        private String modelNumber;
        private Double weight;
        private String color;
        private Map<String, String> specifications;
    }

    public static class Review {
        private String userId;
        private Integer rating;
        private String comment;
        private LocalDateTime reviewDate;
    }
}

// ---- MongoDB Repository ----
package net.guides.springboot2.repository;

import org.springframework.data.mongodb.repository.MongoRepository;
import org.springframework.data.mongodb.repository.Query;
import org.springframework.stereotype.Repository;
```

```
import java.util.List;

@Repository
public interface ProductMongoRepository extends MongoRepository<ProductDocument, String>
{

   // Derived query methods
   List<ProductDocument> findByName(String name);
   List<ProductDocument> findByPriceBetween(Double minPrice, Double maxPrice);
   List<ProductDocument> findByTagsContaining(String tag);
   List<ProductDocument> findByMetadataManufacturer(String manufacturer);

   // Custom queries with MongoDB JSON query
   @Query("{ 'price' : { $gt: ?0 } }")
   List<ProductDocument> findProductsAbovePrice(Double minPrice);

   @Query("{ $and: [ " +
         "{ 'price' : { $gt: ?0 } }, " +
         "{ 'price' : { $lt: ?1 } }, " +
         "{ 'metadata.weight' : { $lt: ?2 } } " +
         "] }")
   List<ProductDocument> findProductsByPriceRangeAndWeight(
      Double minPrice, Double maxPrice, Double maxWeight);

   @Query(value = "{ 'tags' : { $in: ?0 } }", fields = "{ 'name' : 1, 'price' : 1, 'tags' : 1 }")
   List<ProductDocument> findProductNamesByTags(List<String> tags);

   @Query(value = "{ 'reviews.rating' : { $gt: ?0 } }")
   List<ProductDocument> findProductsWithHighRatings(Integer minRating);

   // Aggregation pipeline using @Aggregation
   @Aggregation(pipeline = {
      "{ $group: { _id: '$metadata.manufacturer', totalProducts: { $sum: 1 }, avgPrice: { $avg:
'$price' } } }",
      "{ $sort: { totalProducts: -1 } }"
   })
   List<ManufacturerStats> getManufacturerStats();
}

// ---- Service Layer ----
package net.guides.springboot2.service;

import org.springframework.stereotype.Service;

@Service
public class ProductMongoService {

   @Autowired
   private ProductMongoRepository productRepository;

   public ProductDocument createProduct(ProductDocument product) {
      product.setCreatedAt(LocalDateTime.now());
      product.setUpdatedAt(LocalDateTime.now());
      return productRepository.save(product);
```

```
    }

    public ProductDocument getProduct(String id) {
        return productRepository.findById(id)
            .orElseThrow(() -> new ResourceNotFoundException("Product", id));
    }

    public List<ProductDocument> searchProducts(Double minPrice, Double maxPrice, String tag)
{
        if (tag != null && !tag.isEmpty()) {
            return productRepository.findByTagsContaining(tag);
        } else if (minPrice != null && maxPrice != null) {
            return productRepository.findByPriceBetween(minPrice, maxPrice);
        } else if (minPrice != null) {
            return productRepository.findProductsAbovePrice(minPrice);
        }
        return productRepository.findAll();
    }

    public List<ManufacturerStats> getManufacturerStatistics() {
        return productRepository.getManufacturerStats();
    }

    public void deleteProduct(String id) {
        productRepository.deleteById(id);
    }
}
```

7.4 Transactions (@Transactional)

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Propagation;
import org.springframework.transaction.annotation.Transactional;

@Service
public class OrderProcessingService {

    @Autowired
    private OrderRepository orderRepository;

    @Autowired
    private ProductRepository productRepository;

    @Autowired
    private PaymentService paymentService;

    @Autowired
    private InventoryService inventoryService;
```

```
/**
 * Basic transactional method
 * All operations succeed or all fail
 */
@Transactional
public Order processOrder(OrderRequest request) {
    // 1. Create order
    Order order = new Order();
    order.setCustomerId(request.getCustomerId());
    order.setItems(request.getItems());
    order.setStatus(OrderStatus.PENDING);
    order = orderRepository.save(order);

    // 2. Reserve inventory
    inventoryService.reserveItems(order.getItems());

    // 3. Process payment
    PaymentResult payment = paymentService.processPayment(
        request.getPaymentMethod(),
        order.getTotalAmount()
    );

    if (!payment.isSuccessful()) {
        throw new PaymentProcessingException("Payment failed");
    }

    // 4. Update order status
    order.setStatus(OrderStatus.COMPLETED);
    order.setPaymentId(payment.getTransactionId());

    return orderRepository.save(order);
    // If any exception occurs before this point, everything is rolled back
}

/**
 * Transaction with rollback for specific exceptions
 */
@Transactional(rollbackFor = {InventoryException.class, PaymentException.class})
public Order processOrderWithRollback(OrderRequest request) {
    // This will rollback for InventoryException and PaymentException
    // But not for other exceptions
    return processOrder(request);
}

/**
 * Transaction without rollback for specific exceptions
 */
@Transactional(noRollbackFor = {ValidationException.class})
public Order createOrderWithValidation(OrderRequest request) {
    // Validation exceptions won't cause rollback
    validateOrder(request);
    return processOrder(request);
}
```

```
/**
 * Transaction with propagation settings
 */
@Transactional(propagation = Propagation.REQUIRED)
public void updateOrderStatus(Long orderId, OrderStatus status) {
    // REQUIRED: Uses existing transaction or creates a new one
    orderRepository.updateStatus(orderId, status);
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logOrderEvent(Long orderId, String event) {
    // REQUIRES_NEW: Always creates a new transaction
    // Suspends existing transaction if any
    orderRepository.logEvent(orderId, event);
}

@Transactional(propagation = Propagation.MANDATORY)
public void processInExistingTransaction(Order order) {
    // MANDATORY: Must be called within an existing transaction
    // Throws exception if no transaction exists
    orderRepository.save(order);
}

@Transactional(propagation = Propagation.NESTED)
public void nestedTransactionExample(Order order) {
    // NESTED: Creates a savepoint within existing transaction
    // Can rollback to savepoint if needed
    try {
        orderRepository.save(order);
        processOrderItems(order.getItems());
    } catch (Exception e) {
        // This will rollback only the nested transaction
        // The outer transaction can still be committed
    }
}

/**
 * Transaction with isolation levels
 */
@Transactional(isolation = Isolation.READ_COMMITTED)
public void readCommittedTransaction() {
    // READ_COMMITTED: Prevents dirty reads
    // Allows non-repeatable reads
    Order order = orderRepository.findById(1L).orElseThrow();
    order.setStatus(OrderStatus.PROCESSING);
    orderRepository.save(order);
}

@Transactional(isolation = Isolation.REPEATABLE_READ)
public Order getOrderForUpdate(Long orderId) {
    // REPEATABLE_READ: Prevents dirty reads and non-repeatable reads
    // During this transaction, other transactions cannot modify the same data
    return orderRepository.findById(orderId).orElseThrow();
```

```
}

@Transactional(isolation = Isolation.SERIALIZABLE)
public void serializableTransaction(Order order) {
    // SERIALIZABLE: Highest isolation level
    // Executes transactions as if they were serial
    // Prevents phantom reads
    orderRepository.save(order);
}

/**
 * Transaction with timeout
 */
@Transactional(timeout = 30) // 30 seconds timeout
public void longRunningTransaction() {
    // Will throw exception if transaction runs longer than 30 seconds
    // Prevents long-running operations from holding locks
    try {
        Thread.sleep(25000); // Under timeout
        // Process data...
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

/**
 * Read-only transaction for optimization
 */
@Transactional(readOnly = true)
public List<Order> getOrdersForCustomer(Long customerId) {
    // readOnly = true provides optimization
    // No modifications allowed in this transaction
    return orderRepository.findByCustomerId(customerId);
}

/**
 * Transaction with multiple data sources
 */
@Transactional(transactionManager = "jpaTransactionManager")
public void processWithJpaTransaction() {
    // Using JPA transaction manager
    productRepository.save(new Product());
    orderRepository.save(new Order());
}

@Transactional(transactionManager = "jdbcTransactionManager")
public void processWithJdbcTransaction() {
    // Using JDBC transaction manager
    jdbcTemplate.update("INSERT INTO products VALUES (?)", "Product");
}

/**
 * Manual transaction management
 */
```

```
    @Autowired
    private PlatformTransactionManager transactionManager;

    public void manualTransactionExample() {
        TransactionDefinition def = new DefaultTransactionDefinition();
        TransactionStatus status = transactionManager.getTransaction(def);

        try {
            // Perform operations
            orderRepository.save(new Order());
            productRepository.updateStock(1L, 10);

            // Commit transaction
            transactionManager.commit(status);
        } catch (Exception e) {
            // Rollback transaction
            transactionManager.rollback(status);
            throw e;
        }
    }
}
```

---

8. Database Configuration

8.1 H2 In-Memory Database

Hands-On Code:

xml

CopyDownloadRun

```
<!-- Maven Dependency -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>
```

yaml

CopyDownload

```
# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb
    driver-class-name: org.h2.Driver
    username: sa
    password:

  h2:
    console:
      enabled: true
      path: /h2-console
      settings:
        web-allow-others: true
```

```
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.H2Dialect
        format_sql: true
    defer-datasource-initialization: true

  sql:
    init:
      mode: always
      schema-locations: classpath:db/schema.sql
      data-locations: classpath:db/data.sql
```

8.2 MySQL/PostgreSQL Setup

Hands-On Code:

xml

CopyDownloadRun

```
<!-- MySQL Dependency -->
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- PostgreSQL Dependency -->
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

yaml

CopyDownload

```
# application-prod.yml (MySQL Configuration)
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC
    username: root
    password: ${MYSQL_PASSWORD:secret}
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 20
      minimum-idle: 10
      idle-timeout: 30000
      connection-timeout: 30000
      max-lifetime: 1800000
      pool-name: MySQLHikariCP

  jpa:
    hibernate:
      ddl-auto: validate
```

```
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
        format_sql: true
        use_sql_comments: true
        jdbc:
          batch_size: 50
          fetch_size: 100

# PostgreSQL Configuration
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/mydb
    username: postgres
    password: ${POSTGRES_PASSWORD:secret}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 20
      minimum-idle: 10

  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        jdbc:
          lob:
            non_contextual_creation: true
```

8.3 Connection Pooling (HikariCP)

Hands-On Code:

yaml

CopyDownload

```
# HikariCP Configuration
spring:
  datasource:
    hikari:
      # Maximum number of connections in the pool
      maximum-pool-size: 20

      # Minimum number of idle connections
      minimum-idle: 10

      # Maximum time (ms) a connection can be idle before being removed
      idle-timeout: 300000

      # Maximum time (ms) a connection can live
      max-lifetime: 1800000

      # Maximum time (ms) to wait for a connection from the pool
      connection-timeout: 30000
```

```
    # Name of the pool
    pool-name: MyAppHikariCP

    # Enable JMX monitoring
    register-mbeans: true

    # Connection test query
    connection-test-query: SELECT 1

    # Validation timeout
    validation-timeout: 5000

    # Leak detection threshold (ms)
    leak-detection-threshold: 60000

    # Allow pool suspension
    allow-pool-suspension: false

    # Auto-commit for connections
    auto-commit: true

    # Connection init SQL
    connection-init-sql: SET NAMES utf8mb4

    # Metrics and monitoring
    metric-registry:
    health-check-registry:
```

8.4 Schema Initialization

Hands-On Code:

sql

CopyDownload

```
-- src/main/resources/db/schema.sql
-- H2 / MySQL / PostgreSQL compatible schema

CREATE TABLE IF NOT EXISTS categories (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(500),
    parent_id BIGINT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (parent_id) REFERENCES categories(id)
);

CREATE TABLE IF NOT EXISTS products (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
```

```
    category_id BIGINT NOT NULL,
    version BIGINT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(id),
    INDEX idx_products_category (category_id),
    INDEX idx_products_price (price)
);

CREATE TABLE IF NOT EXISTS orders (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    customer_id BIGINT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2) NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
    payment_id VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS order_items (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    order_id BIGINT NOT NULL,
    product_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    total_price DECIMAL(10,2) AS (quantity * unit_price) STORED,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(id),
    INDEX idx_order_items_product (product_id)
);
sql
```

CopyDownload

```
-- src/main/resources/db/data.sql
-- Initial test data

-- Categories
INSERT INTO categories (id, name, description) VALUES
(1, 'Electronics', 'Electronic devices and accessories'),
(2, 'Books', 'Books and publications'),
(3, 'Clothing', 'Apparel and fashion items'),
(4, 'Home & Garden', 'Home improvement and garden supplies'),
(5, 'Sports', 'Sports equipment and accessories');

-- Products
INSERT INTO products (name, description, price, stock_quantity, category_id) VALUES
('Smartphone', 'Latest generation smartphone with 5G support', 699.99, 50, 1),
('Laptop', 'High-performance laptop with 16GB RAM', 1299.99, 30, 1),
('Tablet', '10-inch tablet for entertainment', 399.99, 40, 1),
('Programming Book', 'Complete guide to Java programming', 49.99, 100, 2),
('Cookbook', 'Delicious recipes for beginners', 29.99, 75, 2),
```

```
('T-Shirt', 'Cotton t-shirt for everyday use', 19.99, 200, 3),
('Jeans', 'Classic denim jeans', 49.99, 150, 3),
('Garden Tools', 'Set of essential garden tools', 39.99, 80, 4),
('Plant Pot', 'Modern ceramic plant pot', 14.99, 120, 4),
('Football', 'Professional football for all weather', 24.99, 60, 5),
('Tennis Racket', 'Lightweight tennis racket', 79.99, 45, 5),
('Yoga Mat', 'Non-slip yoga exercise mat', 34.99, 90, 5);

-- Coupons and discounts
INSERT INTO coupons (code, discount_type, discount_value, expiry_date, active) VALUES
('SAVE10', 'PERCENTAGE', 10.00, DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY), true),
('SAVE20', 'PERCENTAGE', 20.00, DATE_ADD(CURRENT_DATE, INTERVAL 15 DAY), true),
('FREESHIP', 'FIXED', 5.00, DATE_ADD(CURRENT_DATE, INTERVAL 60 DAY), true);
```

java

CopyDownload

```
// ---- Programmatic Schema Initialization ----
@Configuration
public class DatabaseInitializer {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    @Autowired
    private DataSource dataSource;

    @PostConstruct
    public void initializeDatabase() {
        try {
            // Check if tables exist
            String checkTableSql = "SELECT COUNT(*) FROM information_schema.tables " +
                                   "WHERE table_name = 'products'";
            int count = jdbcTemplate.queryForObject(checkTableSql, Integer.class);

            if (count == 0) {
                // Create tables programmatically
                createTables();
                // Insert initial data
                insertInitialData();
                System.out.println("Database initialized successfully");
            } else {
                System.out.println("Database already initialized");
            }
        } catch (Exception e) {
            System.err.println("Error initializing database: " + e.getMessage());
        }
    }

    private void createTables() {
        String createProductsTable = """
            CREATE TABLE products (
                id BIGINT PRIMARY KEY AUTO_INCREMENT,
                name VARCHAR(200) NOT NULL,
                description TEXT,
                price DECIMAL(10,2) NOT NULL,
```

```
            stock_quantity INT NOT NULL DEFAULT 0,
            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
        """;
    jdbcTemplate.execute(createProductsTable);
  }

  private void insertInitialData() {
    String insertSql = "INSERT INTO products (name, price, stock_quantity) VALUES (?, ?, ?)";
    jdbcTemplate.update(insertSql, "Sample Product", 19.99, 100);
  }
}
```

---

9. Validation in Spring Boot

9.1 Bean Validation

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.dto;

import jakarta.validation.constraints.*;
import java.time.LocalDate;
import java.util.List;

public class UserRegistrationDto {

  @NotBlank(message = "First name is required")
  @Size(min = 2, max = 50, message = "First name must be between 2 and 50 characters")
  @Pattern(regexp = "^[a-zA-Z\\s-]+$", message = "First name can only contain letters, spaces,
and hyphens")
  private String firstName;

  @NotBlank(message = "Last name is required")
  @Size(min = 2, max = 50, message = "Last name must be between 2 and 50 characters")
  @Pattern(regexp = "^[a-zA-Z\\s-]+$", message = "Last name can only contain letters, spaces,
and hyphens")
  private String lastName;

  @NotBlank(message = "Email is required")
  @Email(message = "Invalid email format")
  @Size(max = 100, message = "Email must not exceed 100 characters")
  private String email;

  @NotBlank(message = "Password is required")
  @Size(min = 8, max = 100, message = "Password must be between 8 and 100 characters")
  @Pattern(
    regexp = "^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z])(?=.*[@#$%^&+=]).*$",
    message = "Password must contain at least one digit, one uppercase, one lowercase, and
one special character"
  )
  private String password;
```

```
    @NotBlank(message = "Confirm password is required")
    @Size(min = 8, max = 100)
    private String confirmPassword;

    @NotNull(message = "Date of birth is required")
    @Past(message = "Date of birth must be in the past")
    @Min(value = 18, message = "User must be at least 18 years old")
    private LocalDate dateOfBirth;

    @Pattern(regexp = "^\\+?[0-9\\s-]{10,15}$", message = "Invalid phone number format")
    private String phoneNumber;

    @NotNull(message = "Terms acceptance is required")
    @AssertTrue(message = "You must accept the terms and conditions")
    private Boolean acceptTerms;

    // Getters and Setters...
}
```

9.2 Using @Valid and @Validated

Hands-On Code:

java

CopyDownload

```
@RestController
@RequestMapping("/api/users")
@Validated // For method-level validation
public class UserController {

    @Autowired
    private UserService userService;

    /**
     * @Valid triggers validation on the request body
     */
    @PostMapping
    public ResponseEntity<User> createUser(@Valid @RequestBody UserRegistrationDto dto) {
        User user = userService.registerUser(dto);
        return ResponseEntity.status(HttpStatus.CREATED).body(user);
    }

    /**
     * @Valid with nested objects
     */
    @PostMapping("/orders")
    public ResponseEntity<Order> createOrder(@Valid @RequestBody OrderRequest request) {
        return ResponseEntity.ok(orderService.createOrder(request));
    }

    /**
     * Method-level validation for path variables and query parameters
     */
    @GetMapping("/{id}")
```

```
    public ResponseEntity<User> getUserById(
            @PathVariable @Min(1) Long id,
            @RequestParam @Pattern(regexp = "^[A-Za-z]{2}$") String countryCode) {
        return userService.findById(id)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * Validating path variables
     */
    @GetMapping("/email/{email}")
    public ResponseEntity<User> getUserByEmail(
            @PathVariable @Email String email) {
        return userService.findByEmail(email)
                .map(ResponseEntity::ok)
                .orElse(ResponseEntity.notFound().build());
    }

    /**
     * Validating query parameters
     */
    @GetMapping("/search")
    public List<User> searchUsers(
            @RequestParam(required = false)
            @Size(min = 2, max = 50) String name,
            @RequestParam(defaultValue = "0")
            @Min(0) int page,
            @RequestParam(defaultValue = "10")
            @Min(1) @Max(100) int size) {
        return userService.searchUsers(name, page, size);
    }
}

/**
 * Example with nested objects
 */
@Data
public class OrderRequest {

    @NotNull(message = "User ID is required")
    @Min(1)
    private Long userId;

    @NotEmpty(message = "Order must contain at least one item")
    @Valid // Validate each item in the list
    private List<OrderItemRequest> items;

    @Valid // Validate nested object
    private ShippingAddress shippingAddress;
}

@Data
public class OrderItemRequest {
```

```
    @NotNull(message = "Product ID is required")
    @Min(1)
    private Long productId;

    @NotNull(message = "Quantity is required")
    @Min(1)
    @Max(999)
    private Integer quantity;
}

@Data
public class ShippingAddress {
    @NotBlank(message = "Street address is required")
    @Size(max = 200)
    private String street;

    @NotBlank(message = "City is required")
    @Size(max = 100)
    private String city;

    @NotBlank(message = "State is required")
    @Size(max = 100)
    private String state;

    @NotBlank(message = "Zip code is required")
    @Pattern(regexp = "^\\d{5}(-\\d{4})?$", message = "Invalid zip code format")
    private String zipCode;

    @NotBlank(message = "Country is required")
    @Size(max = 100)
    private String country;
}
```

9.3 Custom Validators

Hands-On Code:

java

CopyDownload

```
// ---- Custom Annotation ----
package net.guides.springboot2.validation;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import java.lang.annotation.*;

@Target({ElementType.FIELD, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UsernameValidator.class)
@Documented
public @interface ValidUsername {
    String message() default "Username must be between 5-20 characters and contain only letters,
numbers, and underscores";

    Class<?>[] groups() default {};
```

```
    Class<? extends Payload>[] payload() default {};

    int min() default 5;
    int max() default 20;
}

// ---- Custom Validator Implementation ----
package net.guides.springboot2.validation;

import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;

public class UsernameValidator implements ConstraintValidator<ValidUsername, String> {

    private int min;
    private int max;

    @Override
    public void initialize(ValidUsername constraintAnnotation) {
        this.min = constraintAnnotation.min();
        this.max = constraintAnnotation.max();
    }

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) {
            return false; // Not blank validation should handle null
        }

        // Validate length
        if (value.length() < min || value.length() > max) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                String.format("Username must be between %d and %d characters", min, max)
            ).addConstraintViolation();
            return false;
        }

        // Validate allowed characters (letters, numbers, underscore)
        if (!value.matches("^[a-zA-Z0-9_]+$")) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                "Username can only contain letters, numbers, and underscores"
            ).addConstraintViolation();
            return false;
        }

        // Validate no consecutive underscores
        if (value.matches(".*__.*")) {
            context.disableDefaultConstraintViolation();
            context.buildConstraintViolationWithTemplate(
                "Username cannot contain consecutive underscores"
            ).addConstraintViolation();
```

```
            return false;
        }

        return true;
    }
}

// ---- Cross-field Validation ----
package net.guides.springboot2.validation;

import jakarta.validation.Constraint;
import jakarta.validation.Payload;

@Target({ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = PasswordMatchValidator.class)
@Documented
public @interface PasswordMatch {
    String message() default "Passwords do not match";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

// ---- Cross-field Validator ----
public class PasswordMatchValidator implements ConstraintValidator<PasswordMatch, Object>
{

    @Override
    public boolean isValid(Object value, ConstraintValidatorContext context) {
        if (value == null) {
            return true;
        }

        // Assuming the object has getPassword() and getConfirmPassword() methods
        String password = null;
        String confirmPassword = null;

        try {
            // Use reflection or cast to known type
            password = (String) value.getClass().getMethod("getPassword").invoke(value);
            confirmPassword = (String)
value.getClass().getMethod("getConfirmPassword").invoke(value);
        } catch (Exception e) {
            return false;
        }

        if (password == null) {
            return confirmPassword == null;
        }

        return password.equals(confirmPassword);
    }
}
```

```
// ---- Usage ----
@PasswordMatch(message = "Passwords must match")
public class UserRegistrationDto {
    @ValidUsername
    private String username;

    private String password;
    private String confirmPassword;

    // Getters and Setters...
}

// ---- Custom Validation with Spring Context ----
@Target({ElementType.FIELD})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = UniqueEmailValidator.class)
public @interface UniqueEmail {
    String message() default "Email already exists in the system";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class UniqueEmailValidator implements ConstraintValidator<UniqueEmail, String> {

    @Autowired
    private UserRepository userRepository;

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        if (value == null) {
            return true;
        }
        return !userRepository.existsByEmail(value);
    }
}

// ---- Custom Validation with Groups ----
public interface CreateGroup {}
public interface UpdateGroup {}

public class ProductDto {

    @NotNull(groups = UpdateGroup.class)
    private Long id;

    @NotBlank(groups = {CreateGroup.class, UpdateGroup.class})
    @Size(min = 3, max = 100, groups = {CreateGroup.class, UpdateGroup.class})
    private String name;

    @Min(value = 0, groups = {CreateGroup.class, UpdateGroup.class})
    private Double price;

    // Getters and Setters...
}
```

```
@RestController
public class ProductController {

    @PostMapping("/products")
    public ResponseEntity<Product> createProduct(
            @Validated(CreateGroup.class) @RequestBody ProductDto dto) {
        // Create product
    }

    @PutMapping("/products/{id}")
    public ResponseEntity<Product> updateProduct(
            @PathVariable Long id,
            @Validated(UpdateGroup.class) @RequestBody ProductDto dto) {
        // Update product
    }
}
```

---

10. Security in Spring Boot

10.1 Spring Security Basics

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

java

CopyDownload

```
// ---- Basic Security Configuration ----
@Configuration
@EnableWebSecurity
public class BasicSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/", "/home", "/public/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutSuccessUrl("/")
```

```
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .permitAll()
        )
        .rememberMe(remember -> remember
            .key("uniqueAndSecretKey")
            .tokenValiditySeconds(86400)
        );

    return http.build();
  }

  @Bean
  public InMemoryUserDetailsManager userDetailsService() {
    UserDetails user = User.builder()
      .username("user")
      .password(passwordEncoder().encode("password"))
      .roles("USER")
      .build();

    UserDetails admin = User.builder()
      .username("admin")
      .password(passwordEncoder().encode("admin123"))
      .roles("ADMIN")
      .build();

    return new InMemoryUserDetailsManager(user, admin);
  }

  @Bean
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }
}
```

10.2 Authentication and Authorization

Hands-On Code:

java

CopyDownload

```
// ---- Custom UserDetails Service ----
@Service
public class CustomUserDetailsService implements UserDetailsService {

  @Autowired
  private UserRepository userRepository;

  @Override
  public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
    User user = userRepository.findByEmail(username)
      .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

    Set<GrantedAuthority> authorities = user.getRoles().stream()
```

```
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role.getName()))
            .collect(Collectors.toSet());

        return new org.springframework.security.core.userdetails.User(
            user.getEmail(),
            user.getPassword(),
            user.isActive(),
            true,
            true,
            true,
            authorities
        );
    }
}

// ---- Custom Authentication Provider ----
@Component
public class CustomAuthenticationProvider implements AuthenticationProvider {

    @Autowired
    private UserDetailsService userDetailsService;

    @Autowired
    private PasswordEncoder passwordEncoder;

    @Override
    public Authentication authenticate(Authentication authentication) throws
AuthenticationException {
        String username = authentication.getName();
        String password = authentication.getCredentials().toString();

        UserDetails user = userDetailsService.loadUserByUsername(username);

        if (passwordEncoder.matches(password, user.getPassword())) {
            return new UsernamePasswordAuthenticationToken(
                user,
                password,
                user.getAuthorities()
            );
        } else {
            throw new BadCredentialsException("Invalid credentials");
        }
    }

    @Override
    public boolean supports(Class<?> authentication) {
        return UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication);
    }
}

// ---- Security Configuration with Custom Authentication ----
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

```
    @Autowired
    private CustomAuthenticationProvider authProvider;

    @Autowired
    private JwtAuthenticationFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/manager/**").hasAnyRole("ADMIN", "MANAGER")
                .requestMatchers("/api/user/**").authenticated()
                .anyRequest().authenticated()
            )
            .authenticationProvider(authProvider)
            .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
            .sessionManagement(session -> session
                .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
            )
            .build();
    }
}

// ---- Method-Level Security ----
@RestController
@PreAuthorize("isAuthenticated()")
public class UserController {

    @GetMapping("/api/user/profile")
    @PreAuthorize("#userId == principal.userId or hasRole('ADMIN')")
    public User getProfile(@PathVariable Long userId) {
        return userService.findById(userId);
    }

    @GetMapping("/api/admin/users")
    @PreAuthorize("hasRole('ADMIN')")
    public List<User> getAllUsers() {
        return userService.findAll();
    }

    @PostMapping("/api/admin/users")
    @PreAuthorize("hasRole('ADMIN') and hasAuthority('CREATE_USER')")
    public User createUser(@Valid @RequestBody User user) {
        return userService.save(user);
    }

    @DeleteMapping("/api/admin/users/{id}")
    @PreAuthorize("hasRole('ADMIN') and #id != principal.userId")
    public void deleteUser(@PathVariable Long id) {
```

```
        userService.delete(id);
    }

    @GetMapping("/api/user/orders")
    @PostAuthorize("returnObject.customerId == principal.userId or hasRole('ADMIN')")
    public Order getOrder(@PathVariable Long orderId) {
        return orderService.findById(orderId);
    }
}

// ---- Configuration for Method Security ----
@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig {

    @Bean
    public MethodSecurityExpressionHandler expressionHandler() {
        return new DefaultMethodSecurityExpressionHandler();
    }
}
```

10.3 Password Encoding

Hands-On Code:

java

CopyDownload

```
@Configuration
public class PasswordConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        // BCrypt with strength 10 (default is 10)
        return new BCryptPasswordEncoder(10);
    }

    @Bean
    public PasswordEncoder multiplePasswordEncoders() {
        // Use DelegatingPasswordEncoder for supporting multiple encodings
        String encodingId = "bcrypt";
        Map<String, PasswordEncoder> encoders = Map.of(
            "bcrypt", new BCryptPasswordEncoder(),
            "pbkdf2", new Pbkdf2PasswordEncoder(),
            "scrypt", new SCryptPasswordEncoder(),
            "argon2", new Argon2PasswordEncoder()
        );
        return new DelegatingPasswordEncoder(encodingId, encoders);
    }
}

@Service
public class UserRegistrationService {

    @Autowired
    private PasswordEncoder passwordEncoder;
```

```
    @Autowired
    private UserRepository userRepository;

    public User registerUser(UserRegistrationDto dto) {
        // Validate password
        if (!isValidPassword(dto.getPassword())) {
            throw new InvalidPasswordException("Password is too weak");
        }

        User user = new User();
        user.setEmail(dto.getEmail());
        user.setFirstName(dto.getFirstName());
        user.setLastName(dto.getLastName());

        // Encrypt password before storing
        user.setPassword(passwordEncoder.encode(dto.getPassword()));

        return userRepository.save(user);
    }

    public boolean checkPassword(User user, String rawPassword) {
        return passwordEncoder.matches(rawPassword, user.getPassword());
    }

    public void updatePassword(User user, String newPassword) {
        // Re-encrypt password on update
        user.setPassword(passwordEncoder.encode(newPassword));
        userRepository.save(user);
    }

    private boolean isValidPassword(String password) {
        // Custom password strength validation
        if (password.length() < 8) return false;
        if (!password.matches(".*[A-Z].*")) return false;
        if (!password.matches(".*[a-z].*")) return false;
        if (!password.matches(".*[0-9].*")) return false;
        if (!password.matches(".*[!@#$%^&*()_+].*")) return false;
        return true;
    }
}
```

10.4 JWT Token-Based Security

Hands-On Code:

java

CopyDownload

```
// ---- JWT Token Provider ----
@Component
public class JwtTokenProvider {

    @Value("${jwt.secret}")
    private String jwtSecret;
```

```
@Value("${jwt.expiration}")
private int jwtExpirationMs;

@Value("${jwt.refresh-expiration}")
private int refreshExpirationMs;

private SecretKey getSigningKey() {
    byte[] keyBytes = jwtSecret.getBytes(StandardCharsets.UTF_8);
    return Keys.hmacShaKeyFor(keyBytes);
}

public String generateToken(Authentication authentication) {
    UserDetails userDetails = (UserDetails) authentication.getPrincipal();
    CustomUserDetails customUser = (CustomUserDetails) userDetails;

    Map<String, Object> claims = new HashMap<>();
    claims.put("userId", customUser.getUserId());
    claims.put("email", customUser.getEmail());
    claims.put("roles", customUser.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority)
        .collect(Collectors.toList()));

    return generateToken(claims, customUser.getUsername());
}

private String generateToken(Map<String, Object> claims, String subject) {
    Instant now = Instant.now();
    Instant expiry = now.plus(jwtExpirationMs, ChronoUnit.MILLIS);

    return Jwts.builder()
        .setClaims(claims)
        .setSubject(subject)
        .setIssuedAt(Date.from(now))
        .setExpiration(Date.from(expiry))
        .signWith(getSigningKey(), SignatureAlgorithm.HS512)
        .compact();
}

public boolean validateToken(String token) {
    try {
        Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token);
        return true;
    } catch (JwtException | IllegalArgumentException e) {
        return false;
    }
}

public String getUsernameFromToken(String token) {
    Claims claims = Jwts.parserBuilder()
        .setSigningKey(getSigningKey())
        .build()
```

```
            .parseClaimsJws(token)
            .getBody();
        return claims.getSubject();
    }

    public Long getUserIdFromToken(String token) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
        return claims.get("userId", Long.class);
    }

    public List<String> getRolesFromToken(String token) {
        Claims claims = Jwts.parserBuilder()
            .setSigningKey(getSigningKey())
            .build()
            .parseClaimsJws(token)
            .getBody();
        return claims.get("roles", List.class);
    }

    public String generateRefreshToken(UserDetails userDetails) {
        Map<String, Object> claims = new HashMap<>();
        claims.put("type", "refresh");
        claims.put("userId", ((CustomUserDetails) userDetails).getUserId());

        Instant now = Instant.now();
        Instant expiry = now.plus(refreshExpirationMs, ChronoUnit.MILLIS);

        return Jwts.builder()
            .setClaims(claims)
            .setSubject(userDetails.getUsername())
            .setIssuedAt(Date.from(now))
            .setExpiration(Date.from(expiry))
            .signWith(getSigningKey(), SignatureAlgorithm.HS512)
            .compact();
    }
}

// ---- JWT Authentication Filter ----
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtTokenProvider tokenProvider;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
```

```
                                FilterChain filterChain) throws ServletException, IOException {
        try {
            String jwt = getJwtFromRequest(request);

            if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
                String username = tokenProvider.getUsernameFromToken(jwt);
                UserDetails userDetails = userDetailsService.loadUserByUsername(username);

                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                        userDetails,
                        null,
                        userDetails.getAuthorities()
                    );
                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        } catch (Exception e) {
            logger.error("Could not set user authentication in security context", e);
        }

        filterChain.doFilter(request, response);
    }

    private String getJwtFromRequest(HttpServletRequest request) {
        String bearerToken = request.getHeader("Authorization");
        if (StringUtils.hasText(bearerToken) && bearerToken.startsWith("Bearer ")) {
            return bearerToken.substring(7);
        }
        return null;
    }
}

// ---- JWT Controller ----
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtTokenProvider tokenProvider;

    @Autowired
    private CustomUserDetailsService userDetailsService;

    @PostMapping("/login")
    public ResponseEntity<JwtResponse> authenticateUser(@Valid @RequestBody LoginRequest
request) {
        Authentication authentication = authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(
                request.getEmail(),
```

```
            request.getPassword()
        )
    );

    SecurityContextHolder.getContext().setAuthentication(authentication);

    String accessToken = tokenProvider.generateToken(authentication);
    String refreshToken = tokenProvider.generateRefreshToken(
        (UserDetails) authentication.getPrincipal()
    );

    UserDetails userDetails = (UserDetails) authentication.getPrincipal();

    return ResponseEntity.ok(new JwtResponse(
        accessToken,
        refreshToken,
        userDetails.getUsername(),
        userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList())
    ));
}

@PostMapping("/refresh")
public ResponseEntity<?> refreshToken(@Valid @RequestBody RefreshTokenRequest request)
{
    String refreshToken = request.getRefreshToken();

    if (!tokenProvider.validateToken(refreshToken)) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body("Invalid refresh token");
    }

    String username = tokenProvider.getUsernameFromToken(refreshToken);
    UserDetails userDetails = userDetailsService.loadUserByUsername(username);

    // Create new authentication token
    UsernamePasswordAuthenticationToken authentication =
        new UsernamePasswordAuthenticationToken(
            userDetails,
            null,
            userDetails.getAuthorities()
        );

    String newAccessToken = tokenProvider.generateToken(authentication);

    return ResponseEntity.ok(new JwtResponse(
        newAccessToken,
        refreshToken,
        username,
        userDetails.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .collect(Collectors.toList())
    ));
```

```
    }

    @PostMapping("/logout")
    public ResponseEntity<?> logout() {
        // Invalidate token (you can add to blacklist)
        SecurityContextHolder.clearContext();
        return ResponseEntity.ok("Logged out successfully");
    }
}

// ---- JWT Response DTOs ----
@Data
@AllArgsConstructor
public class JwtResponse {
    private String accessToken;
    private String refreshToken;
    private String username;
    private List<String> roles;
}

@Data
public class LoginRequest {
    @NotBlank @Email
    private String email;

    @NotBlank
    private String password;
}

@Data
public class RefreshTokenRequest {
    @NotBlank
    private String refreshToken;
}
```

10.5 Role-Based Access Control

Hands-On Code:

java

CopyDownload

```
// ---- Role Entity ----
@Entity
@Table(name = "roles")
public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Enumerated(EnumType.STRING)
    @Column(unique = true)
    private RoleName name;

    @ManyToMany(mappedBy = "roles")
    private Set<User> users = new HashSet<>();
```

```
}

public enum RoleName {
    ROLE_USER,
    ROLE_ADMIN,
    ROLE_MANAGER,
    ROLE_SUPER_ADMIN,
    ROLE_VIEWER
}

// ---- User Entity with Roles ----
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String email;

    private String password;
    private String firstName;
    private String lastName;
    private boolean active = true;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles = new HashSet<>();

    @CreationTimestamp
    private LocalDateTime createdAt;

    @UpdateTimestamp
    private LocalDateTime updatedAt;
}

// ---- Security with RBAC ----
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
```

```
            .requestMatchers("/api/manager/**").hasAnyRole("ADMIN", "MANAGER")
            .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN", "MANAGER")
            .requestMatchers("/api/viewer/**").hasAnyRole("VIEWER", "USER", "ADMIN",
"MANAGER")
            .anyRequest().authenticated()
        )
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        )
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
  }
}

// ---- Controller with Role-Based Access ----
@RestController
@RequestMapping("/api")
public class DashboardController {

  @GetMapping("/admin/dashboard")
  @PreAuthorize("hasRole('ADMIN')")
  public ResponseEntity<AdminDashboard> getAdminDashboard() {
    return ResponseEntity.ok(adminService.getDashboard());
  }

  @GetMapping("/admin/users")
  @PreAuthorize("hasRole('ADMIN')")
  public ResponseEntity<List<User>> getAllUsers() {
    return ResponseEntity.ok(userService.findAll());
  }

  @GetMapping("/admin/users/{id}")
  @PreAuthorize("hasRole('ADMIN') or (hasRole('MANAGER') and #id == principal.userId)")
  public ResponseEntity<User> getUserById(@PathVariable Long id) {
    return userService.findById(id)
      .map(ResponseEntity::ok)
      .orElse(ResponseEntity.notFound().build());
  }

  @PostMapping("/admin/users")
  @PreAuthorize("hasRole('ADMIN') or hasRole('MANAGER')")
  public ResponseEntity<User> createUser(@Valid @RequestBody User user) {
    return ResponseEntity.status(HttpStatus.CREATED)
      .body(userService.createUser(user));
  }

  @PutMapping("/admin/users/{id}")
  @PreAuthorize("hasRole('ADMIN') or (hasRole('MANAGER') and #id == principal.userId)")
  public ResponseEntity<User> updateUser(@PathVariable Long id, @Valid @RequestBody User
user) {
    return ResponseEntity.ok(userService.updateUser(id, user));
  }
```

```
    @DeleteMapping("/admin/users/{id}")
    @PreAuthorize("hasRole('ADMIN')")
    public ResponseEntity<Void> deleteUser(@PathVariable Long id) {
        userService.deleteUser(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/user/profile")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<User> getProfile(Authentication authentication) {
        String email = authentication.getName();
        return userService.findByEmail(email)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/user/orders")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<List<Order>> getUserOrders(Authentication authentication) {
        CustomUserDetails userDetails = (CustomUserDetails) authentication.getPrincipal();
        return ResponseEntity.ok(orderService.findByUserId(userDetails.getUserId()));
    }

    @GetMapping("/user/orders/{orderId}")
    @PreAuthorize("hasRole('ADMIN') or " +
                  "@orderService.isOrderOwner(#orderId, principal.userId)")
    public ResponseEntity<Order> getOrder(@PathVariable Long orderId) {
        return orderService.findById(orderId)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/viewer/reports")
    @PreAuthorize("hasAnyRole('VIEWER', 'USER', 'ADMIN', 'MANAGER')")
    public ResponseEntity<List<Report>> getReports() {
        return ResponseEntity.ok(reportService.getAccessibleReports());
    }
}

// ---- Custom Permission Evaluator ----
@Component
public class CustomPermissionEvaluator implements PermissionEvaluator {

    @Autowired
    private UserService userService;

    @Override
    public boolean hasPermission(Authentication authentication, Object targetDomainObject,
Object permission) {
        // Check if the user has permission on the specific object
        if (targetDomainObject instanceof Order) {
            Order order = (Order) targetDomainObject;
            return hasOrderPermission(authentication, order, permission);
        }
```

```
        return false;
    }

    @Override
    public boolean hasPermission(Authentication authentication, Serializable targetId, String
targetType, Object permission) {
        if ("Order".equals(targetType)) {
            Order order = orderService.findById((Long) targetId);
            return hasOrderPermission(authentication, order, permission);
        }
        return false;
    }

    private boolean hasOrderPermission(Authentication authentication, Order order, Object
permission) {
        CustomUserDetails userDetails = (CustomUserDetails) authentication.getPrincipal();
        Long userId = userDetails.getUserId();

        if (userDetails.getAuthorities().stream().anyMatch(a ->
"ROLE_ADMIN".equals(a.getAuthority()))) {
            return true;
        }

        if ("READ".equals(permission)) {
            return order.getCustomerId().equals(userId);
        }

        if ("WRITE".equals(permission)) {
            return order.getCustomerId().equals(userId) && order.getStatus() ==
OrderStatus.PENDING;
        }

        return false;
    }
}

// ---- Service to check ownership ----
@Service
public class OrderService {

    public boolean isOrderOwner(Long orderId, Long userId) {
        return orderRepository.findById(orderId)
            .map(order -> order.getCustomerId().equals(userId))
            .orElse(false);
    }
}
```

---

11. Testing in Spring Boot

11.1 Unit Testing with JUnit

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.service;

import org.junit.jupiter.api.*;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.*;

import java.time.LocalDateTime;

import static org.junit.jupiter.api.Assertions.*;
import static org.assertj.core.api.Assertions.assertThat;

@DisplayName("User Service Unit Tests")
class UserServiceUnitTest {

    private UserService userService;
    private UserRepository mockRepository;

    @BeforeEach
    void setUp() {
        // Initialize mocks and service
        mockRepository = mock(UserRepository.class);
        userService = new UserService(mockRepository);
    }

    @Test
    @DisplayName("Should register user successfully")
    void shouldRegisterUserSuccessfully() {
        // Arrange
        UserRegistrationDto dto = createValidUserDto();
        User expectedUser = createUser("john@example.com");

        when(mockRepository.existsByEmail(anyString())).thenReturn(false);
        when(mockRepository.save(any(User.class))).thenReturn(expectedUser);

        // Act
        User result = userService.registerUser(dto);

        // Assert
        assertThat(result).isNotNull();
        assertThat(result.getEmail()).isEqualTo("john@example.com");
        verify(mockRepository).save(any(User.class));
    }

    @Test
    @DisplayName("Should throw exception when registering existing user")
    void shouldThrowExceptionWhenRegisteringExistingUser() {
        // Arrange
        UserRegistrationDto dto = createValidUserDto();
        when(mockRepository.existsByEmail(dto.getEmail())).thenReturn(true);

        // Act & Assert
        assertThrows(EmailAlreadyExistsException.class, () -> {
            userService.registerUser(dto);
```

```
        });

        verify(mockRepository, never()).save(any(User.class));
    }

    @ParameterizedTest
    @DisplayName("Should validate email format")
    @ValueSource(strings = {"john@example.com", "jane.doe@test.org",
"admin@company.co.uk"})
    void shouldValidateEmailFormat(String validEmail) {
        assertTrue(userService.isValidEmail(validEmail));
    }

    @ParameterizedTest
    @DisplayName("Should reject invalid email format")
    @ValueSource(strings = {"john@example", "jane.doe@", "admin@company@co.uk"})
    void shouldRejectInvalidEmailFormat(String invalidEmail) {
        assertFalse(userService.isValidEmail(invalidEmail));
    }

    @ParameterizedTest
    @DisplayName("Should calculate user age correctly")
    @CsvSource({
        "1990-01-01, 33",
        "2000-06-15, 23",
        "2010-12-20, 13"
    })
    void shouldCalculateUserAgeCorrectly(LocalDate birthDate, int expectedAge) {
        int age = userService.calculateAge(birthDate);
        assertEquals(expectedAge, age);
    }

    @Test
    @DisplayName("Should update user profile")
    void shouldUpdateUserProfile() {
        // Arrange
        Long userId = 1L;
        User existingUser = new User();
        existingUser.setId(userId);
        existingUser.setFirstName("John");
        existingUser.setLastName("Doe");

        UserUpdateDto updateDto = new UserUpdateDto();
        updateDto.setFirstName("Jonathan");
        updateDto.setLastName("Smith");

        when(mockRepository.findById(userId)).thenReturn(Optional.of(existingUser));
        when(mockRepository.save(any(User.class))).thenReturn(existingUser);

        // Act
        User updatedUser = userService.updateUser(userId, updateDto);

        // Assert
        assertThat(updatedUser.getFirstName()).isEqualTo("Jonathan");
```

```
        assertThat(updatedUser.getLastName()).isEqualTo("Smith");
        verify(mockRepository).save(existingUser);
    }

    @Test
    @DisplayName("Should handle concurrent updates")
    void shouldHandleConcurrentUpdates() {
        // Simulate concurrent modification
        Long userId = 1L;
        User existingUser = new User();
        existingUser.setId(userId);
        existingUser.setVersion(0L);

        UserUpdateDto updateDto = new UserUpdateDto();
        updateDto.setFirstName("NewName");

        when(mockRepository.findById(userId)).thenReturn(Optional.of(existingUser));
        when(mockRepository.save(any(User.class)))
            .thenThrow(OptimisticLockException.class);

        assertThrows(OptimisticLockException.class, () -> {
            userService.updateUser(userId, updateDto);
        });
    }

    private UserRegistrationDto createValidUserDto() {
        return UserRegistrationDto.builder()
            .firstName("John")
            .lastName("Doe")
            .email("john@example.com")
            .password("SecurePass123!")
            .build();
    }
}
```

11.2 Mocking with Mockito

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.service;

import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Arrays;
import java.util.List;
import java.util.Optional;

import static org.mockito.Mockito.*;
import static org.assertj.core.api.Assertions.assertThat;
```

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {

    @Mock
    private OrderRepository orderRepository;

    @Mock
    private ProductRepository productRepository;

    @Mock
    private PaymentService paymentService;

    @InjectMocks
    private OrderService orderService;

    @Captor
    private ArgumentCaptor<Order> orderCaptor;

    @Spy
    private Logger logger = LoggerFactory.getLogger(OrderService.class);

    @Test
    void shouldCreateOrderSuccessfully() {
        // Arrange
        OrderRequest request = createOrderRequest();
        Product product = createProduct(100.0);
        Order expectedOrder = createOrder();

        when(productRepository.findById(1L)).thenReturn(Optional.of(product));
        when(paymentService.processPayment(anyDouble())).thenReturn(true);
        when(orderRepository.save(any(Order.class))).thenReturn(expectedOrder);

        // Act
        Order result = orderService.createOrder(request);

        // Assert
        assertThat(result).isNotNull();
        verify(productRepository).findById(1L);
        verify(paymentService).processPayment(100.0);
        verify(orderRepository).save(orderCaptor.capture());

        Order capturedOrder = orderCaptor.getValue();
        assertThat(capturedOrder.getStatus()).isEqualTo(OrderStatus.COMPLETED);
    }

    @Test
    void shouldHandleOutOfStockProducts() {
        // Arrange
        OrderRequest request = createOrderRequest();
        when(productRepository.findById(1L)).thenReturn(Optional.empty());

        // Act & Assert
        assertThrows(ProductNotFoundException.class, () -> {
            orderService.createOrder(request);
```

```
        });

        verify(paymentService, never()).processPayment(anyDouble());
        verify(orderRepository, never()).save(any(Order.class));
    }

    @Test
    void shouldProcessMultipleOrdersInBatch() {
        // Arrange
        List<OrderRequest> requests = Arrays.asList(
            createOrderRequest(),
            createOrderRequest()
        );

        when(productRepository.findById(1L)).thenReturn(Optional.of(createProduct(100.0)));
        when(paymentService.processPayment(anyDouble())).thenReturn(true);
        when(orderRepository.saveAll(anyList())).thenAnswer(invocation ->
invocation.getArgument(0));

        // Act
        List<Order> orders = orderService.processBatchOrders(requests);

        // Assert
        assertThat(orders).hasSize(2);
        verify(orderRepository, times(1)).saveAll(anyList());
    }

    @Test
    void shouldThrowExceptionWhenPaymentFails() {
        // Arrange
        OrderRequest request = createOrderRequest();
        when(productRepository.findById(1L)).thenReturn(Optional.of(createProduct(100.0)));
        when(paymentService.processPayment(100.0)).thenThrow(new
PaymentException("Payment declined"));
        when(orderRepository.save(any(Order.class))).thenAnswer(invocation ->
invocation.getArgument(0));

        // Act & Assert
        assertThrows(PaymentException.class, () -> {
            orderService.createOrder(request);
        });

        // Verify that order is saved with failed status
        verify(orderRepository).save(orderCaptor.capture());
        Order capturedOrder = orderCaptor.getValue();
        assertThat(capturedOrder.getStatus()).isEqualTo(OrderStatus.FAILED);
    }

    @Test
    void shouldVerifyMethodCallOrder() {
        // Arrange
        OrderRequest request = createOrderRequest();
        when(productRepository.findById(1L)).thenReturn(Optional.of(createProduct(100.0)));
        when(paymentService.processPayment(anyDouble())).thenReturn(true);
```

```
        when(orderRepository.save(any(Order.class))).thenReturn(createOrder());

        // Act
        orderService.createOrder(request);

        // Assert order of method calls
        InOrder inOrder = inOrder(productRepository, paymentService, orderRepository);
        inOrder.verify(productRepository).findById(1L);
        inOrder.verify(paymentService).processPayment(100.0);
        inOrder.verify(orderRepository).save(any(Order.class));
    }
}
```

11.3 Integration Testing

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.boot.test.web.server.LocalServerPort;
import org.springframework.http.*;
import org.springframework.test.context.ActiveProfiles;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

import static org.assertj.core.api.Assertions.assertThat;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@ActiveProfiles("test")
@Transactional // Rollback after each test
class ApplicationIntegrationTest {

    @LocalServerPort
    private int port;

    @Autowired
    private TestRestTemplate restTemplate;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private OrderRepository orderRepository;

    private String getBaseUrl() {
        return "http://localhost:" + port;
    }
```

```
@Test
void shouldCreateAndRetrieveUser() {
    // Create user
    UserRegistrationDto dto = UserRegistrationDto.builder()
        .firstName("John")
        .lastName("Doe")
        .email("john@example.com")
        .password("SecurePass123!")
        .build();

    ResponseEntity<User> createResponse = restTemplate.postForEntity(
        getBaseUrl() + "/api/users",
        dto,
        User.class
    );

    assertThat(createResponse.getStatusCode()).isEqualTo(HttpStatus.CREATED);
    User createdUser = createResponse.getBody();
    assertThat(createdUser).isNotNull();
    assertThat(createdUser.getId()).isNotNull();
    assertThat(createdUser.getEmail()).isEqualTo("john@example.com");

    // Retrieve user
    ResponseEntity<User> getResponse = restTemplate.getForEntity(
        getBaseUrl() + "/api/users/{id}",
        User.class,
        createdUser.getId()
    );

    assertThat(getResponse.getStatusCode()).isEqualTo(HttpStatus.OK);
    User retrievedUser = getResponse.getBody();
    assertThat(retrievedUser).isNotNull();
    assertThat(retrievedUser.getEmail()).isEqualTo("john@example.com");
}

@Test
void shouldSearchProducts() {
    // Setup - create test products
    Product product1 = new Product("Laptop", 999.99, 10);
    Product product2 = new Product("Monitor", 299.99, 20);
    productRepository.saveAll(Arrays.asList(product1, product2));

    // Search for products
    ResponseEntity<Product[]> response = restTemplate.getForEntity(
        getBaseUrl() + "/api/products/search?keyword=Laptop",
        Product[].class
    );

    assertThat(response.getStatusCode()).isEqualTo(HttpStatus.OK);
    Product[] products = response.getBody();
    assertThat(products).isNotNull();
    assertThat(products).extracting(Product::getName)
        .contains("Laptop")
        .doesNotContain("Monitor");
```

```
    }

    @Test
    void shouldHandleValidationErrors() {
        // Invalid user data (missing first name)
        UserRegistrationDto dto = UserRegistrationDto.builder()
            .lastName("Doe")
            .email("john@example.com")
            .password("weak")
            .build();

        ResponseEntity<ValidationErrorResponse> response = restTemplate.postForEntity(
            getBaseUrl() + "/api/users",
            dto,
            ValidationErrorResponse.class
        );

        assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
        ValidationErrorResponse errorResponse = response.getBody();
        assertThat(errorResponse).isNotNull();
        assertThat(errorResponse.getErrors()).containsKey("firstName");
    }
}
```

11.4 Web Layer Testing (@WebMvcTest)

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.controller;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Arrays;
import java.util.Optional;

import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(UserController.class)
class UserControllerWebTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private UserService userService;
```

```
@Test
void shouldGetAllUsers() throws Exception {
    // Arrange
    User user1 = new User(1L, "John", "Doe", "john@example.com");
    User user2 = new User(2L, "Jane", "Smith", "jane@example.com");
    when(userService.findAll()).thenReturn(Arrays.asList(user1, user2));

    // Act & Assert
    mockMvc.perform(get("/api/users"))
        .andExpect(status().isOk())
        .andExpect(content().contentType(MediaType.APPLICATION_JSON))
        .andExpect(jsonPath("$[0].id").value(1L))
        .andExpect(jsonPath("$[0].firstName").value("John"))
        .andExpect(jsonPath("$[1].id").value(2L))
        .andExpect(jsonPath("$[1].firstName").value("Jane"));
}

@Test
void shouldGetUserById() throws Exception {
    // Arrange
    User user = new User(1L, "John", "Doe", "john@example.com");
    when(userService.findById(1L)).thenReturn(Optional.of(user));

    // Act & Assert
    mockMvc.perform(get("/api/users/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.id").value(1L))
        .andExpect(jsonPath("$.email").value("john@example.com"));
}

@Test
void shouldReturnNotFoundForNonExistentUser() throws Exception {
    // Arrange
    when(userService.findById(999L)).thenReturn(Optional.empty());

    // Act & Assert
    mockMvc.perform(get("/api/users/999"))
        .andExpect(status().isNotFound());
}

@Test
void shouldCreateUser() throws Exception {
    // Arrange
    User user = new User(1L, "John", "Doe", "john@example.com");
    when(userService.registerUser(any(UserRegistrationDto.class))).thenReturn(user);

    String userJson = """
        {
            "firstName": "John",
            "lastName": "Doe",
            "email": "john@example.com",
            "password": "SecurePass123!"
        }
```

```
        """;

    // Act & Assert
    mockMvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(userJson))
        .andExpect(status().isCreated())
        .andExpect(jsonPath("$.id").value(1L))
        .andExpect(jsonPath("$.email").value("john@example.com"));
}

@Test
void shouldReturnBadRequestForInvalidUserData() throws Exception {
    // Invalid user data (missing first name)
    String userJson = """
        {
            "lastName": "Doe",
            "email": "john@example.com",
            "password": "weak"
        }
        """;

    mockMvc.perform(post("/api/users")
            .contentType(MediaType.APPLICATION_JSON)
            .content(userJson))
        .andExpect(status().isBadRequest())
        .andExpect(jsonPath("$.errors.firstName").exists());
}

@Test
void shouldUpdateUser() throws Exception {
    // Arrange
    User user = new User(1L, "Jonathan", "Smith", "jonathan@example.com");
    when(userService.updateUser(anyLong(), any(UserUpdateDto.class))).thenReturn(user);

    String userJson = """
        {
            "firstName": "Jonathan",
            "lastName": "Smith"
        }
        """;

    // Act & Assert
    mockMvc.perform(put("/api/users/1")
            .contentType(MediaType.APPLICATION_JSON)
            .content(userJson))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.firstName").value("Jonathan"))
        .andExpect(jsonPath("$.lastName").value("Smith"));
}

@Test
void shouldDeleteUser() throws Exception {
    // Arrange
```

```
        doNothing().when(userService).deleteUser(1L);

        // Act & Assert
        mockMvc.perform(delete("/api/users/1"))
            .andExpect(status().isNoContent());
    }

    @Test
    void shouldHandleMethodArgumentNotValidException() throws Exception {
        // Invalid email format
        String userJson = """
            {
                "firstName": "John",
                "lastName": "Doe",
                "email": "invalid-email",
                "password": "SecurePass123!"
            }
            """;

        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_JSON)
                .content(userJson))
            .andExpect(status().isBadRequest())
            .andExpect(jsonPath("$.errors.email").exists());
    }

    @Test
    void shouldHandleHttpRequestMethodNotSupportedException() throws Exception {
        // Use POST method where only GET is supported
        mockMvc.perform(post("/api/users/1"))
            .andExpect(status().isMethodNotAllowed());
    }

    @Test
    void shouldHandleMediaTypeNotSupportedException() throws Exception {
        // Send XML when only JSON is expected
        String xmlData = "<user><firstName>John</firstName></user>";

        mockMvc.perform(post("/api/users")
                .contentType(MediaType.APPLICATION_XML)
                .content(xmlData))
            .andExpect(status().isUnsupportedMediaType());
    }
}
```

11.5 Data Layer Testing (@DataJpaTest)

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.repository;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.boot.test.autoconfigure.orm.jpa.TestEntityManager;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.test.context.ActiveProfiles;

import java.util.List;
import java.util.Optional;

import static org.assertj.core.api.Assertions.assertThat;

@DataJpaTest
@ActiveProfiles("test")
class UserRepositoryTest {

    @Autowired
    private TestEntityManager entityManager;

    @Autowired
    private UserRepository userRepository;

    @Test
    void shouldSaveUser() {
        // Arrange
        User user = new User("John", "Doe", "john@example.com", "password");

        // Act
        User savedUser = userRepository.save(user);

        // Assert
        assertThat(savedUser).isNotNull();
        assertThat(savedUser.getId()).isNotNull();
        assertThat(savedUser.getEmail()).isEqualTo("john@example.com");
        assertThat(savedUser.getCreatedAt()).isNotNull();
        assertThat(savedUser.getUpdatedAt()).isNotNull();
    }

    @Test
    void shouldFindUserByEmail() {
        // Arrange
        User user = new User("John", "Doe", "john@example.com", "password");
        entityManager.persist(user);
        entityManager.flush();

        // Act
        Optional<User> foundUser = userRepository.findByEmail("john@example.com");

        // Assert
        assertThat(foundUser).isPresent();
        assertThat(foundUser.get().getEmail()).isEqualTo("john@example.com");
        assertThat(foundUser.get().getFirstName()).isEqualTo("John");
    }
```

```
@Test
void shouldCheckIfUserExistsByEmail() {
    // Arrange
    User user = new User("John", "Doe", "john@example.com", "password");
    entityManager.persist(user);
    entityManager.flush();

    // Act & Assert
    assertThat(userRepository.existsByEmail("john@example.com")).isTrue();
    assertThat(userRepository.existsByEmail("nonexistent@example.com")).isFalse();
}

@Test
void shouldFindUsersByLastName() {
    // Arrange
    User user1 = new User("John", "Doe", "john@example.com", "password");
    User user2 = new User("Jane", "Doe", "jane@example.com", "password");
    User user3 = new User("Bob", "Smith", "bob@example.com", "password");
    entityManager.persist(user1);
    entityManager.persist(user2);
    entityManager.persist(user3);
    entityManager.flush();

    // Act
    List<User> doeUsers = userRepository.findByLastName("Doe");

    // Assert
    assertThat(doeUsers).hasSize(2);
    assertThat(doeUsers).extracting(User::getFirstName)
        .containsExactlyInAnyOrder("John", "Jane");
}

@Test
void shouldFindActiveUsers() {
    // Arrange
    User user1 = new User("John", "Doe", "john@example.com", "password");
    user1.setActive(true);
    User user2 = new User("Jane", "Smith", "jane@example.com", "password");
    user2.setActive(false);
    User user3 = new User("Bob", "Johnson", "bob@example.com", "password");
    user3.setActive(true);

    entityManager.persist(user1);
    entityManager.persist(user2);
    entityManager.persist(user3);
    entityManager.flush();

    // Act
    List<User> activeUsers = userRepository.findByActiveTrue();

    // Assert
    assertThat(activeUsers).hasSize(2);
    assertThat(activeUsers).extracting(User::getEmail)
        .containsExactlyInAnyOrder("john@example.com", "bob@example.com");
```

```
}

@Test
void shouldPageActiveUsers() {
    // Arrange
    for (int i = 0; i < 20; i++) {
        User user = new User("User" + i, "Test", "user" + i + "@example.com", "password");
        user.setActive(i % 2 == 0); // 10 active users
        entityManager.persist(user);
    }
    entityManager.flush();

    // Act
    Page<User> page1 = userRepository.findAllByActiveTrue(
        PageRequest.of(0, 5, Sort.by("firstName"))
    );
    Page<User> page2 = userRepository.findAllByActiveTrue(
        PageRequest.of(1, 5, Sort.by("firstName"))
    );

    // Assert
    assertThat(page1.getTotalElements()).isEqualTo(10);
    assertThat(page1.getContent()).hasSize(5);
    assertThat(page2.getContent()).hasSize(5);
    assertThat(page1.getContent()).isNotEqualTo(page2.getContent());
}

@Test
void shouldUpdateUserActiveStatus() {
    // Arrange
    User user = new User("John", "Doe", "john@example.com", "password");
    user.setActive(true);
    entityManager.persist(user);
    entityManager.flush();

    // Act
    int updated = userRepository.updateUserActiveStatus(user.getId(), false);

    // Assert
    assertThat(updated).isEqualTo(1);

    User updatedUser = userRepository.findById(user.getId()).orElseThrow();
    assertThat(updatedUser.isActive()).isFalse();
}

@Test
void shouldDeleteUserById() {
    // Arrange
    User user = new User("John", "Doe", "john@example.com", "password");
    entityManager.persist(user);
    entityManager.flush();
    Long userId = user.getId();

    // Act
```

```
        userRepository.deleteUserById(userId);

        // Assert
        assertThat(userRepository.findById(userId)).isEmpty();
    }

    @Test
    void shouldFindUsersByDomainUsingCustomQuery() {
        // Arrange
        User user1 = new User("John", "Doe", "john@gmail.com", "password");
        User user2 = new User("Jane", "Smith", "jane@yahoo.com", "password");
        User user3 = new User("Bob", "Johnson", "bob@gmail.com", "password");
        entityManager.persist(user1);
        entityManager.persist(user2);
        entityManager.persist(user3);
        entityManager.flush();

        // Act
        List<User> gmailUsers = userRepository.findUsersByDomain("gmail.com");

        // Assert
        assertThat(gmailUsers).hasSize(2);
        assertThat(gmailUsers).extracting(User::getEmail)
            .containsExactlyInAnyOrder("john@gmail.com", "bob@gmail.com");
    }
}
```

---

12. Logging in Spring Boot

12.1 SLF4J and Logback

Hands-On Code:

java

CopyDownload

```
package net.guides.springboot2.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/logging")
public class LoggingController {

    // Create logger instance for this class
    private static final Logger logger = LoggerFactory.getLogger(LoggingController.class);

    @GetMapping("/test")
    public String testLogging() {
        // Different log levels
        logger.trace("This is a TRACE message - finest level, for detailed debugging");
        logger.debug("This is a DEBUG message - for development and debugging");
        logger.info("This is an INFO message - general application events");
```

```
        logger.warn("This is a WARN message - potential issues to investigate");
        logger.error("This is an ERROR message - serious issues that need attention");

        // Log with parameters (avoid string concatenation)
        String userId = "123";
        String action = "login";
        logger.info("User {} performed action: {}", userId, action);

        // Log with exceptions
        try {
            throw new RuntimeException("Simulated error");
        } catch (Exception e) {
            // Log exception with stack trace
            logger.error("An error occurred while processing request", e);
        }

        // Conditional logging (avoid expensive operations)
        if (logger.isDebugEnabled()) {
            // Only executed if debug level is enabled
            String expensiveResult = performExpensiveOperation();
            logger.debug("Expensive operation result: {}", expensiveResult);
        }

        return "Check logs for output";
    }

    private String performExpensiveOperation() {
        // Simulate expensive operation
        return "Result from expensive operation";
    }
}

// ---- Service with Logging ----
@Service
@Slf4j // Lombok annotation that generates logger field
public class UserService {

    // With Lombok @Slf4j, you can use 'log' directly without declaring it
    // private static final Logger log = LoggerFactory.getLogger(UserService.class);

    public User createUser(UserRegistrationDto dto) {
        log.info("Creating new user with email: {}", dto.getEmail());

        try {
            // Business logic
            User user = new User();
            user.setEmail(dto.getEmail());
            user.setFirstName(dto.getFirstName());
            user.setLastName(dto.getLastName());

            log.debug("User object created: {}", user);

            // Simulate save operation
            user.setId(1L);
```

```
            log.info("User created successfully with ID: {}", user.getId());
            return user;
        } catch (Exception e) {
            log.error("Failed to create user with email: {}", dto.getEmail(), e);
            throw e;
        }
    }
}
```

12.2 Configuring Log Levels

Hands-On Code:

yaml

CopyDownload

```
# application.yml - Logging Configuration
logging:
  # Global root level
  level:
    root: INFO
    # Specific package levels
    com.net.guides.springboot2: DEBUG
    com.net.guides.springboot2.controller: TRACE
    com.net.guides.springboot2.service: DEBUG
    org.springframework.web: INFO
    org.hibernate: WARN
    org.hibernate.SQL: DEBUG
    org.hibernate.type.descriptor.sql.BasicBinder: TRACE

  # File output configuration
  file:
    name: logs/application.log
    max-size: 10MB
    max-history: 30
    total-size-cap: 100MB

  # Console output pattern
  pattern:
    console: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"
    file: "%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n"

  # Log groups
  group:
    web: org.springframework.web, org.springframework.http
    sql: org.hibernate.SQL, org.springframework.jdbc
```

12.3 External Log Configuration

Hands-On Code:

xml

CopyDownloadRun

```
<!-- logback-spring.xml in src/main/resources -->
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
```

```
    <!-- Property definitions -->
    <property name="LOG_PATH" value="${LOG_PATH:-./logs}" />
    <property name="LOG_FILE" value="${LOG_PATH}/application" />
    <property name="LOG_PATTERN"
              value="%d{yyyy-MM-dd HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n" />

    <!-- Console appender -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>${LOG_PATTERN}</pattern>
            <charset>UTF-8</charset>
        </encoder>
    </appender>

    <!-- File appender with rotation -->
    <appender name="FILE" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_FILE}.log</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <!-- Daily rollover -->
            <fileNamePattern>${LOG_FILE}.%d{yyyy-MM-dd}.%i.log</fileNamePattern>
            <!-- Keep 30 days of history -->
            <maxHistory>30</maxHistory>
            <!-- Size-based triggering policy -->
            <timeBasedFileNamingAndTriggeringPolicy
    class="ch.qos.logback.core.rolling.SizeAndTimeBasedFNATP">
                <maxFileSize>10MB</maxFileSize>
            </timeBasedFileNamingAndTriggeringPolicy>
        </rollingPolicy>
        <encoder>
            <pattern>${LOG_PATTERN}</pattern>
            <charset>UTF-8</charset>
        </encoder>
    </appender>

    <!-- JSON appender for structured logging -->
    <appender name="JSON" class="ch.qos.logback.core.rolling.RollingFileAppender">
        <file>${LOG_PATH}/application.json</file>
        <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
            <fileNamePattern>${LOG_PATH}/application.%d{yyyy-MM-dd}.json</fileNamePattern>
            <maxHistory>30</maxHistory>
        </rollingPolicy>
        <encoder class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
            <providers>
                <timestamp>
                    <timeZone>UTC</timeZone>
                </timestamp>
                <version/>
                <message/>
                <loggerName/>
                <threadName/>
                <logLevel/>
                <stackTrace/>
            </providers>
        </encoder>
```

```
    </appender>

    <!-- Async appender for performance -->
    <appender name="ASYNC" class="ch.qos.logback.classic.AsyncAppender">
        <appender-ref ref="FILE" />
        <queueSize>500</queueSize>
        <discardingThreshold>0</discardingThreshold>
        <includeCallerData>false</includeCallerData>
    </appender>

    <!-- Profile-specific configurations -->
    <springProfile name="dev">
        <root level="DEBUG">
            <appender-ref ref="CONSOLE" />
            <appender-ref ref="ASYNC" />
        </root>
        <!-- More verbose logging in development -->
        <logger name="com.net.guides" level="DEBUG" />
        <logger name="org.springframework.web" level="DEBUG" />
    </springProfile>

    <springProfile name="prod">
        <root level="INFO">
            <appender-ref ref="ASYNC" />
            <appender-ref ref="JSON" />
        </root>
        <!-- Less verbose in production -->
        <logger name="com.net.guides" level="INFO" />
        <logger name="org.springframework.web" level="WARN" />
    </springProfile>

    <!-- Default configuration (when no profile is active) -->
    <root level="INFO">
        <appender-ref ref="CONSOLE" />
        <appender-ref ref="FILE" />
    </root>

    <!-- Specific loggers for third-party libraries -->
    <logger name="org.hibernate.SQL" level="DEBUG" additivity="false">
        <appender-ref ref="CONSOLE" />
        <appender-ref ref="FILE" />
    </logger>

    <logger name="org.springframework.transaction" level="DEBUG" />
    <logger name="org.springframework.security" level="DEBUG" />
</configuration>
```

---

13. Actuator & Monitoring

13.1 Spring Boot Actuator Overview

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

yaml

CopyDownload

```
# application.yml - Actuator Configuration
management:
  # Endpoint configuration
  endpoints:
    web:
      exposure:
        # Include specific endpoints
        include: health,info,metrics,env,beans,mappings,loggers,threaddump,heapdump
        # Exclude specific endpoints
        exclude: shutdown
      base-path: /actuator
      cors:
        allowed-origins: "*"
        allowed-methods: GET,POST
  endpoint:
    health:
      show-details: always
      show-components: always
      probes:
        enabled: true
    metrics:
      enabled: true
    info:
      enabled: true
    loggers:
      enabled: true
  info:
    env:
      enabled: true
  health:
    defaults:
      enabled: true
    status:
      http-mapping:
        DOWN: 503
        OUT_OF_SERVICE: 503
```

13.2 Common Actuator Endpoints

Hands-On Code:

java

CopyDownload

```
// ---- Custom Info Endpoint ----
@Component
public class CustomInfoContributor implements InfoContributor {

    @Override
```

```
    public void contribute(Info.Builder builder) {
        builder
            .withDetail("app", Map.of(
                "name", "MySpringBootApp",
                "version", "1.0.0",
                "description", "Spring Boot Application with Monitoring"
            ))
            .withDetail("build", Map.of(
                "time", "2023-10-15T10:30:00Z",
                "artifact", "myapp",
                "group", "com.example"
            ))
            .withDetail("environment", Map.of(
                "activeProfiles", Arrays.toString(getActiveProfiles()),
                "javaVersion", System.getProperty("java.version")
            ));
    }

    private String[] getActiveProfiles() {
        return new String[]{"production"};
    }
}

// ---- Custom Health Check ----
@Component
public class DatabaseHealthIndicator implements HealthIndicator {

    @Autowired
    private DataSource dataSource;

    @Override
    public Health health() {
        try {
            // Check if database is accessible
            try (Connection conn = dataSource.getConnection()) {
                if (conn.isValid(5)) {
                    return Health.up()
                        .withDetail("database", "available")
                        .withDetail("url", dataSource.getConnection().getMetaData().getURL())
                        .build();
                }
            }
            return Health.down()
                .withDetail("database", "not responding")
                .build();
        } catch (Exception e) {
            return Health.down()
                .withDetail("database", "error")
                .withDetail("error", e.getMessage())
                .build();
        }
    }
}
```

```
// ---- Custom Health with System Status ----
@Component
public class SystemHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        Health.Builder status = Health.up();

        // Check memory
        long freeMemory = Runtime.getRuntime().freeMemory();
        long totalMemory = Runtime.getRuntime().totalMemory();
        long maxMemory = Runtime.getRuntime().maxMemory();
        double memoryUsedPercent = (1 - (double) freeMemory / totalMemory) * 100;

        if (memoryUsedPercent > 90) {
            status = Health.down();
        }

        status
            .withDetail("freeMemory", formatBytes(freeMemory))
            .withDetail("totalMemory", formatBytes(totalMemory))
            .withDetail("maxMemory", formatBytes(maxMemory))
            .withDetail("memoryUsedPercent", String.format("%.2f%%", memoryUsedPercent));

        // Check CPU
        if (getCpuLoad() > 0.9) {
            status = Health.down();
        }

        status.withDetail("cpuLoad", String.format("%.2f%%", getCpuLoad() * 100));

        return status.build();
    }

    private String formatBytes(long bytes) {
        return String.format("%.2f MB", bytes / (1024.0 * 1024.0));
    }

    private double getCpuLoad() {
        // Simulate CPU load check
        return 0.25;
    }
}
```

13.3 Custom Metrics

Hands-On Code:

java

CopyDownload

```
// ---- Custom Metrics with Micrometer ----
@Component
public class CustomMetricsService {

    @Autowired
```

```
private MeterRegistry meterRegistry;

// Create counters, timers, gauges, etc.

@PostConstruct
public void registerMetrics() {
    // Counter for API calls
    Counter apiCounter = Counter.builder("api.calls")
        .description("Number of API calls")
        .tag("type", "rest")
        .register(meterRegistry);

    // Timer for method execution
    Timer methodTimer = Timer.builder("method.execution.time")
        .description("Method execution time")
        .register(meterRegistry);

    // Gauge for active users
    AtomicInteger activeUsers = new AtomicInteger(0);
    Gauge.builder("active.users", activeUsers, AtomicInteger::get)
        .description("Number of active users")
        .register(meterRegistry);

    // Distribution summary for request size
    DistributionSummary requestSize = DistributionSummary.builder("request.size")
        .description("Request payload size")
        .baseUnit("bytes")
        .register(meterRegistry);
}

// Method with timing
public void processRequest() {
    // Record execution time
    Timer.Sample sample = Timer.start(meterRegistry);

    try {
        // Business logic
        Thread.sleep(100);

        // Increment counter
        Counter.builder("api.calls")
            .tag("status", "success")
            .register(meterRegistry)
            .increment();
    } catch (Exception e) {
        Counter.builder("api.calls")
            .tag("status", "error")
            .register(meterRegistry)
            .increment();
    } finally {
        sample.stop(Timer.builder("method.execution.time")
            .tag("method", "processRequest")
            .register(meterRegistry));
    }
```

```
    }

    // Track active users
    public void userLoggedIn() {
        Gauge.builder("active.users", new AtomicInteger(1))
            .register(meterRegistry);
    }
}

// ---- Controller with Metrics ----
@RestController
@RequestMapping("/api/metrics")
@Timed // Micrometer annotation for timing
public class MetricsController {

    @Autowired
    private MeterRegistry meterRegistry;

    private final Counter requestCounter;
    private final Timer requestTimer;

    public MetricsController(MeterRegistry meterRegistry) {
        this.meterRegistry = meterRegistry;
        this.requestCounter = Counter.builder("api.requests")
            .description("Total API requests")
            .register(meterRegistry);
        this.requestTimer = Timer.builder("api.request.duration")
            .description("API request duration")
            .register(meterRegistry);
    }

    @GetMapping("/test")
    @Timed(value = "test.endpoint", histogram = true, percentiles = {0.95, 0.99})
    public ResponseEntity<String> testMetric() {
        Timer.Sample sample = Timer.start(meterRegistry);
        requestCounter.increment();

        try {
            // Simulate processing
            Thread.sleep((long) (Math.random() * 500));

            // Record custom metrics
            requestTimer.record(Duration.ofMillis(100));

            return ResponseEntity.ok("Metric recorded");
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).build();
        } finally {
            sample.stop(requestTimer);
        }
    }
}
```

13.4 Prometheus/Grafana Integration

Hands-On Code:

xml

CopyDownloadRun

```
<!-- Add Prometheus dependency -->
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

yaml

CopyDownload

```
# application.yml - Prometheus Configuration
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  metrics:
    export:
      prometheus:
        enabled: true
        step: 10s
    tags:
      application: MySpringBootApp
      environment: production
  endpoint:
    prometheus:
      enabled: true
    metrics:
      enabled: true
```

java

CopyDownload

```
// ---- Prometheus Configuration ----
@Configuration
public class PrometheusConfig {

    @Bean
    public MeterRegistryCustomizer<MeterRegistry> metricsCommonTags() {
        return registry -> registry.config().commonTags(
            "application", "myapp",
            "environment", System.getProperty("spring.profiles.active", "dev")
        );
    }

    @Bean
    public MeterRegistry meterRegistry() {
        return new PrometheusMeterRegistry(PrometheusConfig.DEFAULT);
    }
}
```

yaml

CopyDownload

```
# prometheus.yml - Prometheus Configuration
global:
```

```
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'spring-boot-app'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['localhost:8080']
```

---

14. Error Handling in Spring Boot

14.1 Default Error Handling

Hands-On Code:

java

CopyDownload

```
// ---- Custom Error Controller ----
@RestControllerAdvice
public class CustomErrorController implements ErrorController {

    @Autowired
    private ErrorAttributes errorAttributes;

    @RequestMapping("/error")
    public ResponseEntity<ErrorResponse> handleError(HttpServletRequest request) {
        Map<String, Object> errorAttributes = this.errorAttributes.getErrorAttributes(
            request, ErrorAttributeOptions.defaults()
        );

        ErrorResponse response = ErrorResponse.builder()
            .timestamp((Date) errorAttributes.get("timestamp"))
            .status((Integer) errorAttributes.get("status"))
            .error((String) errorAttributes.get("error"))
            .message((String) errorAttributes.get("message"))
            .path((String) errorAttributes.get("path"))
            .build();

        return ResponseEntity
            .status((Integer) errorAttributes.get("status"))
            .body(response);
    }
}
```

14.2 Custom Error Pages

Hands-On Code:

html

CopyDownloadRun

```
<!-- src/main/resources/templates/error/404.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Page Not Found</title>
```

```
    <link rel="stylesheet" th:href="@{/css/error.css}" />
</head>
<body>
    <div class="error-container">
        <h1>404</h1>
        <h2>Page Not Found</h2>
        <p>The page you are looking for might have been removed or is temporarily unavailable.</p>
        <a th:href="@{/}">Return to Home</a>
    </div>
</body>
</html>
```

14.3 Global Exception Handling

Hands-On Code:

java

CopyDownload

```
// ---- Comprehensive Global Exception Handler ----
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger logger = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleResourceNotFound(ResourceNotFoundException ex) {
        logger.error("Resource not found: {}", ex.getMessage());
        ErrorResponse error = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.NOT_FOUND.value())
            .error("Not Found")
            .message(ex.getMessage())
            .build();
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(error);
    }

    @ExceptionHandler(ValidationException.class)
    public ResponseEntity<ValidationErrorResponse> handleValidation(ValidationException ex) {
        logger.warn("Validation error: {}", ex.getErrors());
        ValidationErrorResponse response = new ValidationErrorResponse();
        response.setTimestamp(LocalDateTime.now());
        response.setStatus(HttpStatus.BAD_REQUEST.value());
        response.setError("Validation Failed");
        response.setErrors(ex.getErrors());
        return ResponseEntity.badRequest().body(response);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGenericException(Exception ex) {
        logger.error("Unexpected error occurred", ex);
        ErrorResponse error = ErrorResponse.builder()
            .timestamp(LocalDateTime.now())
            .status(HttpStatus.INTERNAL_SERVER_ERROR.value())
            .error("Internal Server Error")
```

```
            .message("An unexpected error occurred. Please try again later.")
            .build();
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}
```

---

15. Developer Tools

15.1 Spring Boot DevTools

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-devtools</artifactId>
    <optional>true</optional>
</dependency>
```

yaml

CopyDownload

```
# application-dev.yml - DevTools Configuration
spring:
  devtools:
    restart:
      enabled: true
      additional-paths: src/main/java
      exclude: static/**,public/**
      poll-interval: 1000
      quiet-period: 400
      log-condition-evaluation-delta: false
    livereload:
      enabled: true
      port: 35729
```

15.2 Auto Restart

Hands-On Code:

java

CopyDownload

```
// ---- DevTools Configuration ----
@Configuration
@ConditionalOnDevtools
public class DevToolsConfig {

    private static final Logger logger = LoggerFactory.getLogger(DevToolsConfig.class);

    @Bean
    public DevToolsProperties devToolsProperties() {
        DevToolsProperties properties = new DevToolsProperties();
        properties.getRestart().setEnabled(true);
        properties.getRestart().setPollInterval(1000);
        return properties;
```

```
    }

    @EventListener
    public void onApplicationEvent(ApplicationReadyEvent event) {
        logger.info("DevTools is active - Auto-restart is enabled");
    }
}
```

15.3 Live Reload

Hands-On Code:

html

CopyDownloadRun

```
<!-- Static HTML with LiveReload support -->
<!DOCTYPE html>
<html>
<head>
    <title>LiveReload Demo</title>
    <!-- LiveReload script injected automatically when DevTools is active -->
</head>
<body>
    <h1>Hello, Spring Boot!</h1>
    <p>This page will reload automatically when changes are detected.</p>
</body>
</html>
```

---

16. Caching in Spring Boot

16.1 Using @Cacheable and @CacheEvict

Hands-On Code:

java

CopyDownload

```
// ---- Enable Caching ----
@SpringBootApplication
@EnableCaching
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

// ---- Service with Caching ----
@Service
public class ProductCachingService {

    private static final Logger logger = LoggerFactory.getLogger(ProductCachingService.class);

    @Autowired
    private ProductRepository productRepository;

    /**
     * @Cacheable - Caches the result of the method
```

```
 */
@Cacheable(value = "products", key = "#id")
public Product getProduct(Long id) {
    logger.info("Fetching product from database: {}", id);
    return productRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Product", id));
}

/**
 * @Cacheable with condition
 */
@Cacheable(value = "products", key = "#name", condition = "#name.length() > 3")
public List<Product> findProductsByName(String name) {
    logger.info("Searching products by name: {}", name);
    return productRepository.findByNameContaining(name);
}

/**
 * @Cacheable with unless condition
 */
@Cacheable(value = "expensiveProducts", key = "#minPrice",
        unless = "#result.isEmpty()")
public List<Product> getExpensiveProducts(Double minPrice) {
    logger.info("Finding expensive products above: {}", minPrice);
    return productRepository.findByPriceGreaterThanEqual(minPrice);
}

/**
 * @CacheEvict - Removes entries from cache
 */
@CacheEvict(value = "products", key = "#id")
public void deleteProduct(Long id) {
    logger.info("Deleting product: {}", id);
    productRepository.deleteById(id);
}

/**
 * @CacheEvict with multiple caches
 */
@CacheEvict(value = {"products", "expensiveProducts"}, allEntries = true)
public void clearAllCaches() {
    logger.info("Clearing all caches");
}

/**
 * @CachePut - Updates cache without interfering with method execution
 */
@CachePut(value = "products", key = "#result.id")
public Product updateProductStock(Long id, Integer newStock) {
    logger.info("Updating stock for product: {}", id);
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new ResourceNotFoundException("Product", id));
    product.setStockQuantity(newStock);
    return productRepository.save(product);
```

```
    }

    /**
     * Cache with TTL configuration
     */
    @Cacheable(value = "users", key = "#email", unless = "#result == null")
    public User findUserByEmail(String email) {
        logger.info("Finding user by email: {}", email);
        return userRepository.findByEmail(email)
            .orElse(null);
    }
}
```

16.2 Cache Providers

Hands-On Code:

yaml

CopyDownload

```
# application.yml - Cache Provider Configuration
spring:
  cache:
    # Use Caffeine provider
    type: caffeine
    caffeine:
      spec: maximumSize=500, expireAfterAccess=300s, recordStats

    # Use Redis provider (distributed caching)
    # type: redis
    # redis:
    #   time-to-live: 60000
    #   cache-null-values: false
    #   key-prefix: myapp:
```

java

CopyDownload

```
// ---- Caffeine Cache Configuration ----
@Configuration
public class CacheConfig {

    @Bean
    public CacheManager cacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager("products", "users",
"expensiveProducts");
        cacheManager.setCaffeine(Caffeine.newBuilder()
            .maximumSize(500)
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .expireAfterAccess(5, TimeUnit.MINUTES)
            .recordStats()
        );
        return cacheManager;
    }

    @Bean
    public CacheManager customCacheManager() {
        CaffeineCacheManager cacheManager = new CaffeineCacheManager();
```

```
        cacheManager.registerCustomCache("products",
            Caffeine.newBuilder()
                .maximumSize(1000)
                .expireAfterAccess(1, TimeUnit.HOURS)
                .build()
        );
        cacheManager.registerCustomCache("users",
            Caffeine.newBuilder()
                .maximumSize(2000)
                .expireAfterWrite(2, TimeUnit.HOURS)
                .build()
        );
        return cacheManager;
    }
}

// ---- Redis Cache Configuration ----
@Configuration
public class RedisCacheConfig {

    @Bean
    public RedisCacheManager cacheManager(RedisConnectionFactory connectionFactory) {
        RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues()
            .serializeValuesWith(
                RedisSerializationContext.SerializationPair.fromSerializer(
                    new Jackson2JsonRedisSerializer<>(Object.class)
                )
            );

        return RedisCacheManager.builder(connectionFactory)
            .cacheDefaults(config)
            .withCacheConfiguration("products",
                RedisCacheConfiguration.defaultCacheConfig()
                    .entryTtl(Duration.ofMinutes(5))
            )
            .withCacheConfiguration("users",
                RedisCacheConfiguration.defaultCacheConfig()
                    .entryTtl(Duration.ofHours(1))
            )
            .build();
    }
}
```

---

17. Scheduling & Asynchronous Execution

17.1 Scheduling Tasks (@Scheduled)

Hands-On Code:

java

CopyDownload

```
// ---- Enable Scheduling ----
```

```
@SpringBootApplication
@EnableScheduling
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

// ---- Scheduled Tasks ----
@Component
public class ScheduledTasks {

    private static final Logger logger = LoggerFactory.getLogger(ScheduledTasks.class);

    /**
     * Fixed rate - runs every 5 seconds
     */
    @Scheduled(fixedRate = 5000)
    public void reportCurrentTime() {
        logger.info("Scheduled task - Current time: {}", LocalDateTime.now());
    }

    /**
     * Fixed delay - runs 5 seconds after previous execution completes
     */
    @Scheduled(fixedDelay = 5000, initialDelay = 10000)
    public void processPendingOrders() {
        logger.info("Processing pending orders");
        // Business logic
    }

    /**
     * Cron expression - runs at 9:00 AM every day
     * Format: second minute hour day month day-of-week
     */
    @Scheduled(cron = "0 0 9 * * ?")
    public void generateDailyReport() {
        logger.info("Generating daily report");
        // Business logic
    }

    /**
     * Cron expression - runs every hour at minute 0
     */
    @Scheduled(cron = "0 0 * * * ?")
    public void hourlyTask() {
        logger.info("Running hourly task");
        // Business logic
    }

    /**
     * Cron expression - runs at 6:00 PM every Friday
     */
    @Scheduled(cron = "0 0 18 * * FRI")
```

```
    public void weeklyReport() {
        logger.info("Generating weekly report");
        // Business logic
    }

    /**
     * Cron expression - runs at midnight on the first day of every month
     */
    @Scheduled(cron = "0 0 0 1 * ?")
    public void monthlyCleanup() {
        logger.info("Running monthly cleanup task");
        // Business logic
    }

    /**
     * Scheduled task with error handling
     */
    @Scheduled(cron = "0 */5 * * * ?")
    public void scheduledTaskWithErrorHandling() {
        try {
            // Business logic that might throw exception
            logger.info("Running scheduled task with error handling");
        } catch (Exception e) {
            logger.error("Scheduled task failed", e);
            // Retry logic or notification
        }
    }
}
```

17.2 Asynchronous Execution (@Async)

Hands-On Code:

java

CopyDownload

```
// ---- Enable Async ----
@SpringBootApplication
@EnableAsync
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}

// ---- Async Configuration ----
@Configuration
public class AsyncConfig implements AsyncConfigurer {

    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(10);
        executor.setMaxPoolSize(50);
        executor.setQueueCapacity(100);
        executor.setThreadNamePrefix("async-");
```

```
        executor.initialize();
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return (throwable, method, params) -> {
            logger.error("Async method failed: {} - {}", method, throwable.getMessage());
        };
    }
}

// ---- Async Service ----
@Service
public class AsyncService {

    private static final Logger logger = LoggerFactory.getLogger(AsyncService.class);

    /**
     * Async method with void return
     */
    @Async
    public void sendEmailAsync(String to, String subject, String body) {
        logger.info("Sending email asynchronously to: {}", to);
        try {
            // Simulate email sending
            Thread.sleep(2000);
            logger.info("Email sent to: {}", to);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            logger.error("Email sending interrupted for: {}", to);
        }
    }

    /**
     * Async method with CompletableFuture return
     */
    @Async
    public CompletableFuture<User> findUserAsync(Long userId) {
        logger.info("Finding user asynchronously: {}", userId);
        try {
            // Simulate database operation
            Thread.sleep(1000);
            User user = userRepository.findById(userId)
                .orElseThrow(() -> new ResourceNotFoundException("User", userId));
            return CompletableFuture.completedFuture(user);
        } catch (Exception e) {
            logger.error("Failed to find user: {}", userId, e);
            return CompletableFuture.failedFuture(e);
        }
    }

    /**
     * Async method with custom executor
```

```
     */
    @Async("customExecutor")
    public void processWithCustomExecutor() {
        logger.info("Running with custom executor");
        // Business logic
    }
}

// ---- Controller using Async ----
@RestController
@RequestMapping("/api/async")
public class AsyncController {

    @Autowired
    private AsyncService asyncService;

    @PostMapping("/email")
    public ResponseEntity<String> sendEmail(@RequestBody EmailRequest request) {
        // Method returns immediately, email is sent asynchronously
        asyncService.sendEmailAsync(request.getTo(), request.getSubject(), request.getBody());
        return ResponseEntity.accepted().body("Email queued for sending");
    }

    @GetMapping("/user/{id}")
    public CompletableFuture<ResponseEntity<User>> getUserAsync(@PathVariable Long id) {
        return asyncService.findUserAsync(id)
            .thenApply(user -> ResponseEntity.ok(user))
            .exceptionally(ex -> ResponseEntity.notFound().build());
    }
}
```

---

18. Messaging in Spring Boot

18.1 RabbitMQ

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

yaml

CopyDownload

```
# application.yml - RabbitMQ Configuration
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest
    virtual-host: /
```

```
    connection-timeout: 5000
    template:
      exchange: my-exchange
      routing-key: my-routing-key
    listener:
      simple:
        concurrency: 3
        max-concurrency: 10
        prefetch: 1
        default-requeue-rejected: false
```

java

CopyDownload

```
// ---- RabbitMQ Configuration ----
@Configuration
public class RabbitMQConfig {

    @Value("${spring.rabbitmq.host}")
    private String host;

    @Value("${spring.rabbitmq.port}")
    private int port;

    @Value("${spring.rabbitmq.username}")
    private String username;

    @Value("${spring.rabbitmq.password}")
    private String password;

    @Bean
    public ConnectionFactory connectionFactory() {
        CachingConnectionFactory connectionFactory = new CachingConnectionFactory(host, port);
        connectionFactory.setUsername(username);
        connectionFactory.setPassword(password);
        return connectionFactory;
    }

    @Bean
    public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory) {
        RabbitTemplate rabbitTemplate = new RabbitTemplate(connectionFactory);
        rabbitTemplate.setMessageConverter(jackson2JsonMessageConverter());
        return rabbitTemplate;
    }

    @Bean
    public Jackson2JsonMessageConverter jackson2JsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
}

// ---- RabbitMQ Producer ----
@Service
public class RabbitMQProducer {

    private static final Logger logger = LoggerFactory.getLogger(RabbitMQProducer.class);
```

```
    @Autowired
    private RabbitTemplate rabbitTemplate;

    @Value("${app.rabbitmq.exchange}")
    private String exchange;

    @Value("${app.rabbitmq.routing-key}")
    private String routingKey;

    public void sendMessage(String message) {
        logger.info("Sending message: {}", message);
        rabbitTemplate.convertAndSend(exchange, routingKey, message);
        logger.info("Message sent successfully");
    }

    public void sendObjectMessage(Order order) {
        logger.info("Sending order object: {}", order);
        rabbitTemplate.convertAndSend(exchange, "order.queue", order);
    }

    @Async
    public void sendMessageAsync(String message) {
        try {
            rabbitTemplate.convertAndSend(exchange, routingKey, message);
            logger.info("Async message sent: {}", message);
        } catch (Exception e) {
            logger.error("Failed to send async message", e);
        }
    }
}

// ---- RabbitMQ Consumer ----
@Component
public class RabbitMQConsumer {

    private static final Logger logger = LoggerFactory.getLogger(RabbitMQConsumer.class);

    @RabbitListener(queues = "my-queue")
    public void receiveMessage(String message) {
        logger.info("Received message: {}", message);
        // Process message
    }

    @RabbitListener(queues = "order-queue")
    public void receiveOrder(Order order) {
        logger.info("Received order: {}", order);
        // Process order
    }

    @RabbitListener(queues = "my-queue", concurrency = "3-10")
    public void receiveMessageWithConcurrency(String message) {
        logger.info("Concurrent message processing: {}", message);
        // Process with multiple consumers
```

```
    }
}
```

18.2 Apache Kafka

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-kafka</artifactId>
</dependency>
```

yaml

CopyDownload

```
# application.yml - Kafka Configuration
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
    consumer:
      group-id: my-group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: "*"
        spring.json.type.mapping: order:com.example.Order
    listener:
      missing-topics-fatal: false
      concurrency: 3
    admin:
      auto-create: true
      fail-fast: false
```

java

CopyDownload

```
// ---- Kafka Configuration ----
@Configuration
public class KafkaConfig {

    @Bean
    public KafkaTemplate<String, Object> kafkaTemplate(ProducerFactory<String, Object>
producerFactory) {
        return new KafkaTemplate<>(producerFactory);
    }

    @Bean
    public ProducerFactory<String, Object> producerFactory() {
        Map<String, Object> configProps = new HashMap<>();
        configProps.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        configProps.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        configProps.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
        return new DefaultKafkaProducerFactory<>(configProps);
```

```
    }
}

// ---- Kafka Producer ----
@Service
public class KafkaProducer {

    private static final Logger logger = LoggerFactory.getLogger(KafkaProducer.class);

    @Autowired
    private KafkaTemplate<String, Object> kafkaTemplate;

    @Value("${app.kafka.topic}")
    private String topic;

    public void sendMessage(String message) {
        logger.info("Sending message to Kafka: {}", message);
        kafkaTemplate.send(topic, message);
        logger.info("Message sent to Kafka");
    }

    public void sendObject(Order order) {
        logger.info("Sending order to Kafka: {}", order);
        kafkaTemplate.send("order-topic", order);
    }

    public void sendWithPartitionKey(String key, String message) {
        logger.info("Sending message with key: {} - {}", key, message);
        kafkaTemplate.send(topic, key, message);
    }
}

// ---- Kafka Consumer ----
@Component
public class KafkaConsumer {

    private static final Logger logger = LoggerFactory.getLogger(KafkaConsumer.class);

    @KafkaListener(topics = "my-topic", groupId = "my-group")
    public void consume(String message) {
        logger.info("Consumed message from Kafka: {}", message);
        // Process message
    }

    @KafkaListener(topics = "order-topic", groupId = "order-group")
    public void consumeOrder(Order order) {
        logger.info("Consumed order from Kafka: {}", order);
        // Process order
    }

    @KafkaListener(topics = "my-topic", groupId = "my-group", concurrency = "3")
    public void consumeWithConcurrency(String message) {
        logger.info("Concurrent consumer: {}", message);
        // Process with multiple consumers
```

```
    }
}
```

18.3 JMS Integration

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-artemis</artifactId>
</dependency>
```

yaml

CopyDownload

```
# application.yml - JMS Configuration
spring:
  artemis:
    broker-url: tcp://localhost:61616
    user: admin
    password: admin
    mode: native
    embedded:
      enabled: false
  jms:
    pub-sub-domain: false
    template:
      default-destination: my-queue
      delivery-mode: non-persistent
```

java

CopyDownload

```
// ---- JMS Producer ----
@Service
public class JmsProducer {

    private static final Logger logger = LoggerFactory.getLogger(JmsProducer.class);

    @Autowired
    private JmsTemplate jmsTemplate;

    public void sendMessage(String message) {
        logger.info("Sending JMS message: {}", message);
        jmsTemplate.convertAndSend("my-queue", message);
        logger.info("JMS message sent");
    }

    public void sendObject(Order order) {
        logger.info("Sending JMS order: {}", order);
        jmsTemplate.convertAndSend("order-queue", order);
    }
}

// ---- JMS Consumer ----
@Component
```

```
public class JmsConsumer {

    private static final Logger logger = LoggerFactory.getLogger(JmsConsumer.class);

    @JmsListener(destination = "my-queue")
    public void receiveMessage(String message) {
        logger.info("Received JMS message: {}", message);
        // Process message
    }

    @JmsListener(destination = "order-queue")
    public void receiveOrder(Order order) {
        logger.info("Received JMS order: {}", order);
        // Process order
    }
}
```

---

19. File Upload & Download

19.1 Multipart File Upload

Hands-On Code:

yaml

CopyDownload

```
# application.yml - File Upload Configuration
spring:
  servlet:
    multipart:
      enabled: true
      max-file-size: 10MB
      max-request-size: 10MB
      location: /tmp/uploads
      file-size-threshold: 2MB
```

java

CopyDownload

```
// ---- File Upload Controller ----
@RestController
@RequestMapping("/api/files")
public class FileUploadController {

    private static final Logger logger = LoggerFactory.getLogger(FileUploadController.class);

    @Autowired
    private FileStorageService fileStorageService;

    /**
     * Single file upload
     */
    @PostMapping("/upload")
    public ResponseEntity<FileResponse> uploadFile(@RequestParam("file") MultipartFile file) {
        logger.info("Uploading file: {}", file.getOriginalFilename());
```

```
    if (file.isEmpty()) {
        throw new ValidationException("File is empty");
    }

    String fileName = fileStorageService.storeFile(file);
    FileResponse response = FileResponse.builder()
        .fileName(fileName)
        .originalName(file.getOriginalFilename())
        .fileSize(file.getSize())
        .message("File uploaded successfully")
        .build();

    return ResponseEntity.status(HttpStatus.CREATED).body(response);
}

/**
 * Multiple files upload
 */
@PostMapping("/upload/multiple")
public ResponseEntity<List<FileResponse>> uploadMultipleFiles(
        @RequestParam("files") MultipartFile[] files) {
    logger.info("Uploading {} files", files.length);

    List<FileResponse> responses = new ArrayList<>();
    for (MultipartFile file : files) {
        String fileName = fileStorageService.storeFile(file);
        responses.add(FileResponse.builder()
            .fileName(fileName)
            .originalName(file.getOriginalFilename())
            .fileSize(file.getSize())
            .message("File uploaded successfully")
            .build());
    }

    return ResponseEntity.status(HttpStatus.CREATED).body(responses);
}

/**
 * Upload with validation
 */
@PostMapping("/upload/validated")
public ResponseEntity<FileResponse> uploadValidatedFile(
        @RequestParam("file") MultipartFile file,
        @RequestParam String category) {

    // Validate file type
    String contentType = file.getContentType();
    if (!Arrays.asList("image/jpeg", "image/png", "application/pdf").contains(contentType)) {
        throw new ValidationException("Invalid file type. Only JPEG, PNG, and PDF are allowed");
    }

    // Validate file size
    if (file.getSize() > 5 * 1024 * 1024) { // 5MB
        throw new ValidationException("File size exceeds 5MB limit");
```

```
        }

        String fileName = fileStorageService.storeFile(file, category);
        return ResponseEntity.status(HttpStatus.CREATED).body(
            FileResponse.builder()
                .fileName(fileName)
                .originalName(file.getOriginalFilename())
                .fileSize(file.getSize())
                .message("File uploaded with category: " + category)
                .build()
        );
    }
}
```

19.2 File Storage and Download Handling

Hands-On Code:

java

CopyDownload

```
// ---- File Storage Service ----
@Service
public class FileStorageService {

    private static final Logger logger = LoggerFactory.getLogger(FileStorageService.class);

    @Value("${app.upload.dir:/tmp/uploads}")
    private String uploadDir;

    @PostConstruct
    public void init() {
        try {
            Files.createDirectories(Paths.get(uploadDir));
            logger.info("Upload directory created: {}", uploadDir);
        } catch (IOException e) {
            logger.error("Could not create upload directory", e);
            throw new RuntimeException("Could not initialize upload directory", e);
        }
    }

    public String storeFile(MultipartFile file) {
        return storeFile(file, "");
    }

    public String storeFile(MultipartFile file, String category) {
        try {
            // Sanitize filename
            String originalFileName = file.getOriginalFilename();
            String cleanFileName = sanitizeFileName(originalFileName);

            // Generate unique filename
            String extension = getFileExtension(cleanFileName);
            String uniqueFileName = UUID.randomUUID().toString() + extension;

            // Create directory structure
```

```
            Path uploadPath = Paths.get(uploadDir);
            if (!category.isEmpty()) {
                uploadPath = uploadPath.resolve(category);
            }
            Files.createDirectories(uploadPath);

            // Save file
            Path filePath = uploadPath.resolve(uniqueFileName);
            Files.write(filePath, file.getBytes());

            logger.info("File stored: {}", filePath);
            return uniqueFileName;

        } catch (IOException e) {
            logger.error("Failed to store file", e);
            throw new FileStorageException("Failed to store file", e);
        }
    }

    public Resource loadFile(String fileName) {
        return loadFile(fileName, "");
    }

    public Resource loadFile(String fileName, String category) {
        try {
            Path filePath = Paths.get(uploadDir);
            if (!category.isEmpty()) {
                filePath = filePath.resolve(category);
            }
            filePath = filePath.resolve(fileName);

            Resource resource = new UrlResource(filePath.toUri());
            if (!resource.exists() || !resource.isReadable()) {
                throw new FileNotFoundException("File not found: " + fileName);
            }

            return resource;

        } catch (MalformedURLException | FileNotFoundException e) {
            logger.error("File not found: {}", fileName, e);
            throw new FileNotFoundException("File not found: " + fileName);
        }
    }

    public boolean deleteFile(String fileName) {
        return deleteFile(fileName, "");
    }

    public boolean deleteFile(String fileName, String category) {
        try {
            Path filePath = Paths.get(uploadDir);
            if (!category.isEmpty()) {
                filePath = filePath.resolve(category);
            }
```

```
            filePath = filePath.resolve(fileName);

            return Files.deleteIfExists(filePath);

        } catch (IOException e) {
            logger.error("Failed to delete file: {}", fileName, e);
            return false;
        }
    }

    private String sanitizeFileName(String fileName) {
        if (fileName == null) {
            return "unknown";
        }
        // Remove path traversal characters
        return fileName.replaceAll("[^a-zA-Z0-9._-]", "");
    }

    private String getFileExtension(String fileName) {
        int lastDotIndex = fileName.lastIndexOf('.');
        if (lastDotIndex > 0 && lastDotIndex < fileName.length() - 1) {
            return fileName.substring(lastDotIndex);
        }
        return "";
    }
}

// ---- File Download Controller ----
@RestController
@RequestMapping("/api/files")
public class FileDownloadController {

    private static final Logger logger = LoggerFactory.getLogger(FileDownloadController.class);

    @Autowired
    private FileStorageService fileStorageService;

    /**
     * Download file
     */
    @GetMapping("/download/{fileName}")
    public ResponseEntity<Resource> downloadFile(@PathVariable String fileName) {
        logger.info("Downloading file: {}", fileName);

        Resource resource = fileStorageService.loadFile(fileName);
        String contentType = determineContentType(fileName);

        return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(contentType))
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "attachment; filename=\"" + fileName + "\"")
            .header(HttpHeaders.CONTENT_LENGTH,
                String.valueOf(resource.contentLength()))
            .body(resource);
```

```
    }

    /**
     * Download file with inline display
     */
    @GetMapping("/view/{fileName}")
    public ResponseEntity<Resource> viewFile(@PathVariable String fileName) {
        logger.info("Viewing file: {}", fileName);

        Resource resource = fileStorageService.loadFile(fileName);
        String contentType = determineContentType(fileName);

        return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(contentType))
            .header(HttpHeaders.CONTENT_DISPOSITION,
                "inline; filename=\"" + fileName + "\"")
            .body(resource);
    }

    private String determineContentType(String fileName) {
        String extension = getFileExtension(fileName).toLowerCase();
        switch (extension) {
            case ".pdf":
                return "application/pdf";
            case ".jpg":
            case ".jpeg":
                return "image/jpeg";
            case ".png":
                return "image/png";
            case ".txt":
                return "text/plain";
            case ".doc":
                return "application/msword";
            case ".docx":
                return "application/vnd.openxmlformats-officedocument.wordprocessingml.document";
            default:
                return "application/octet-stream";
        }
    }

    private String getFileExtension(String fileName) {
        int lastDotIndex = fileName.lastIndexOf('.');
        if (lastDotIndex > 0 && lastDotIndex < fileName.length() - 1) {
            return fileName.substring(lastDotIndex);
        }
        return "";
    }
}
```

---

20. Frontend Integration

20.1 Serving Static Files

Hands-On Code:

java

CopyDownload

```
// ---- Static Resource Configuration ----
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        // Custom static resource locations
        registry.addResourceHandler("/static/**")
            .addResourceLocations("classpath:/static/")
            .setCachePeriod(3600);

        registry.addResourceHandler("/public/**")
            .addResourceLocations("classpath:/public/")
            .setCachePeriod(3600);

        registry.addResourceHandler("/images/**")
            .addResourceLocations("classpath:/images/")
            .setCachePeriod(86400);

        // External directory for uploads
        registry.addResourceHandler("/uploads/**")
            .addResourceLocations("file:uploads/");
    }

    @Override
    public void addViewControllers(ViewControllerRegistry registry) {
        registry.addViewController("/").setViewName("index");
        registry.addViewController("/home").setViewName("home");
        registry.addViewController("/login").setViewName("login");
        registry.addViewController("/error/**").setViewName("error");
    }
}
```

20.2 Using Thymeleaf

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

html

CopyDownloadRun

```
<!-- src/main/resources/templates/index.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Spring Boot App</title>
```

```
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
    <header>
        <h1 th:text="${appName}">Spring Boot Application</h1>
    </header>

    <main>
        <div th:if="${user != null}">
            <h2>Welcome, <span th:text="${user.firstName}">User</span>!</h2>
        </div>

        <div th:unless="${user != null}">
            <a href="/login">Login</a>
        </div>

        <!-- Iteration -->
        <ul>
            <li th:each="product : ${products}" th:text="${product.name}">
                Product Name
            </li>
        </ul>

        <!-- Conditional rendering -->
        <div th:switch="${userRole}">
            <p th:case="'ADMIN'">Admin Dashboard</p>
            <p th:case="'USER'">User Dashboard</p>
            <p th:case="*">Guest Dashboard</p>
        </div>
    </main>

    <footer>
        <p th:text="'© ' + ${currentYear} + ' My App'">© 2023 My App</p>
    </footer>
</body>
</html>
```

java

CopyDownload

```
// ---- Thymeleaf Controller ----
@Controller
public class ThymeleafController {

    @GetMapping("/")
    public String index(Model model) {
        model.addAttribute("appName", "Spring Boot App");
        model.addAttribute("currentYear", LocalDate.now().getYear());

        // Get current user if logged in
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();
        if (auth != null && auth.isAuthenticated() && !(auth instanceof
AnonymousAuthenticationToken)) {
            model.addAttribute("user", auth.getPrincipal());
            model.addAttribute("userRole", getRole(auth));
        }
```

```
        // Sample products
        model.addAttribute("products", Arrays.asList(
            new Product("Laptop", 999.99),
            new Product("Monitor", 299.99),
            new Product("Keyboard", 49.99)
        ));

        return "index";
    }

    private String getRole(Authentication auth) {
        return auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .findFirst()
            .orElse("GUEST");
    }
}
```

20.3 Enabling CORS

Hands-On Code:

java

CopyDownload

```
// ---- Global CORS Configuration ----
@Configuration
public class CorsConfig {

    @Bean
    public WebMvcConfigurer corsConfigurer() {
        return new WebMvcConfigurer() {
            @Override
            public void addCorsMappings(CorsRegistry registry) {
                registry.addMapping("/api/**")
                    .allowedOrigins("http://localhost:3000", "http://localhost:4200", "https://myapp.com")
                    .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS", "PATCH")
                    .allowedHeaders("*")
                    .exposedHeaders("Authorization", "X-Requested-With")
                    .allowCredentials(true)
                    .maxAge(3600);

                registry.addMapping("/api/auth/**")
                    .allowedOrigins("http://localhost:3000")
                    .allowedMethods("POST", "OPTIONS")
                    .allowedHeaders("*")
                    .allowCredentials(true)
                    .maxAge(3600);
            }
        };
    }
}

// ---- Controller-level CORS ----
@RestController
```

```
@RequestMapping("/api/products")
@CrossOrigin(origins = "http://localhost:3000")
public class ProductController {

    // All methods in this controller will allow CORS from localhost:3000
}

// ---- Method-level CORS ----
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping
    @CrossOrigin(origins = {"http://localhost:3000", "http://localhost:4200"})
    public List<User> getUsers() {
        // Only this method allows CORS from these origins
    }

    @PostMapping
    @CrossOrigin(origins = "http://localhost:3000", maxAge = 3600)
    public User createUser(@RequestBody User user) {
        // This method has specific CORS configuration
    }
}
```

---

21. Build & Deployment

21.1 Building JAR/WAR

Hands-On Code:

xml

CopyDownloadRun

```
<!-- pom.xml - JAR Packaging -->
<packaging>jar</packaging>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <executable>true</executable>
                <mainClass>com.example.Application</mainClass>
            </configuration>
        </plugin>
    </plugins>
</build>
```

xml

CopyDownloadRun

```
<!-- pom.xml - WAR Packaging -->
<packaging>war</packaging>
```

```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <mainClass>com.example.Application</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

java

CopyDownload

```
// ---- WAR Deployment Configuration ----
@SpringBootApplication
public class Application extends SpringBootServletInitializer {

  @Override
  protected SpringApplicationBuilder configure(SpringApplicationBuilder builder) {
    return builder.sources(Application.class);
  }

  public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
  }
}
```

21.2 Dockerizing Spring Boot Application

Hands-On Code:

dockerfile

CopyDownload

```
# Dockerfile
FROM openjdk:17-jdk-slim AS builder
WORKDIR /app
COPY . .
RUN ./mvnw clean package -DskipTests

FROM openjdk:17-jdk-slim
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
COPY --from=builder /app/src/main/resources/application-prod.yml application.yml

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

yaml

CopyDownload

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build: .
```

```
    ports:
      - "8080:8080"
    environment:
      SPRING_PROFILES_ACTIVE: prod
      SPRING_DATASOURCE_URL: jdbc:mysql://db:3306/mydb
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: password
    depends_on:
      - db
      - redis

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: password
      MYSQL_DATABASE: mydb
    ports:
      - "3306:3306"

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

  rabbitmq:
    image: rabbitmq:management
    ports:
      - "5672:5672"
      - "15672:15672"
```

21.3 Deploying to Cloud

Hands-On Code:

yaml

CopyDownload

```
# AWS Elastic Beanstalk - application.yml
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: update
```

bash

CopyDownload

```
# AWS Deployment Commands
# Install EB CLI
pip install awsebcli

# Initialize EB application
eb init -p java-17 my-app

# Create environment
```

```
eb create my-app-env

# Deploy application
eb deploy

# View application
eb open
```

yaml

CopyDownload

```
# Google Cloud Platform - app.yaml
runtime: java17
env: standard
instance_class: F1
automatic_scaling:
  min_instances: 1
  max_instances: 10

env_variables:
  SPRING_PROFILES_ACTIVE: gcp
  SPRING_DATASOURCE_URL: jdbc:postgresql://${DB_HOST}:5432/${DB_NAME}
  SPRING_DATASOURCE_USERNAME: ${DB_USER}
  SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
```

bash

CopyDownload

```
# GCP Deployment Commands
# Install Google Cloud SDK
gcloud init

# Deploy to App Engine
gcloud app deploy

# View application
gcloud app browse
```

21.4 CI/CD Integration

Hands-On Code:

yaml

CopyDownload

```
# .github/workflows/build.yml - GitHub Actions
name: Build and Deploy Spring Boot App

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
```

```
    - uses: actions/checkout@v3

    - name: Set up JDK 17
      uses: actions/setup-java@v3
      with:
        java-version: '17'
        distribution: 'temurin'

    - name: Cache Maven dependencies
      uses: actions/cache@v3
      with:
        path: ~/.m2
        key: ${{ runner.os }}-m2-${{ hashFiles('**/pom.xml') }}
        restore-keys: ${{ runner.os }}-m2

    - name: Build with Maven
      run: mvn clean package -DskipTests

    - name: Run tests
      run: mvn test

    - name: Upload test results
      uses: actions/upload-artifact@v3
      if: always()
      with:
        name: test-results
        path: target/surefire-reports/

    - name: Build Docker image
      run: docker build -t myapp:${{ github.sha }} .

    - name: Push to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}

    - name: Push image
      run: |
        docker tag myapp:${{ github.sha }} ${{ secrets.DOCKER_USERNAME }}/myapp:
${{ github.sha }}
        docker tag myapp:${{ github.sha }} ${{ secrets.DOCKER_USERNAME }}/myapp:latest
        docker push ${{ secrets.DOCKER_USERNAME }}/myapp:${{ github.sha }}
        docker push ${{ secrets.DOCKER_USERNAME }}/myapp:latest

  deploy:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
    - name: Deploy to AWS
      run: |
        # Use AWS CLI to update Elastic Beanstalk
```

```
    # aws elasticbeanstalk update-environment ...
```

groovy

CopyDownload

```
// Jenkinsfile - Jenkins Pipeline
pipeline {
  agent any

  environment {
    DOCKER_REGISTRY = 'myregistry.azurecr.io'
    DOCKER_IMAGE = 'myapp'
    DOCKER_TAG = "${env.BUILD_NUMBER}"
  }

  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }

    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests'
      }
    }

    stage('Test') {
      steps {
        sh 'mvn test'
      }
      post {
        always {
          junit 'target/surefire-reports/*.xml'
        }
      }
    }

    stage('Docker Build') {
      steps {
        script {
          docker.build("${DOCKER_IMAGE}:${DOCKER_TAG}")
          docker.tag("${DOCKER_IMAGE}:${DOCKER_TAG}",
                     "${DOCKER_REGISTRY}/${DOCKER_IMAGE}:${DOCKER_TAG}")
          docker.tag("${DOCKER_IMAGE}:${DOCKER_TAG}",
                     "${DOCKER_REGISTRY}/${DOCKER_IMAGE}:latest")
        }
      }
    }

    stage('Docker Push') {
      steps {
        script {
          docker.withRegistry("https://${DOCKER_REGISTRY}") {
            docker.image("${DOCKER_REGISTRY}/${DOCKER_IMAGE}:
```

```
${DOCKER_TAG}").push()
                    docker.image("${DOCKER_REGISTRY}/${DOCKER_IMAGE}:latest").push()
                }
            }
        }
    }

    stage('Deploy') {
        when {
            branch 'main'
        }
        steps {
            script {
                sh 'kubectl set image deployment/myapp myapp=${DOCKER_REGISTRY}/
${DOCKER_IMAGE}:${DOCKER_TAG}'
                sh 'kubectl rollout status deployment/myapp'
            }
        }
    }
  }
}
```

---

22. Advanced Topics

22.1 Microservices with Spring Boot

22.1.1 Spring Cloud

Hands-On Code:

xml

CopyDownloadRun

```
<!-- Spring Cloud Dependencies -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2022.0.4</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>

<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter</artifactId>
  </dependency>
</dependencies>
```

22.1.2 Eureka Service Discovery

Hands-On Code:

xml

CopyDownloadRun

```
<!-- Eureka Server Dependency -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

java

CopyDownload

```
// ---- Eureka Server ----
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

yaml

CopyDownload

```
# Eureka Server - application.yml
server:
  port: 8761

spring:
  application:
    name: eureka-server

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
  server:
    enable-self-preservation: true
    renewal-percent-threshold: 0.85
```

xml

CopyDownloadRun

```
<!-- Eureka Client Dependency -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

java

CopyDownload

```
// ---- Eureka Client ----
@SpringBootApplication
@EnableDiscoveryClient
public class ProductServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProductServiceApplication.class, args);
    }
}
```

yaml

CopyDownload

```
# Eureka Client - application.yml
spring:
  application:
    name: product-service

server:
  port: 8081

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 30
    lease-expiration-duration-in-seconds: 90
```

22.1.3 Spring Cloud Gateway

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

yaml

CopyDownload

```
# Gateway Configuration
server:
  port: 8080

spring:
  application:
    name: api-gateway

  cloud:
    gateway:
      routes:
        - id: product-service
          uri: lb://product-service
          predicates:
            - Path=/api/products/**
            - Method=GET,POST,PUT,DELETE
          filters:
            - name: CircuitBreaker
              args:
                name: productService
                fallbackUri: forward:/fallback/products
            - name: RequestRateLimiter
              args:
                redis-rate-limiter.replenishRate: 10
                redis-rate-limiter.burstCapacity: 20
```

```
      - id: order-service
        uri: lb://order-service
        predicates:
          - Path=/api/orders/**
          - Method=GET,POST,PUT,DELETE
        filters:
          - name: CircuitBreaker
            args:
              name: orderService
              fallbackUri: forward:/fallback/orders

      - id: auth-service
        uri: lb://auth-service
        predicates:
          - Path=/api/auth/**
        filters:
          - name: CircuitBreaker
            args:
              name: authService
              fallbackUri: forward:/fallback/auth
```

22.1.4 Circuit Breaker (Resilience4j)

Hands-On Code:

xml

CopyDownloadRun

```
<dependency>
   <groupId>org.springframework.cloud</groupId>
   <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

java

CopyDownload

```
// ---- Circuit Breaker Configuration ----
@Configuration
public class Resilience4jConfig {

   @Bean
   public CircuitBreakerConfig circuitBreakerConfig() {
      return CircuitBreakerConfig.custom()
         .slidingWindowSize(10)
         .failureRateThreshold(50)
         .waitDurationInOpenState(Duration.ofSeconds(30))
         .permittedNumberOfCallsInHalfOpenState(5)
         .build();
   }

   @Bean
   public CircuitBreakerRegistry circuitBreakerRegistry() {
      return CircuitBreakerRegistry.of(circuitBreakerConfig());
   }
}

// ---- Circuit Breaker Usage ----
```

```
@Service
public class ProductServiceClient {

    private static final Logger logger = LoggerFactory.getLogger(ProductServiceClient.class);

    @Autowired
    private RestTemplate restTemplate;

    @Autowired
    private CircuitBreakerFactory circuitBreakerFactory;

    public List<Product> getProducts() {
        CircuitBreaker circuitBreaker = circuitBreakerFactory.create("productService");

        return circuitBreaker.run(
            () -> {
                logger.info("Calling product service");
                return restTemplate.getForObject("http://product-service/api/products", List.class);
            },
            throwable -> {
                logger.error("Product service call failed", throwable);
                return getFallbackProducts();
            }
        );
    }

    private List<Product> getFallbackProducts() {
        return Arrays.asList(
            new Product(1L, "Fallback Product 1", 0.0),
            new Product(2L, "Fallback Product 2", 0.0)
        );
    }
}

// ---- Resilience4j with Annotations ----
@Service
public class PaymentService {

    private static final Logger logger = LoggerFactory.getLogger(PaymentService.class);

    @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
    @TimeLimiter(name = "paymentService", fallbackMethod = "timeoutFallback")
    @Retry(name = "paymentService", fallbackMethod = "retryFallback")
    public CompletableFuture<PaymentResult> processPayment(PaymentRequest request) {
        return CompletableFuture.supplyAsync(() -> {
            logger.info("Processing payment: {}", request);
            // Simulate payment processing
            return new PaymentResult(true, "TXN123");
        });
    }

    public CompletableFuture<PaymentResult> fallbackPayment(PaymentRequest request,
Throwable t) {
        logger.error("Payment failed, using fallback: {}", request, t);
```

```
        return CompletableFuture.completedFuture(
            new PaymentResult(false, "Payment failed, please try again")
        );
    }

    public CompletableFuture<PaymentResult> timeoutFallback(PaymentRequest request,
Throwable t) {
        logger.warn("Payment timeout, using fallback: {}", request);
        return CompletableFuture.completedFuture(
            new PaymentResult(false, "Payment timeout")
        );
    }

    public CompletableFuture<PaymentResult> retryFallback(PaymentRequest request, Throwable
t) {
        logger.warn("Payment retry failed: {}", request, t);
        return CompletableFuture.completedFuture(
            new PaymentResult(false, "Payment failed after retries")
        );
    }
}
```

22.1.5 Config Server

Hands-On Code:

xml

CopyDownloadRun

```
<!-- Config Server Dependency -->
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

java

CopyDownload

```
// ---- Config Server ----
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

yaml

CopyDownload

```
# Config Server - application.yml
server:
  port: 8888

spring:
  application:
    name: config-server
  cloud:
    config:
```

```
    server:
      git:
        uri: https://github.com/your-repo/config-repo
        default-label: main
        search-paths: '{application}'
        clone-on-start: true
        basedir: /tmp/config-repo
      overrides:
        spring.security.user.password: ${SECURITY_PASSWORD:default-password}
```

yaml

CopyDownload

```
# Config Client - bootstrap.yml
spring:
  application:
    name: product-service
  cloud:
    config:
      uri: http://localhost:8888
      fail-fast: true
      retry:
        initial-interval: 1000
        max-attempts: 6
        max-interval: 2000
        multiplier: 1.1
  profiles:
    active: dev
```

22.2 ApplicationEventPublisher & Event Listeners

Hands-On Code:

java

CopyDownload

```
// ---- Custom Event ----
public class UserRegistrationEvent extends ApplicationEvent {
    private final User user;
    private final String email;

    public UserRegistrationEvent(Object source, User user) {
        super(source);
        this.user = user;
        this.email = user.getEmail();
    }

    public User getUser() { return user; }
    public String getEmail() { return email; }
}

// ---- Event Publisher ----
@Service
public class UserService {

    @Autowired
    private ApplicationEventPublisher eventPublisher;
```

```
    public User registerUser(UserRegistrationDto dto) {
        // Register user logic
        User user = new User();
        user.setEmail(dto.getEmail());
        user.setFirstName(dto.getFirstName());
        user.setLastName(dto.getLastName());
        user.setPassword(passwordEncoder.encode(dto.getPassword()));
        user.setActive(true);
        user = userRepository.save(user);

        // Publish event
        eventPublisher.publishEvent(new UserRegistrationEvent(this, user));

        return user;
    }
}

// ---- Event Listeners ----
@Component
public class UserEventListener {

    private static final Logger logger = LoggerFactory.getLogger(UserEventListener.class);

    @EventListener
    public void handleUserRegistration(UserRegistrationEvent event) {
        logger.info("User registered: {}", event.getEmail());
        sendWelcomeEmail(event.getEmail());
        createUserDashboard(event.getUser());
        logUserRegistration(event.getUser());
    }

    @EventListener
    @Async
    public void sendWelcomeEmail(String email) {
        try {
            Thread.sleep(1000);
            logger.info("Welcome email sent to: {}", email);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }

    @EventListener
    @Order(1)
    public void createUserDashboard(User user) {
        logger.info("Creating dashboard for user: {}", user.getId());
        // Create user dashboard
    }

    @EventListener
    @Order(2)
    public void logUserRegistration(User user) {
        logger.info("Logging user registration: {}", user.getId());
        // Log registration
```

```
    }
}

// ---- Conditional Event Listener ----
@Component
public class ConditionalEventListener {

    @EventListener(condition = "#event.user.roles.contains('ADMIN')")
    public void handleAdminRegistration(UserRegistrationEvent event) {
        logger.info("Admin user registered: {}", event.getEmail());
        // Additional admin-specific logic
    }
}
```

22.3 Creating Custom Starters

Hands-On Code:

xml

CopyDownloadRun

```
<!-- My Custom Starter - pom.xml -->
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <groupId>com.example</groupId>
    <artifactId>my-spring-boot-starter</artifactId>
    <version>1.0.0</version>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter</artifactId>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-autoconfigure</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-configuration-processor</artifactId>
            <optional>true</optional>
        </dependency>
    </dependencies>
</project>
```

java

CopyDownload

```
// ---- Custom Starter Properties ----
@ConfigurationProperties(prefix = "my.starter")
public class MyStarterProperties {

    private boolean enabled = true;
    private String defaultMessage = "Hello from custom starter!";
    private int timeout = 5000;
    private List<String> features = new ArrayList<>();

    // Getters and Setters
```

```
}

// ---- Auto Configuration ----
@Configuration
@EnableConfigurationProperties(MyStarterProperties.class)
@ConditionalOnProperty(prefix = "my.starter", name = "enabled", havingValue = "true",
matchIfMissing = true)
public class MyStarterAutoConfiguration {

   private static final Logger logger =
LoggerFactory.getLogger(MyStarterAutoConfiguration.class);

   @Bean
   @ConditionalOnMissingBean
   public MyService myService(MyStarterProperties properties) {
      logger.info("Creating MyService with properties: {}", properties);
      return new MyService(properties);
   }

   @Bean
   @ConditionalOnWebApplication
   public MyWebConfig myWebConfig() {
      return new MyWebConfig();
   }

   @Bean
   @ConditionalOnClass(name = "org.springframework.data.jpa.repository.JpaRepository")
   public MyJpaConfig myJpaConfig() {
      return new MyJpaConfig();
   }
}

// ---- Custom Service ----
public class MyService {
   private final MyStarterProperties properties;

   public MyService(MyStarterProperties properties) {
      this.properties = properties;
   }

   public String getMessage() {
      return properties.getDefaultMessage();
   }

   public int getTimeout() {
      return properties.getTimeout();
   }

   public List<String> getFeatures() {
      return properties.getFeatures();
   }
}

// ---- Web Configuration ----
```

```
public class MyWebConfig {

    @Bean
    public FilterRegistrationBean<MyFilter> myFilter() {
        FilterRegistrationBean<MyFilter> registration = new FilterRegistrationBean<>();
        registration.setFilter(new MyFilter());
        registration.addUrlPatterns("/api/*");
        return registration;
    }
}
```

properties

CopyDownload

```
# META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.example.autoconfigure.MyStarterAutoConfiguration
```

xml

CopyDownloadRun

```
<!-- Usage in another project -->
<dependency>
    <groupId>com.example</groupId>
    <artifactId>my-spring-boot-starter</artifactId>
    <version>1.0.0</version>
</dependency>
```

yaml

CopyDownload

```
# Usage Configuration
my:
  starter:
    enabled: true
    default-message: "Welcome to my custom starter!"
    timeout: 10000
    features:
      - feature1
      - feature2
      - feature3
```

This comprehensive guide covers all the essential concepts, features, and hands-on code examples for Spring Boot development. Each section includes practical code that can be directly used in real-world applications, with detailed comments explaining the purpose and usage of each component.